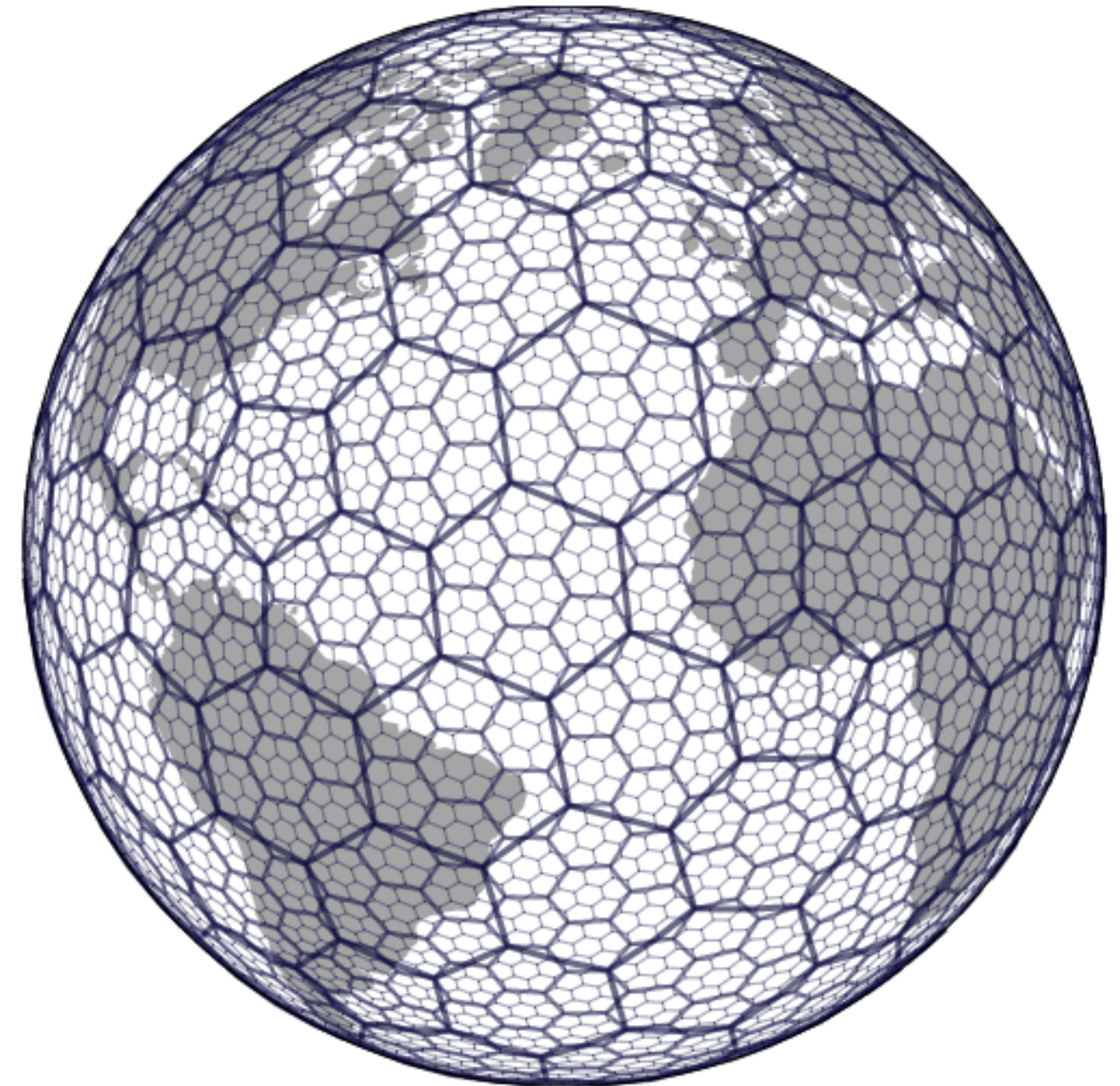


Lecture 4: Big Spatial Data

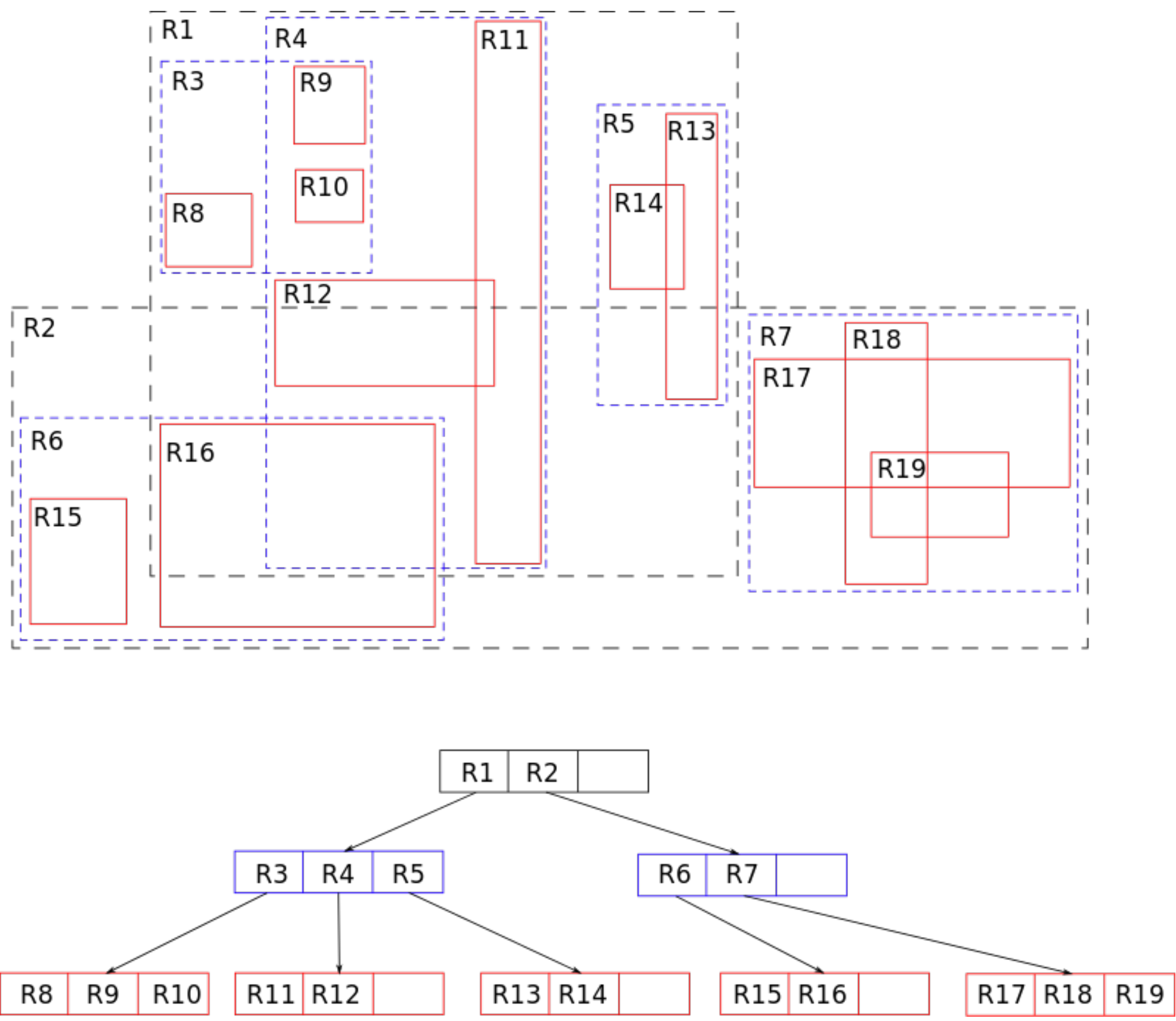
Instructor: Michael Szell

Feb 18, 2026



Today we will learn about....

Spatial indexes



Tools for large geospatial data



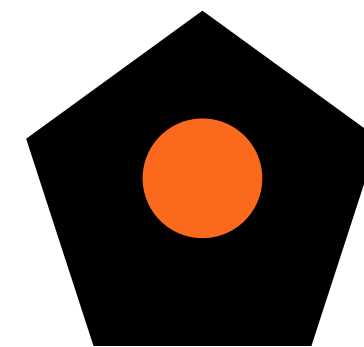
After the course, you should be able to...

- **Explain** what is a spatial index
- **Compare** different spatial indexing methods
- **Integrate** large geospatial data tools in your workflow

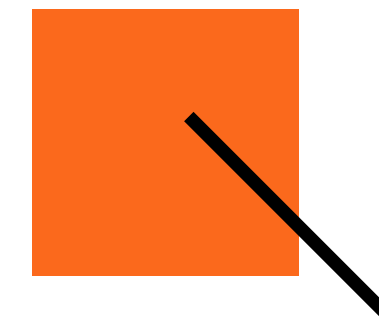
GeoPandas/Shapely allow us to make spatial queries

Spatial queries

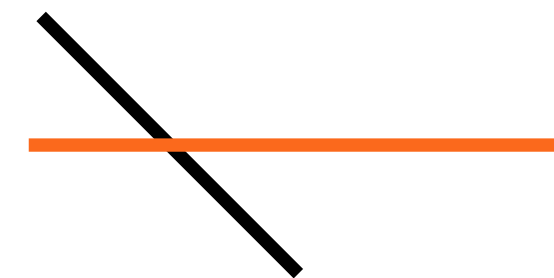
```
point.within(poly)
```



```
line.intersects(poly)
```



```
line1.crosses(line2)
```



```
poly1.touches(poly2)  
etc....
```

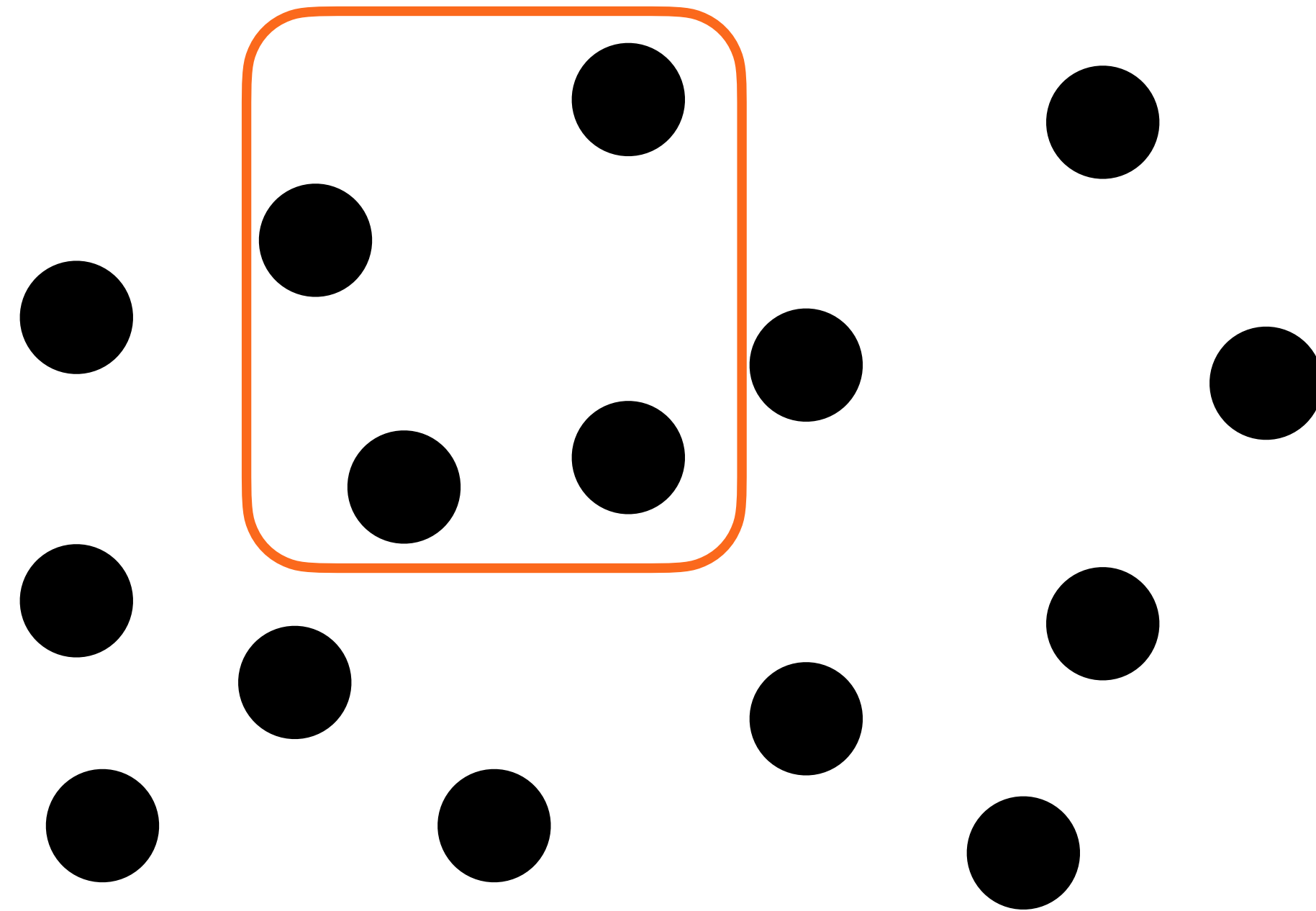


Spatial is slow

Avoid spatial queries and operations if you can

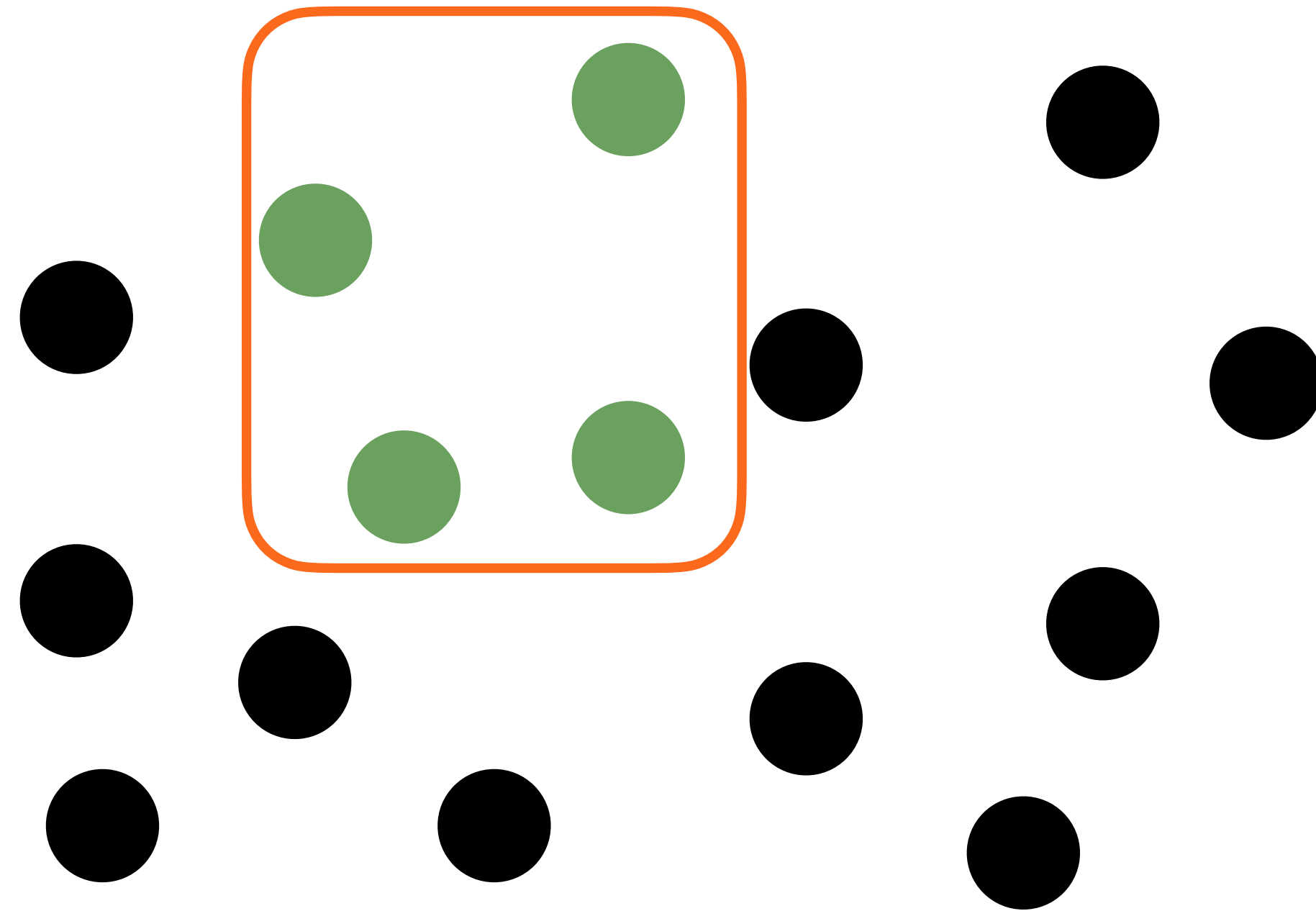
Naive spatial queries would require a lot of test...

Which points are inside the polygon ?



Naive spatial queries would require a lot of test...

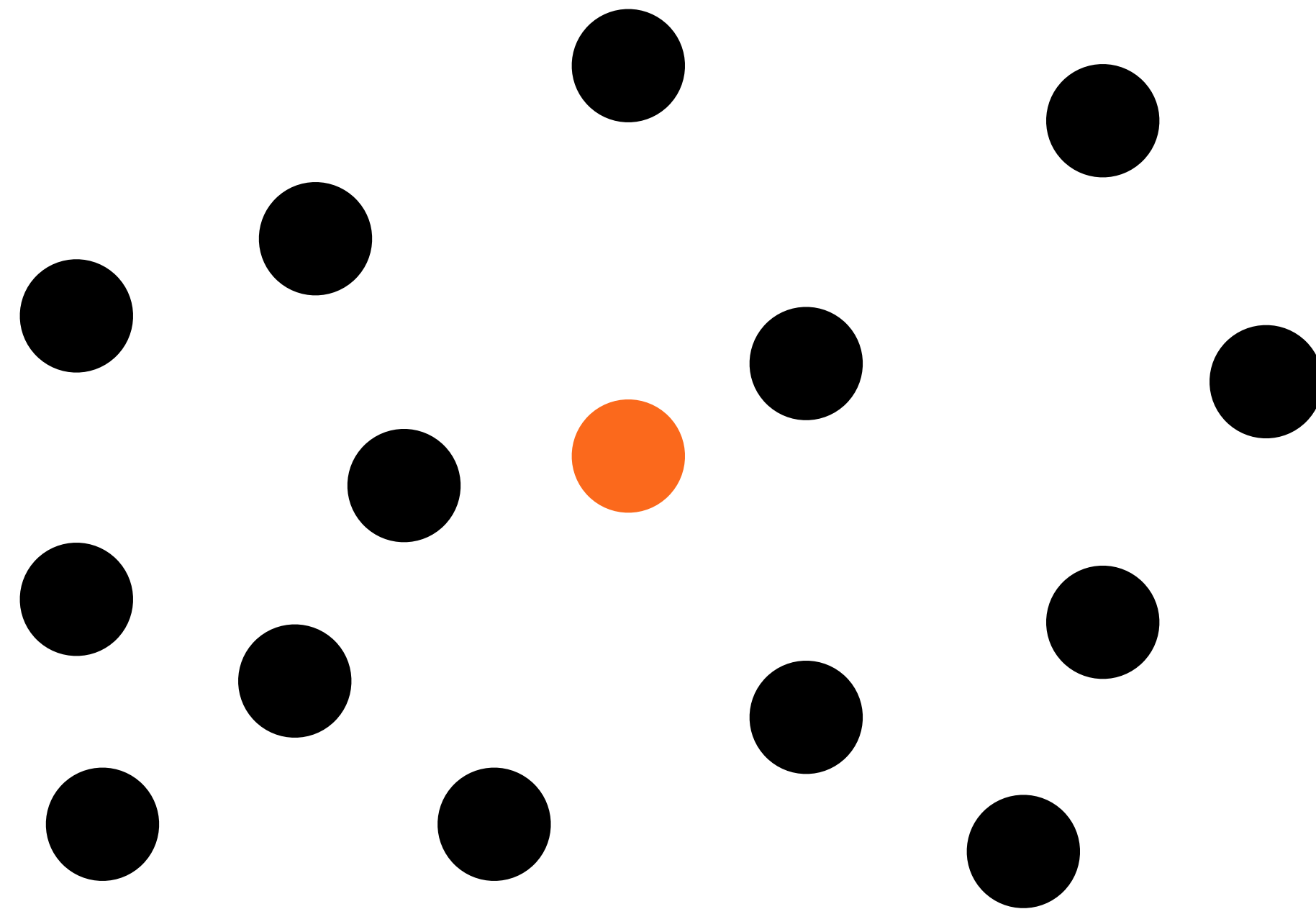
We don't want to test if every point is within the polygon



```
for point in point_list:  
    point.within(polygon)
```

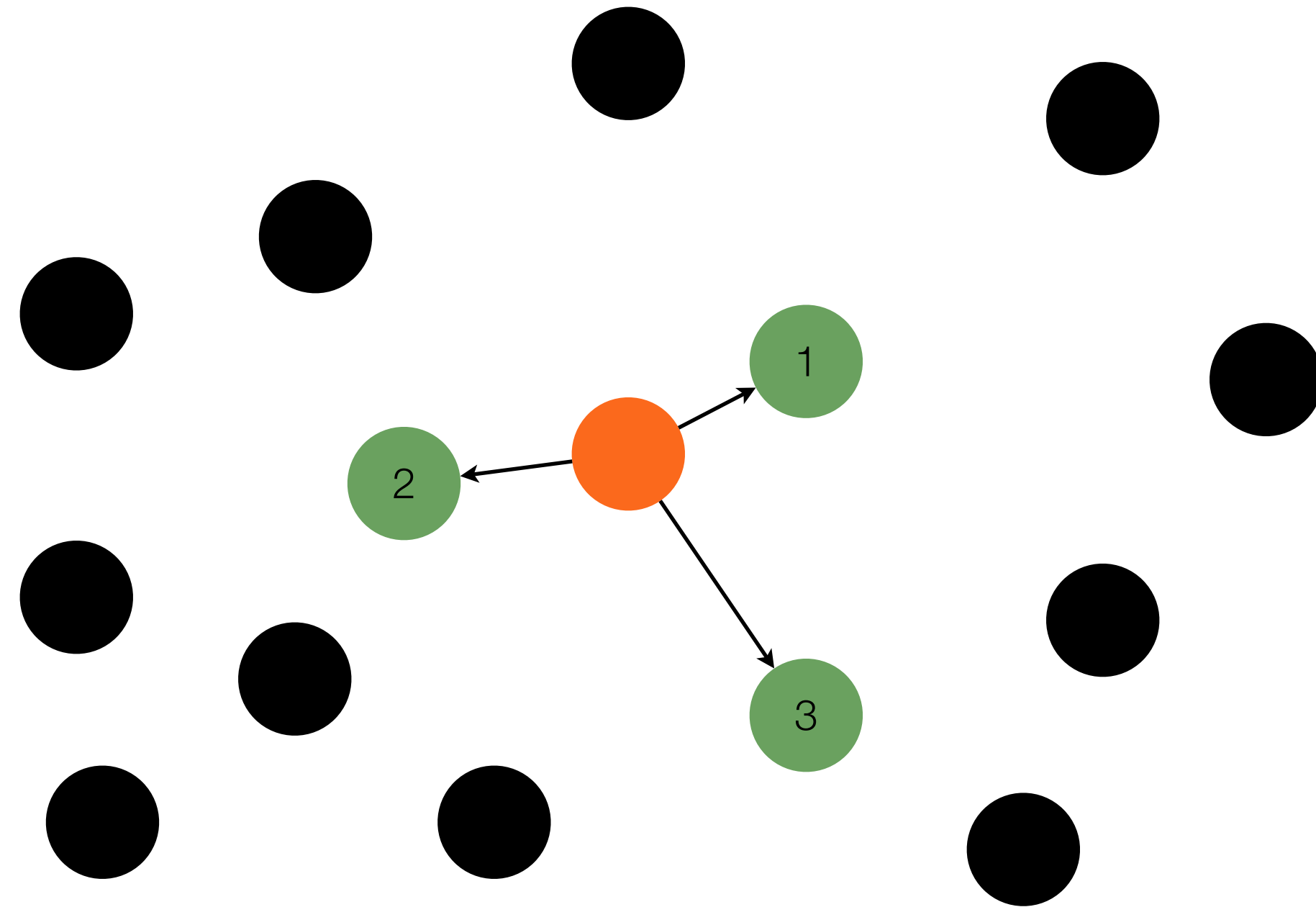
Naive spatial queries would require a lot of test...

Who are the 3 closest neighbors to the orange point ?



Naive spatial queries would require a lot of test...

We don't want to compute the distance to all other points



```
for point in point_black:  
    distance(point_orange, point)
```

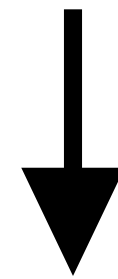
How would you solve
these problems?

Since spatial is slow...

```
polygdf.overlay(polygdf2, how="intersection")
```

... use **non-spatial queries** whenever possible !

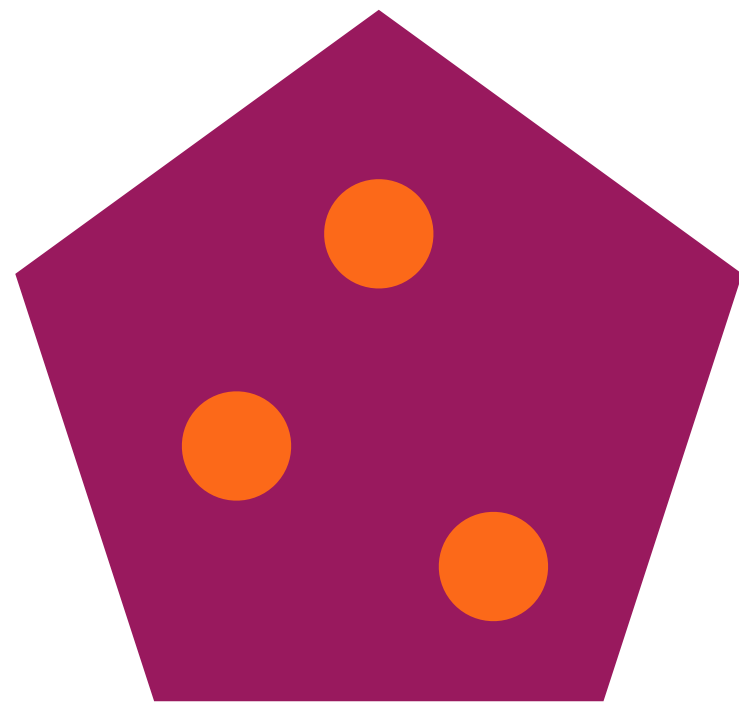
```
polygdf.overlay(polygdf2, how="intersection")
```



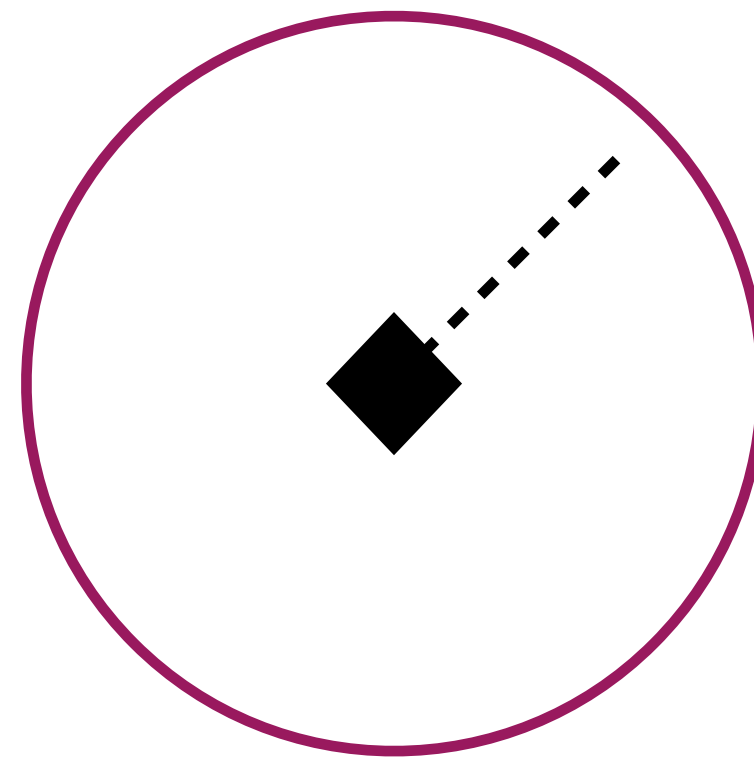
```
polygdf.loc[polygdf["area_name"]=='my_area']
```



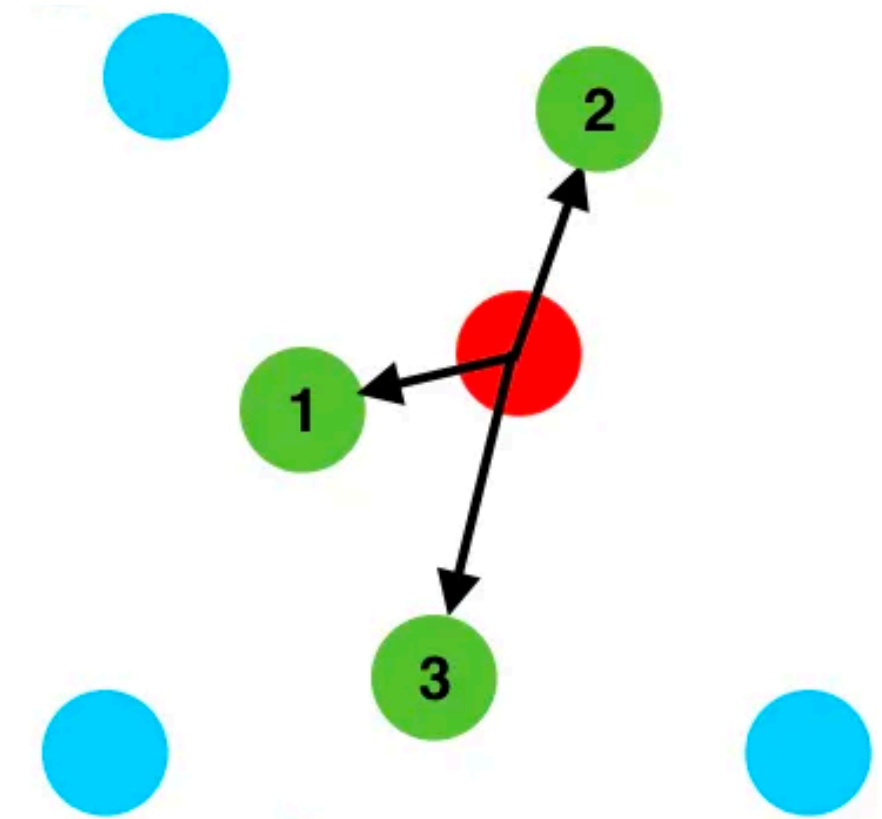
What if you have to do a spatial query?



Spatial join



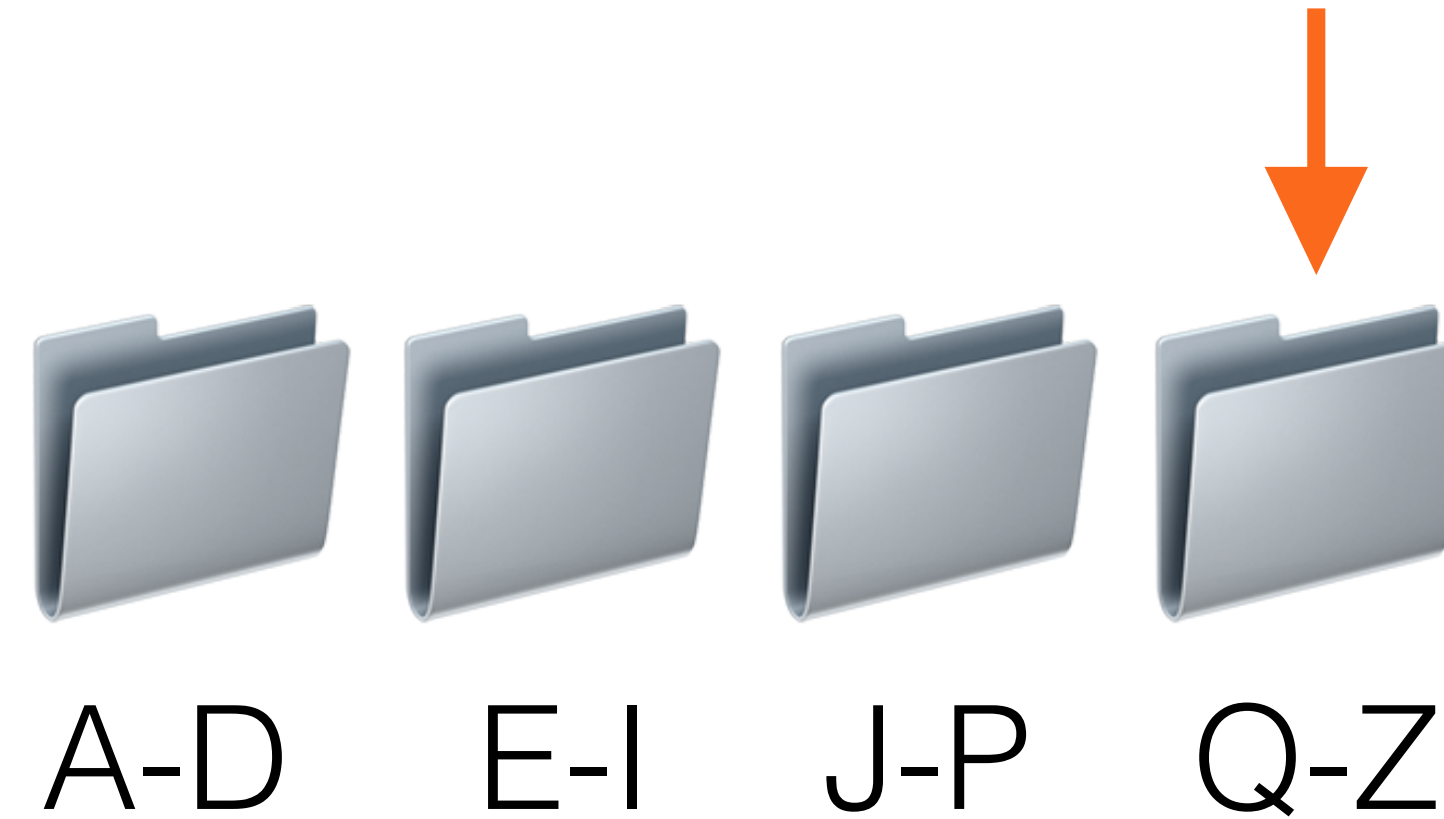
Range query



K-Nearest
Neighbor (KNN)

You should (potentially) use a **spatial index** !

John Snow ? 🤔



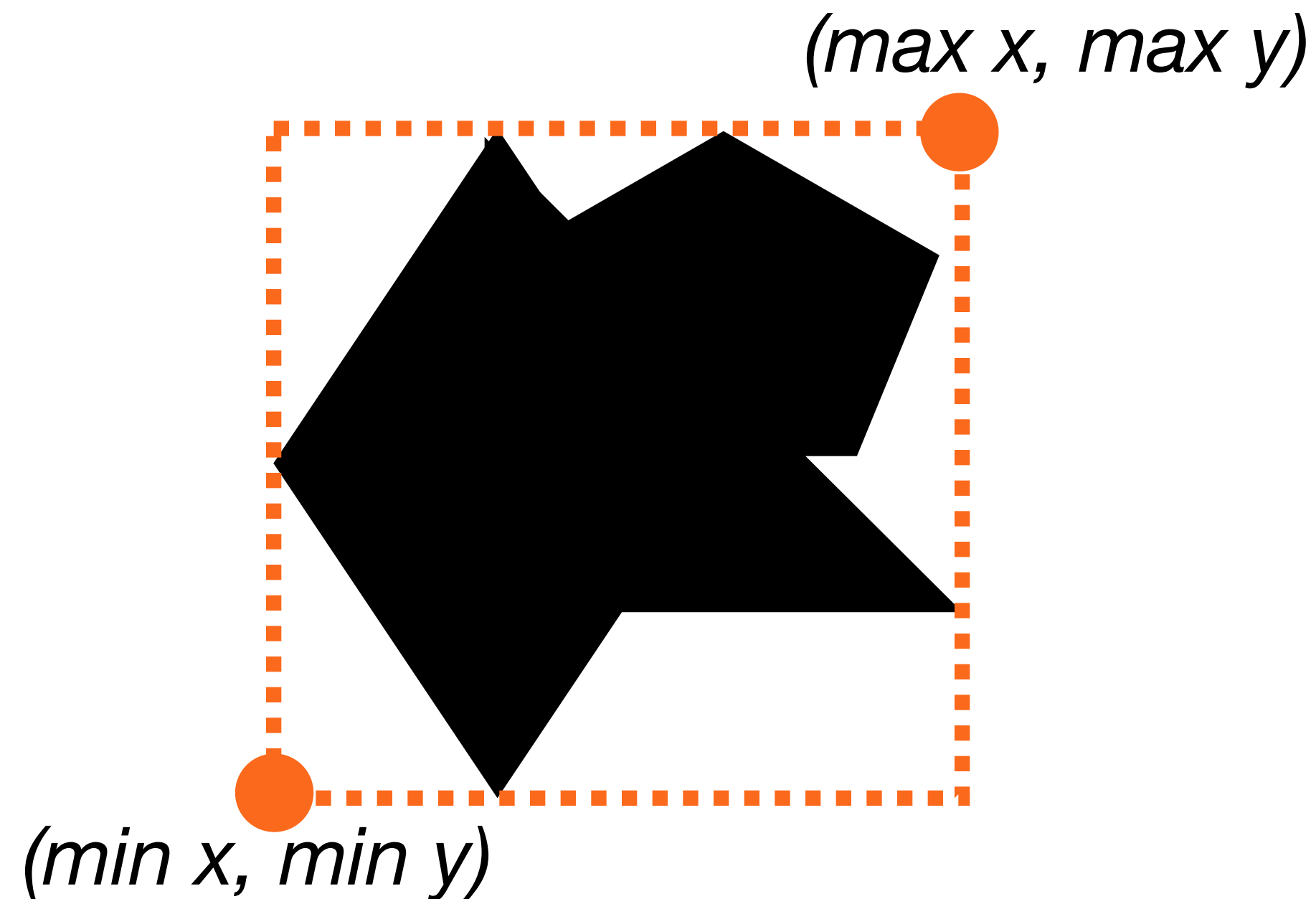
We use indices to speed up data retrieval

Avoid searching every entry during look-up
(in worst-case scenario)

Here tailored to spatial data

Spatial indexing uses the **Minimum Bounding Box (MBB)**

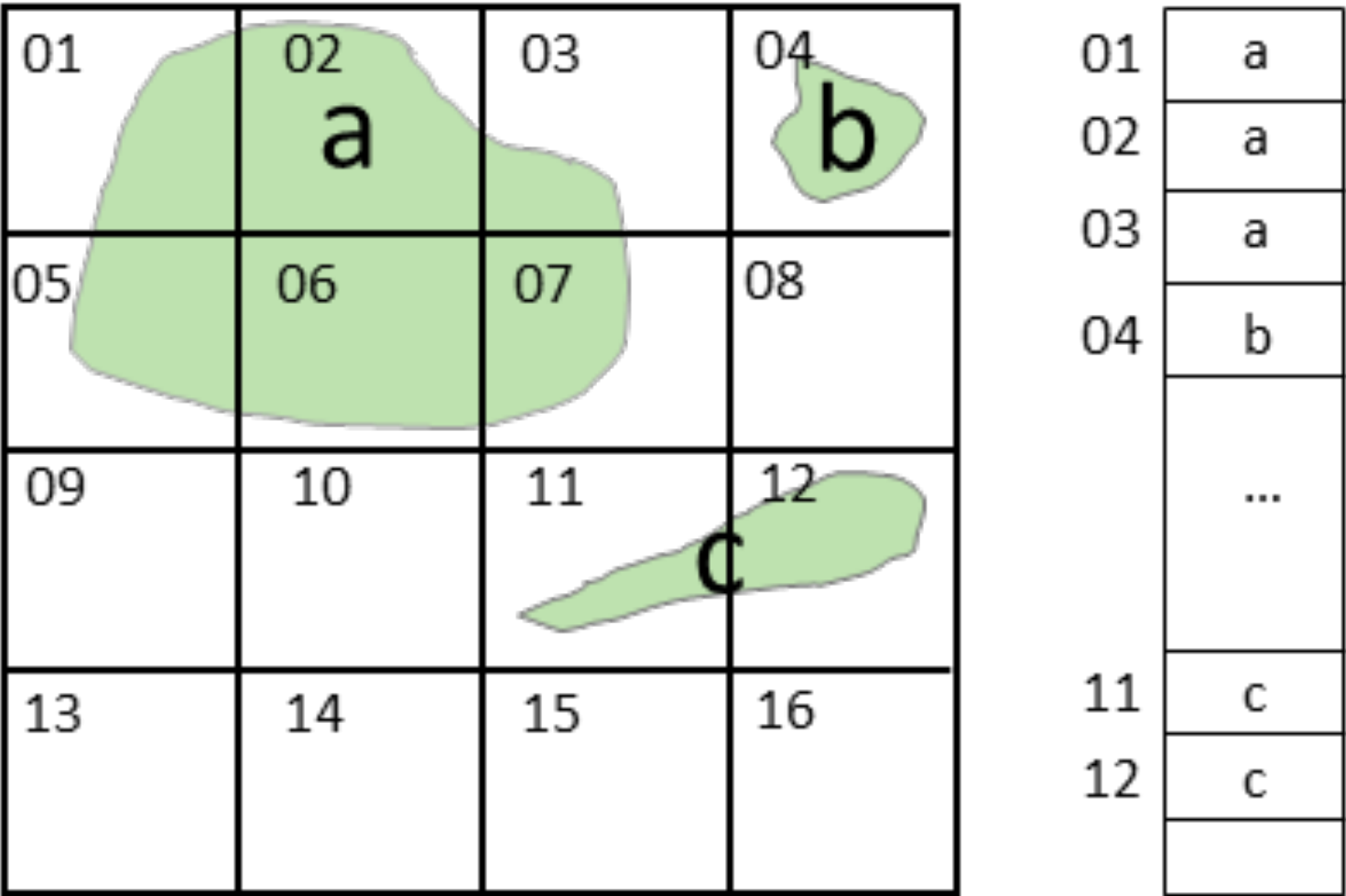
The smallest box that can fit a spatial object defined by $(\min x, \min y)$, $(\max x, \max y)$



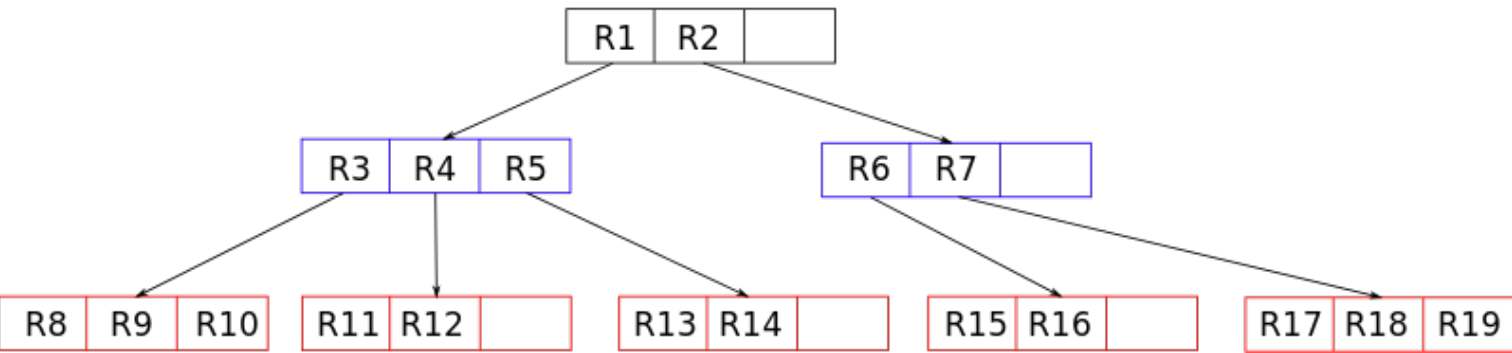
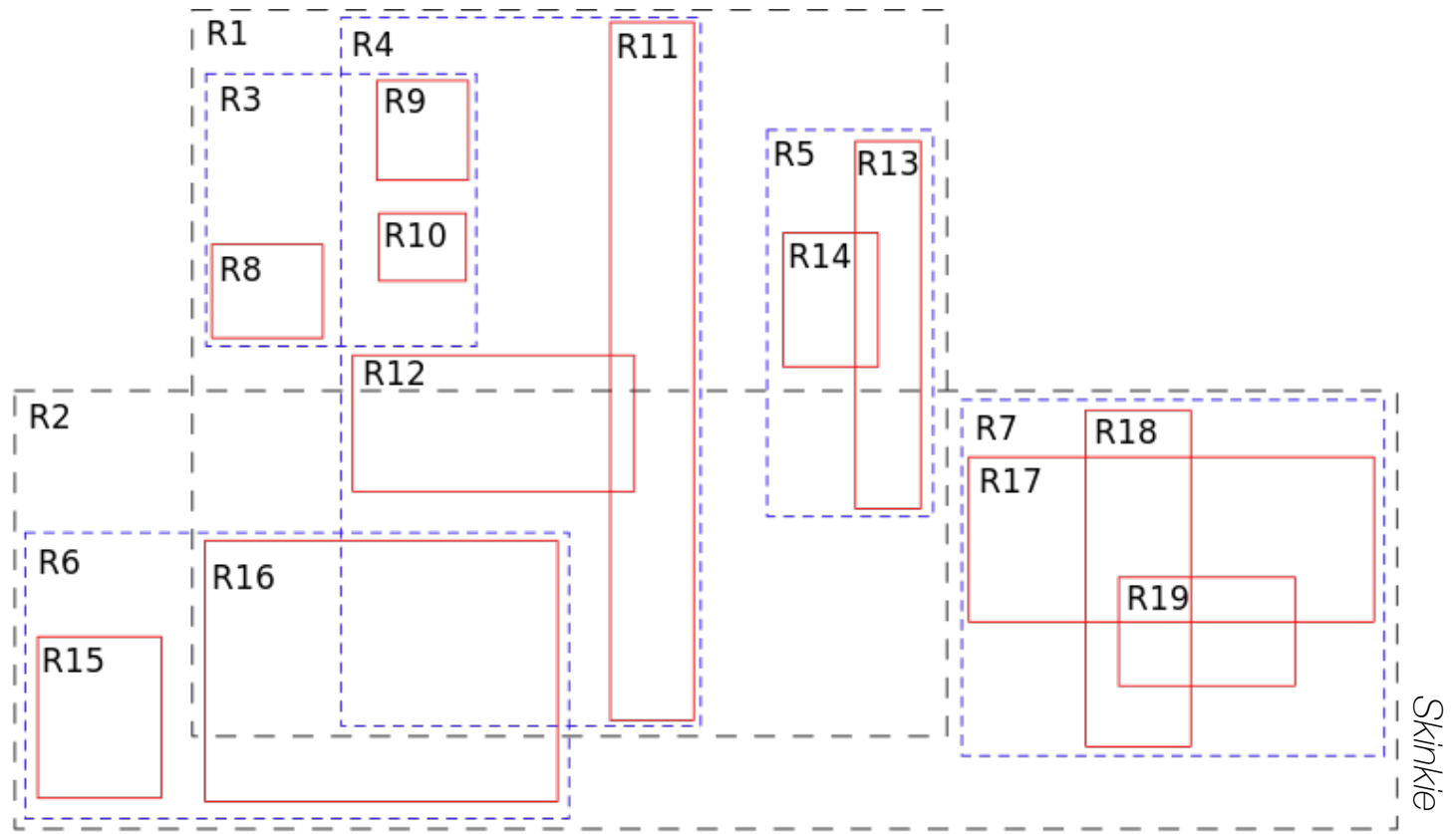
Query: first uses the MBB, then uses the actual shape → Faster

Spatial indexes can be based on two kind of structure

Space-driven:
partitioning the space
into grid cells



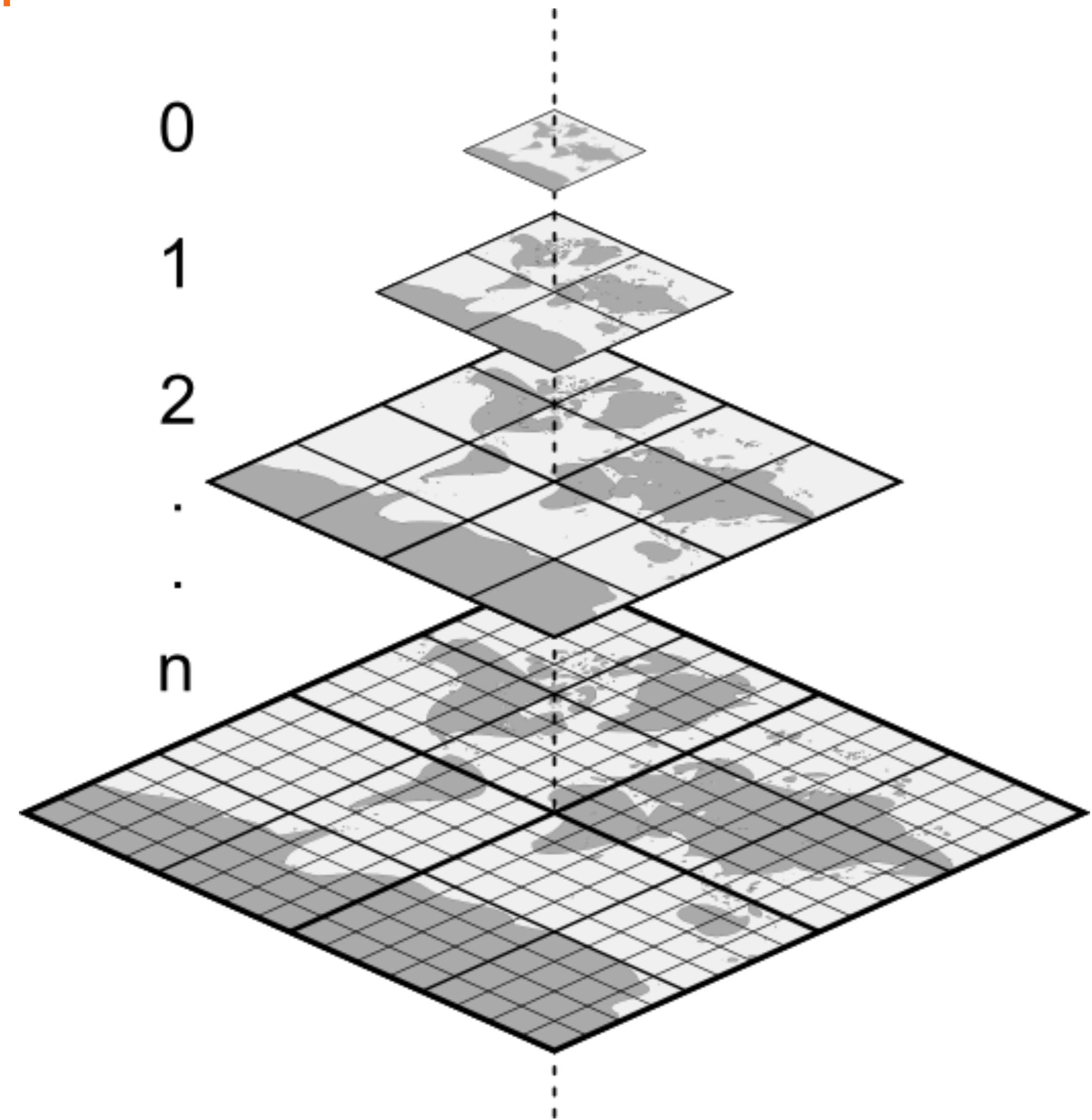
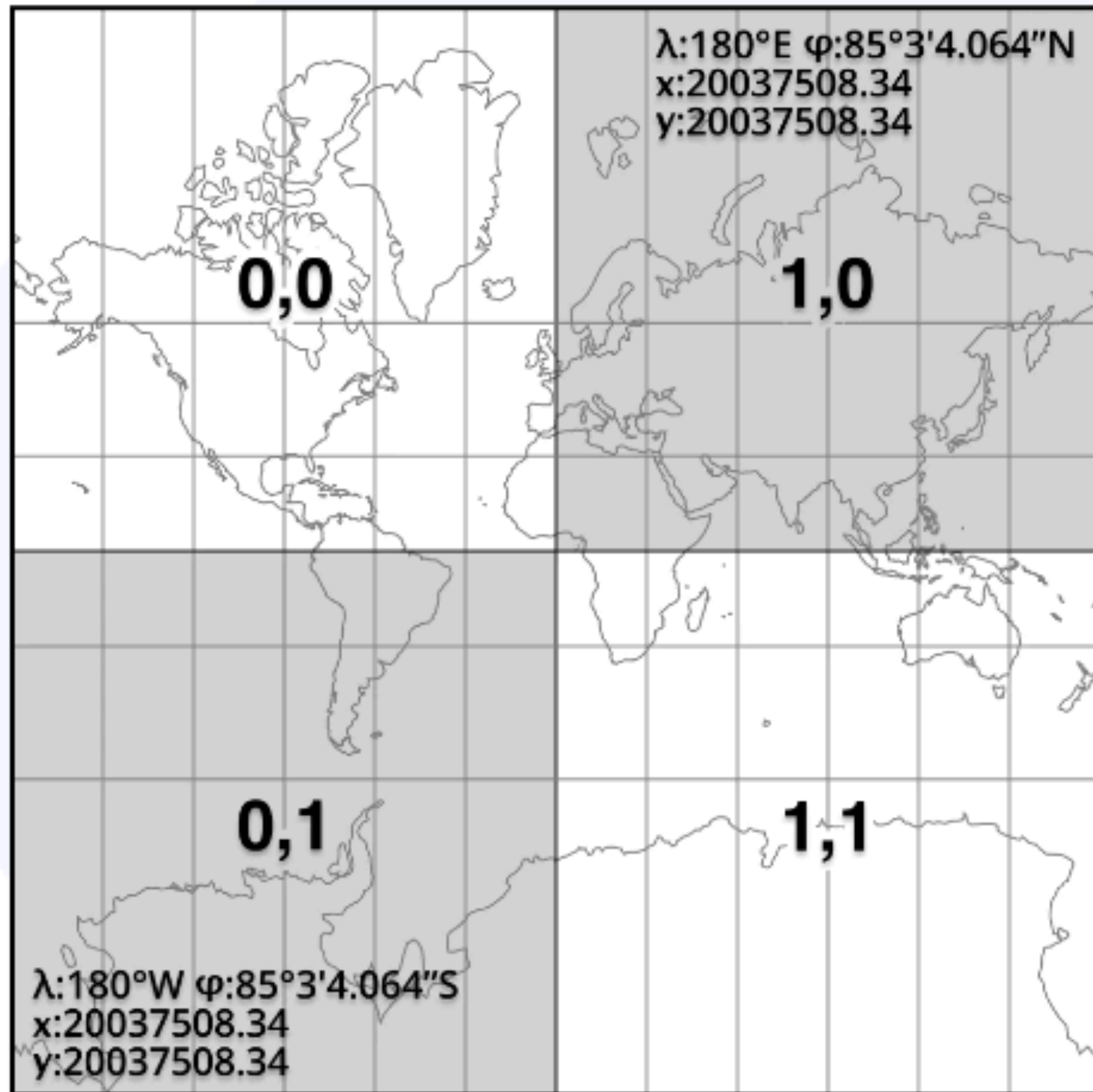
Data-driven:
partitioning the
collection of spatial
objects



Space-driven Spatial Indices

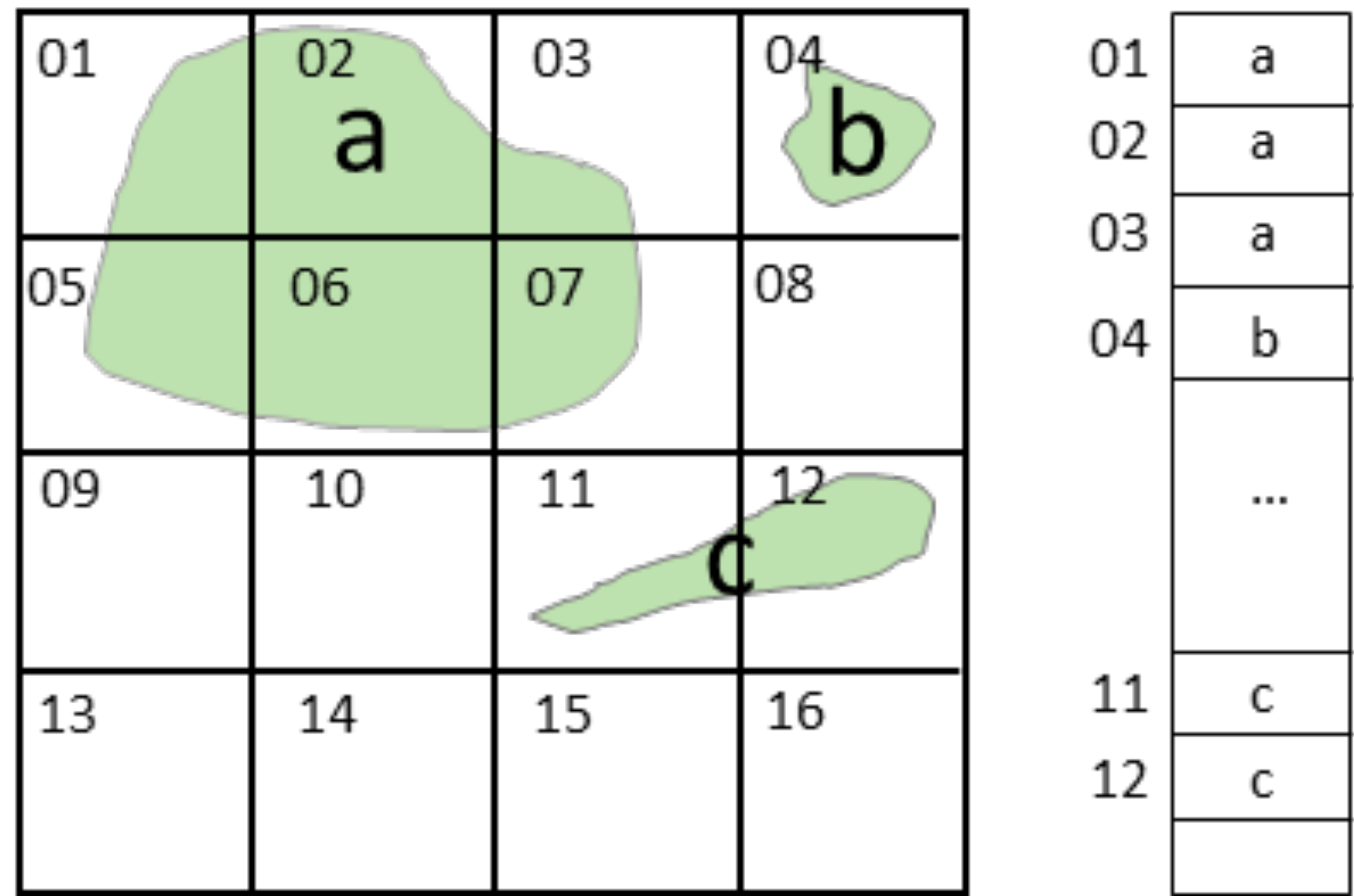
...use a **partitioning strategy**
to decompose the whole space into cells.

Example: Map tiles



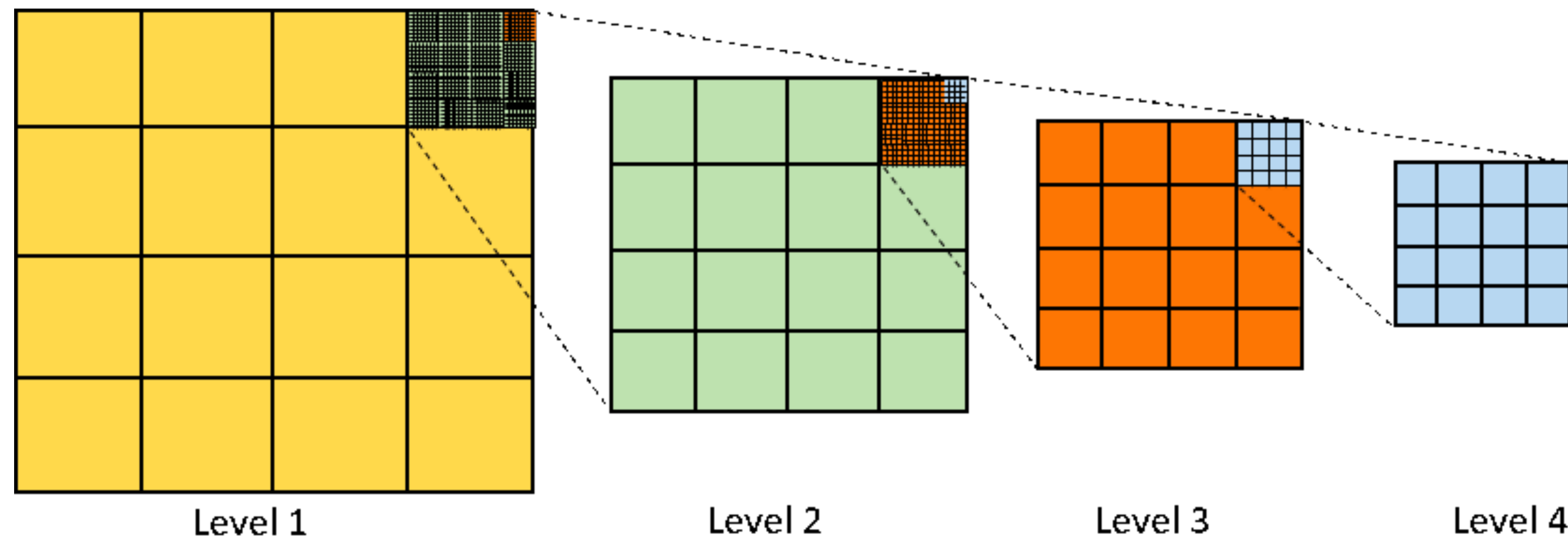
A simple example is the **Fixed Grid Index**

The space is divided into a $n \times n$ array of **equal size cells**



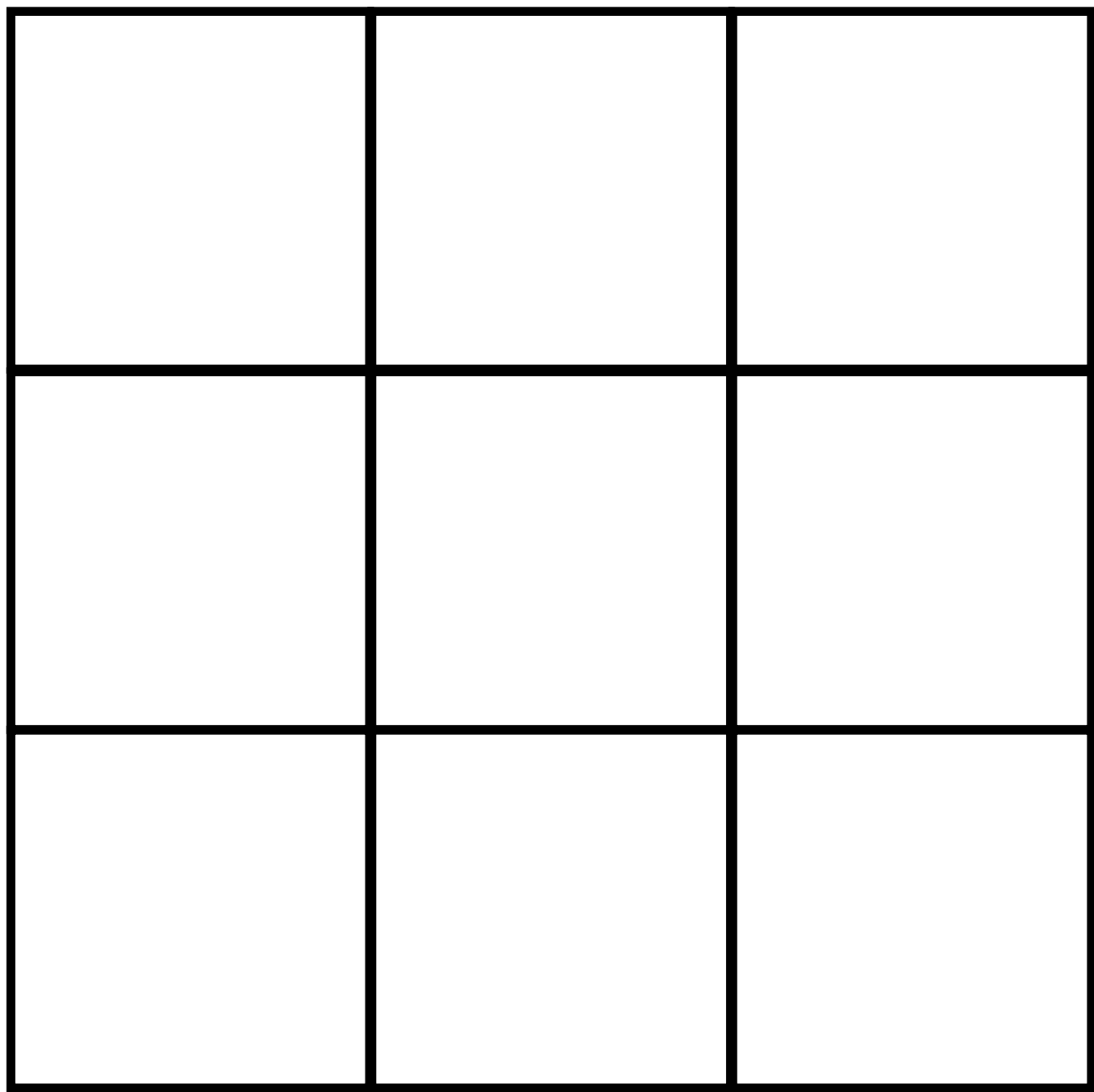
Each cell contains the objects intersecting with its boundaries

The cells can have **multiples scales**

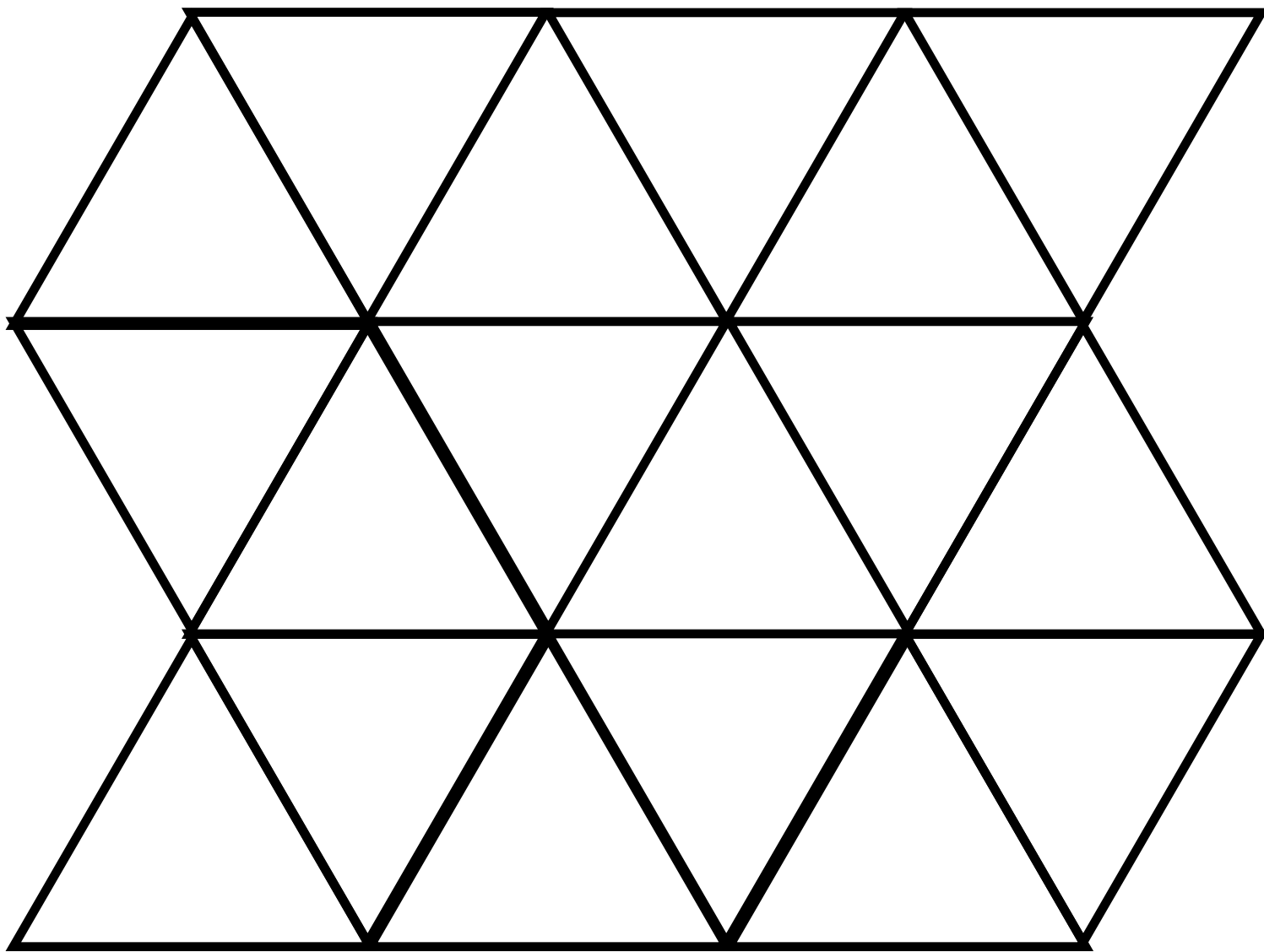


Different kind of **lattice** can be used

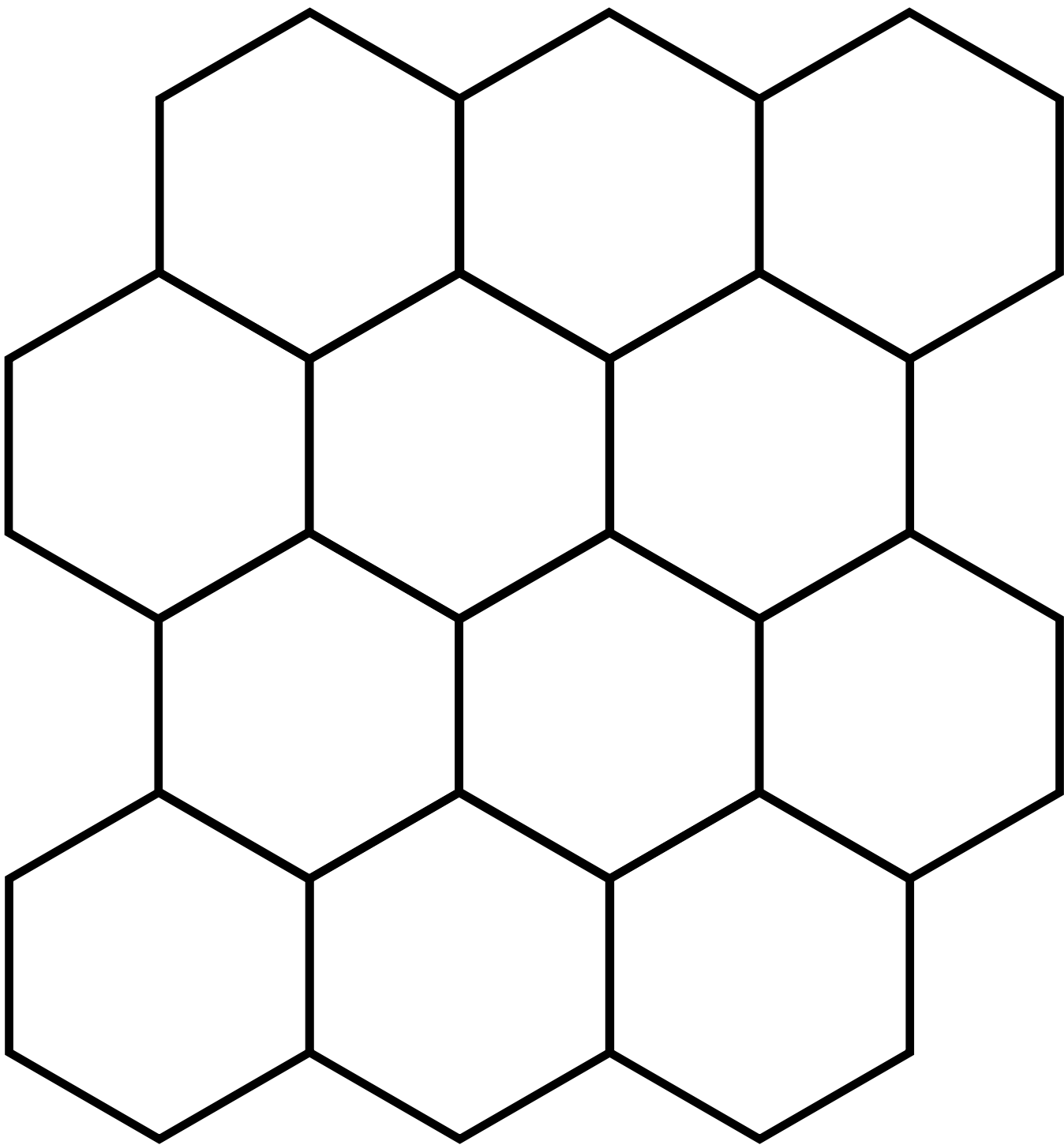
Grid/Rectangular



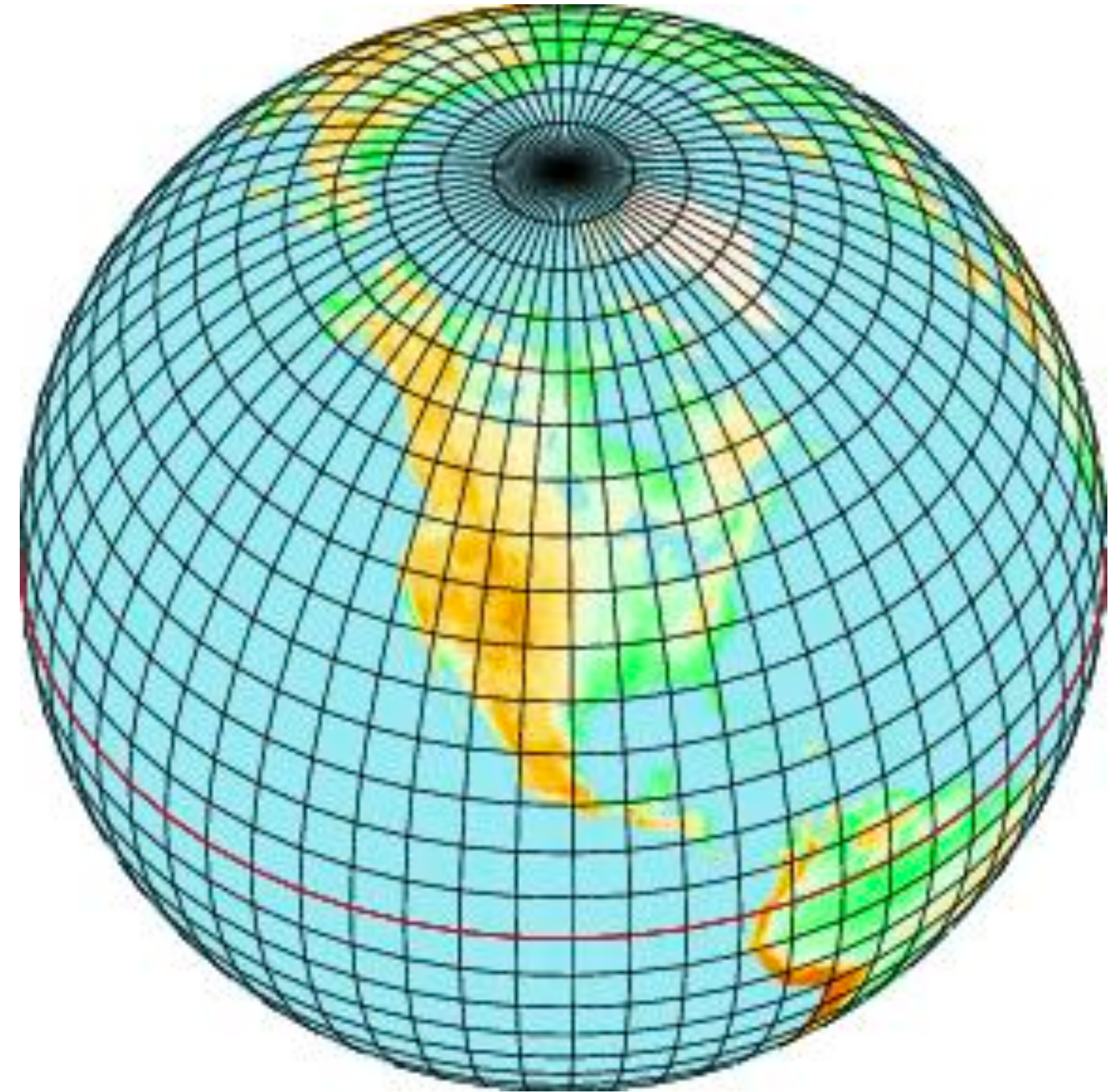
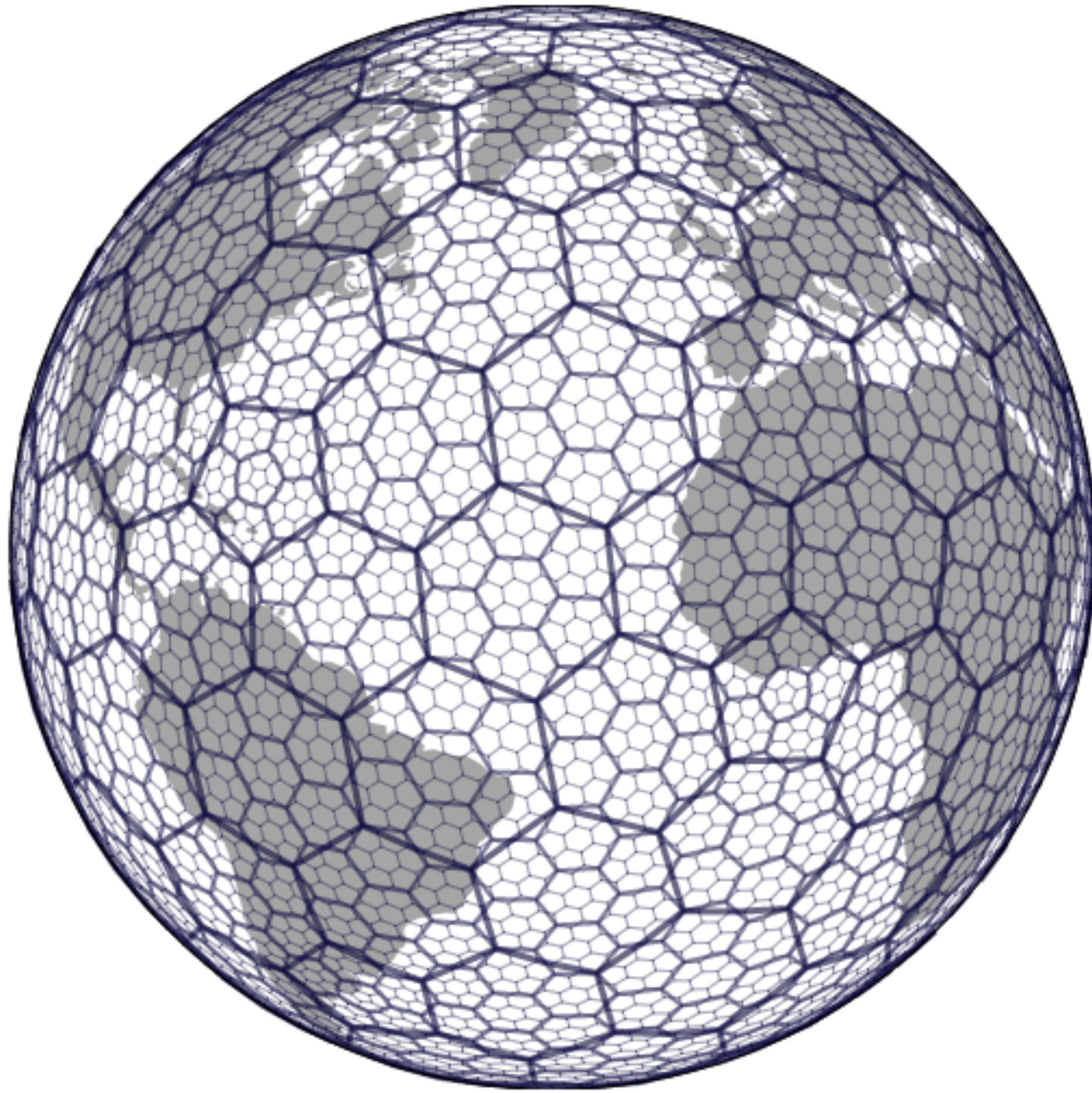
Triangular



Honeycomb

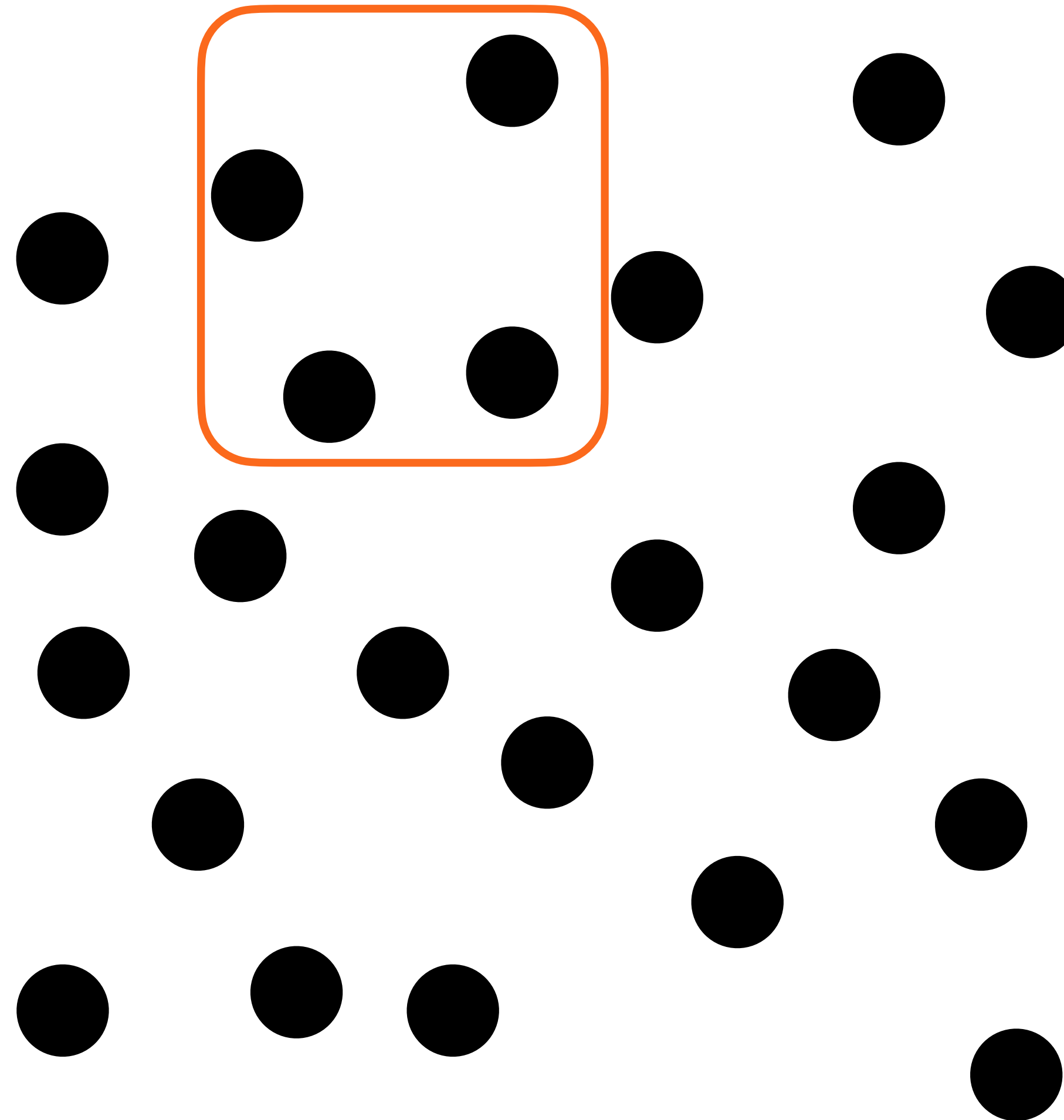


Some grid-based spatial indices are global



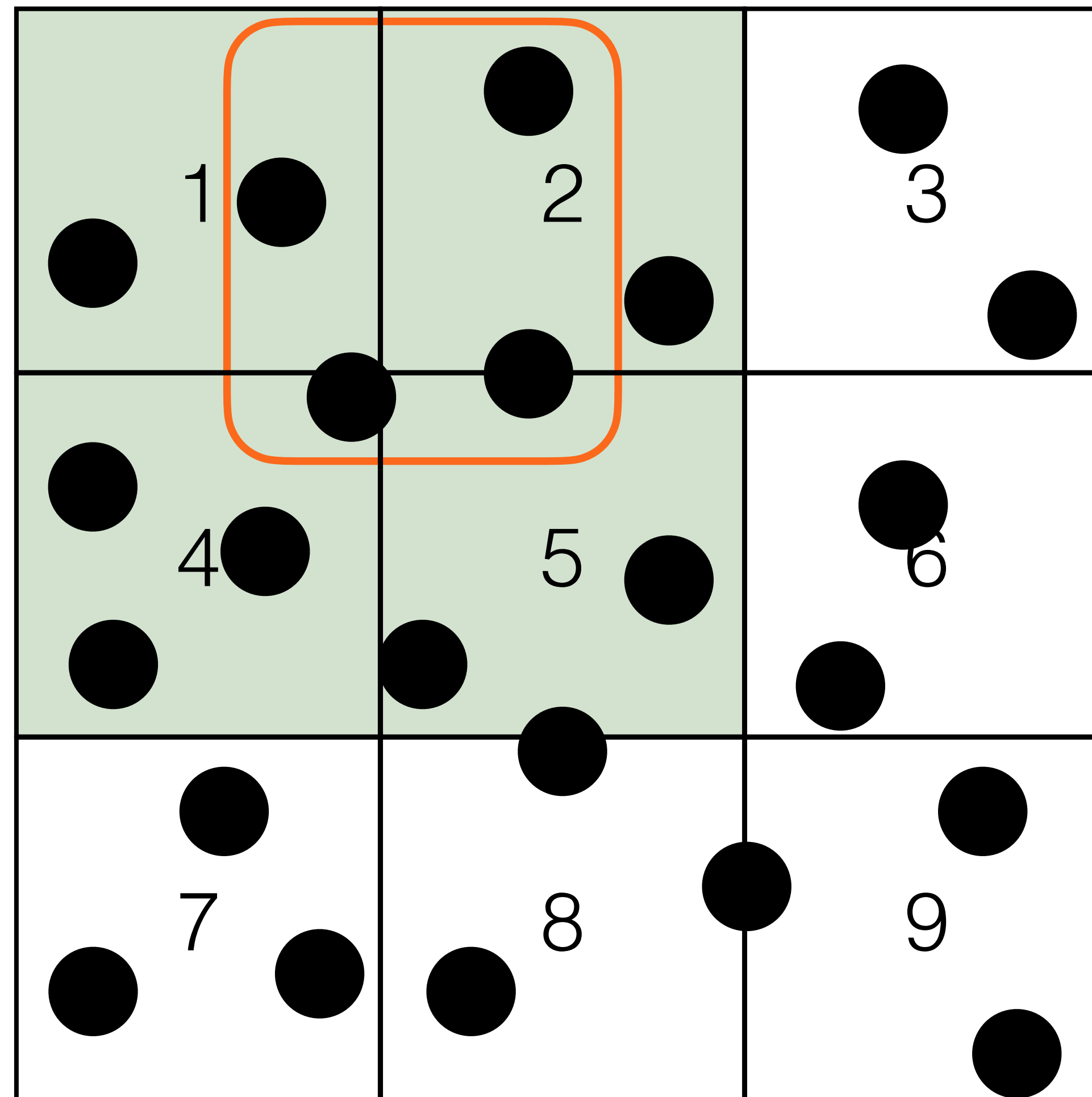
Application to the first problem

Which points are inside the polygon ?

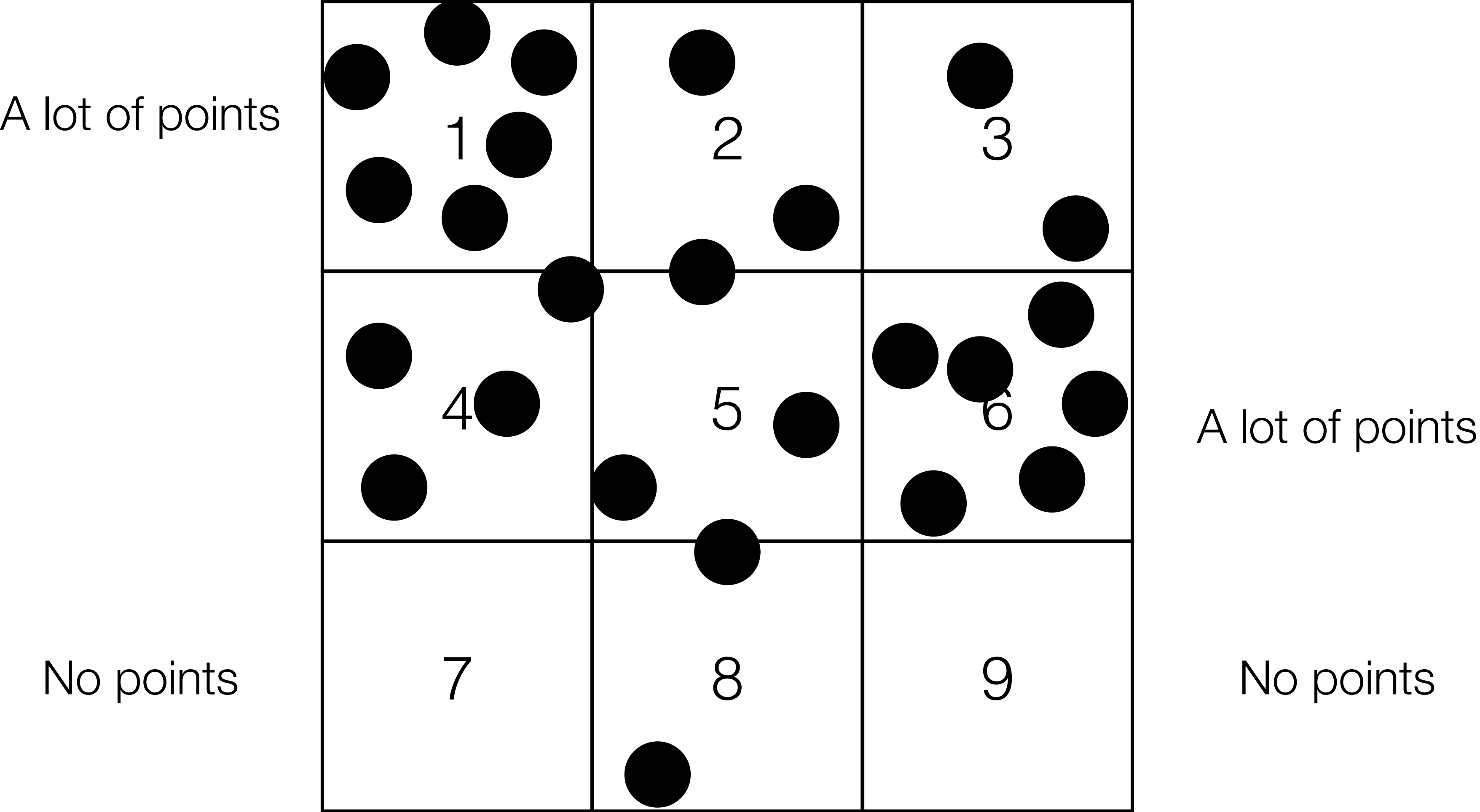


Application to the first problem

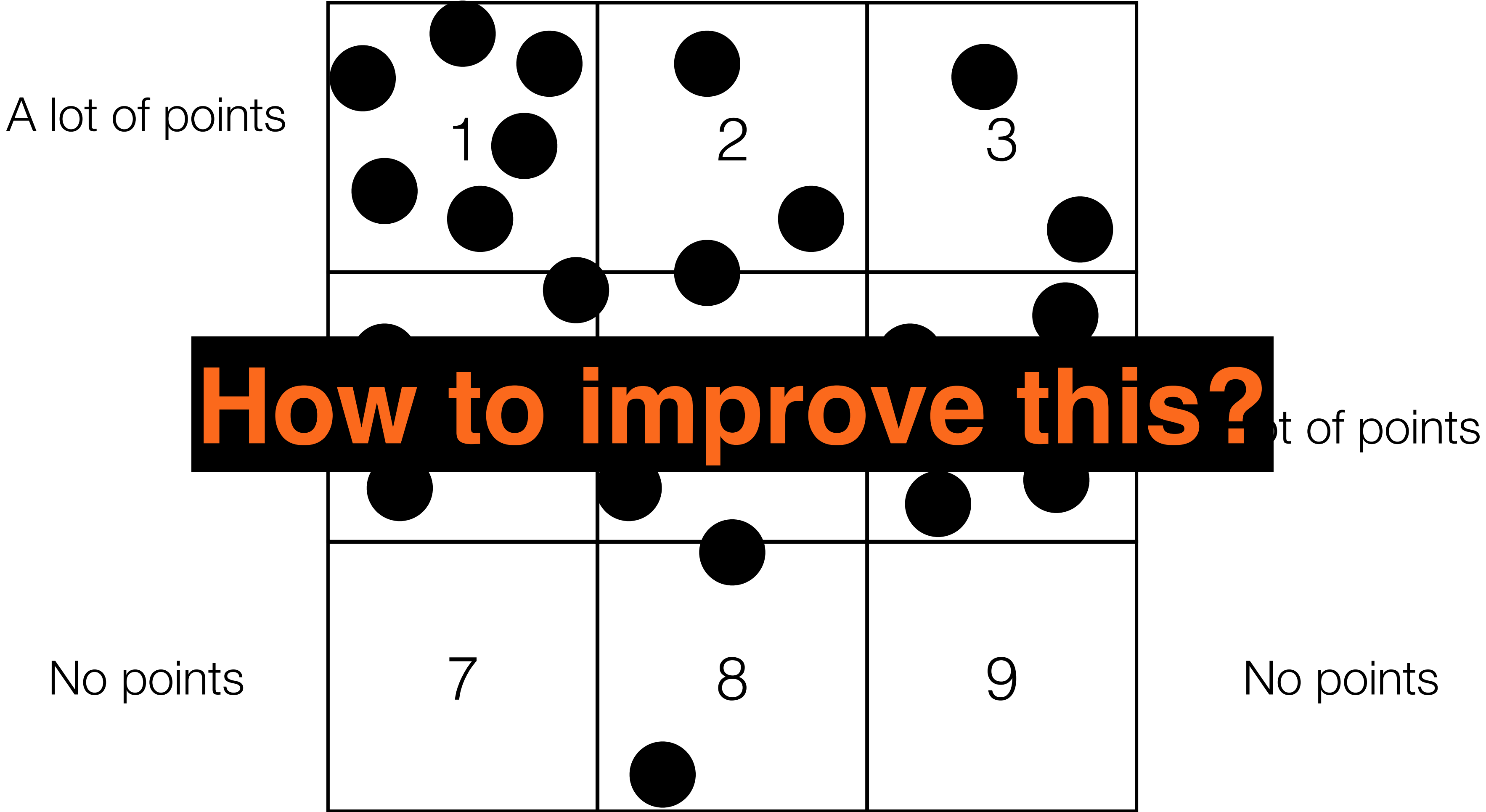
Test only the points in cells containing the polygon !



The fixed sizes are not adapted to the data distribution

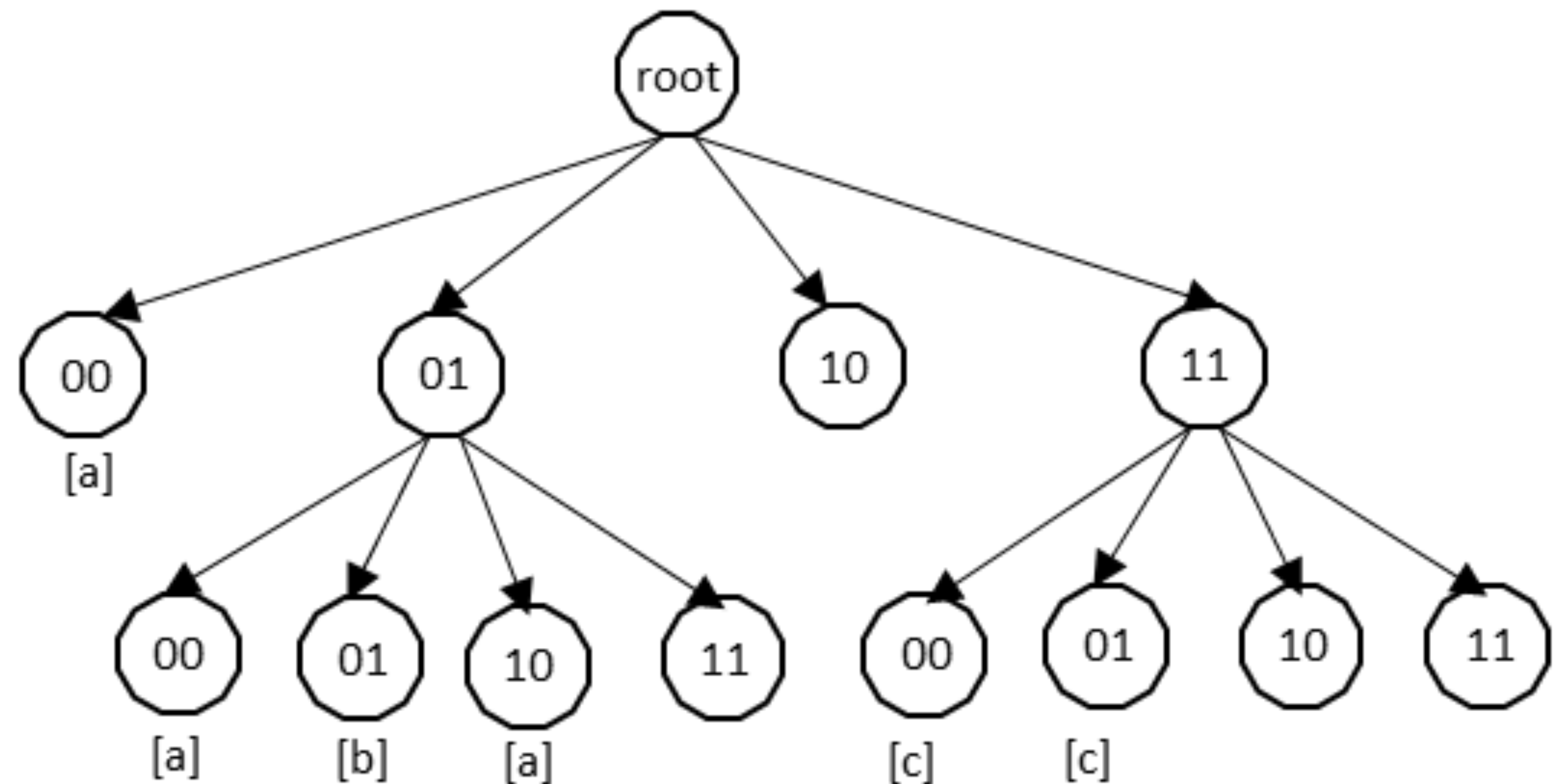
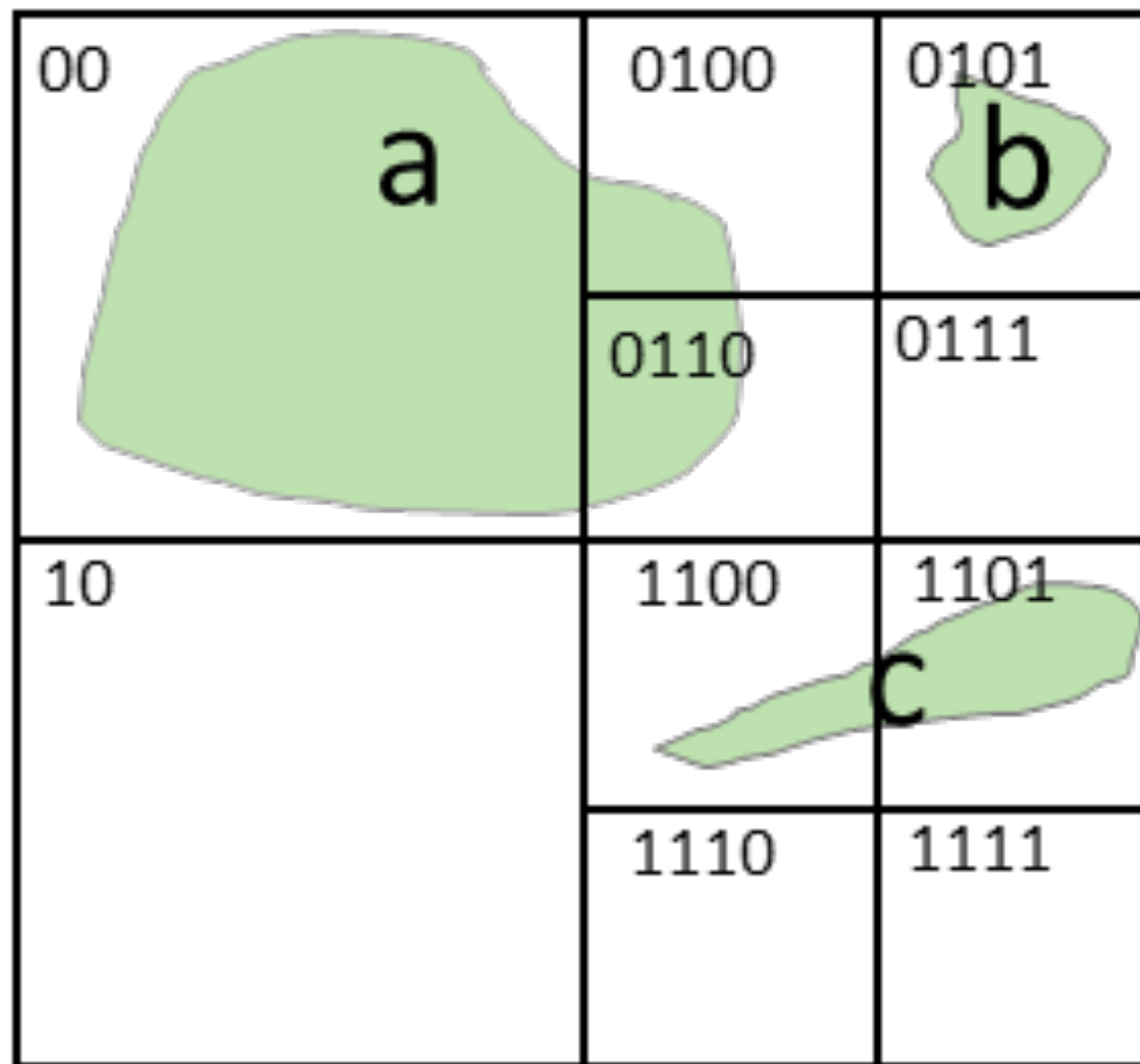


The fixed sizes are not adapted to the data distribution



Quadtree creates sub-divisions of cells

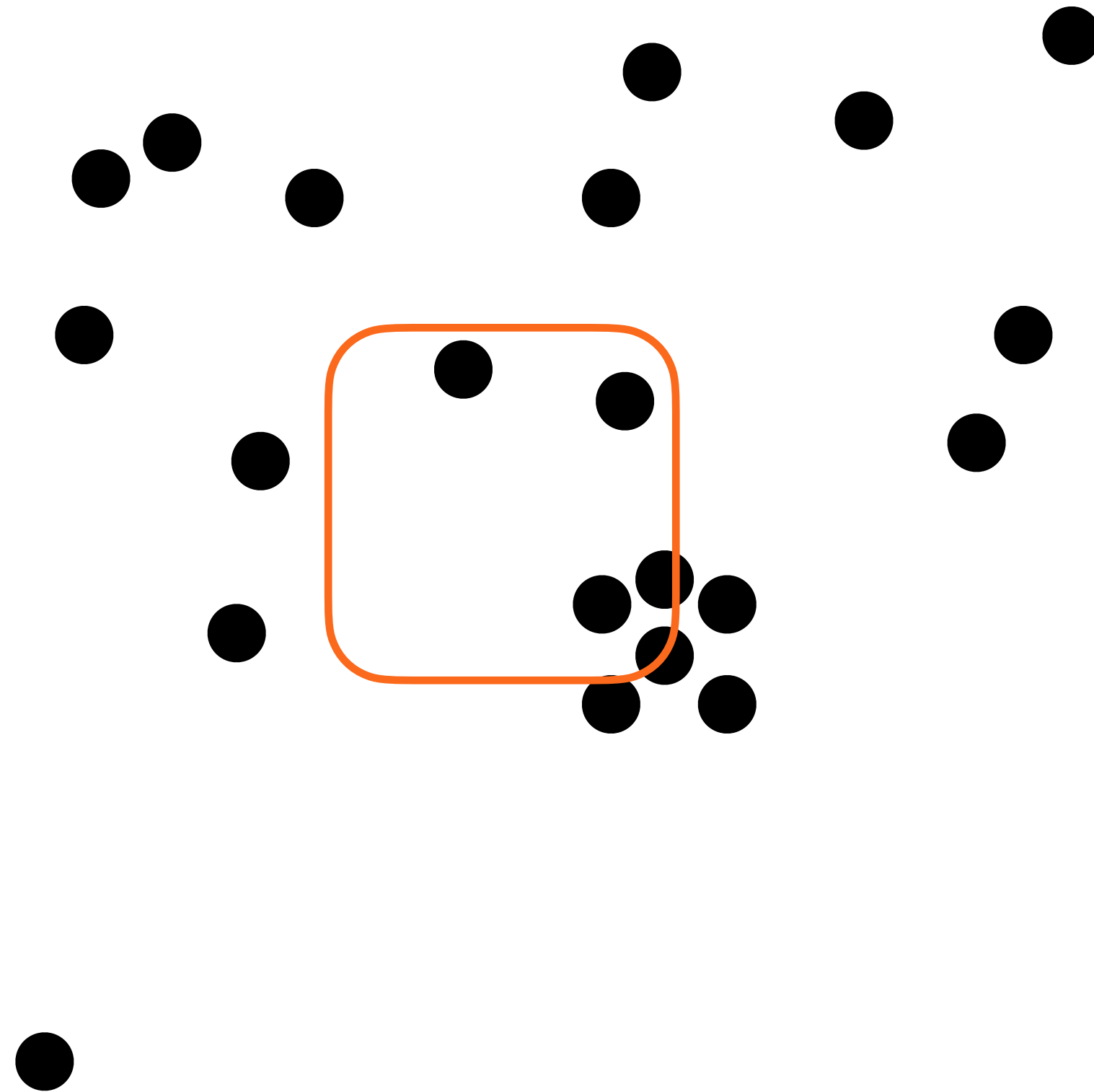
Each cell can have **children** depending on the **density** of objects



Invented by (Finkel and Bentley, 1974)

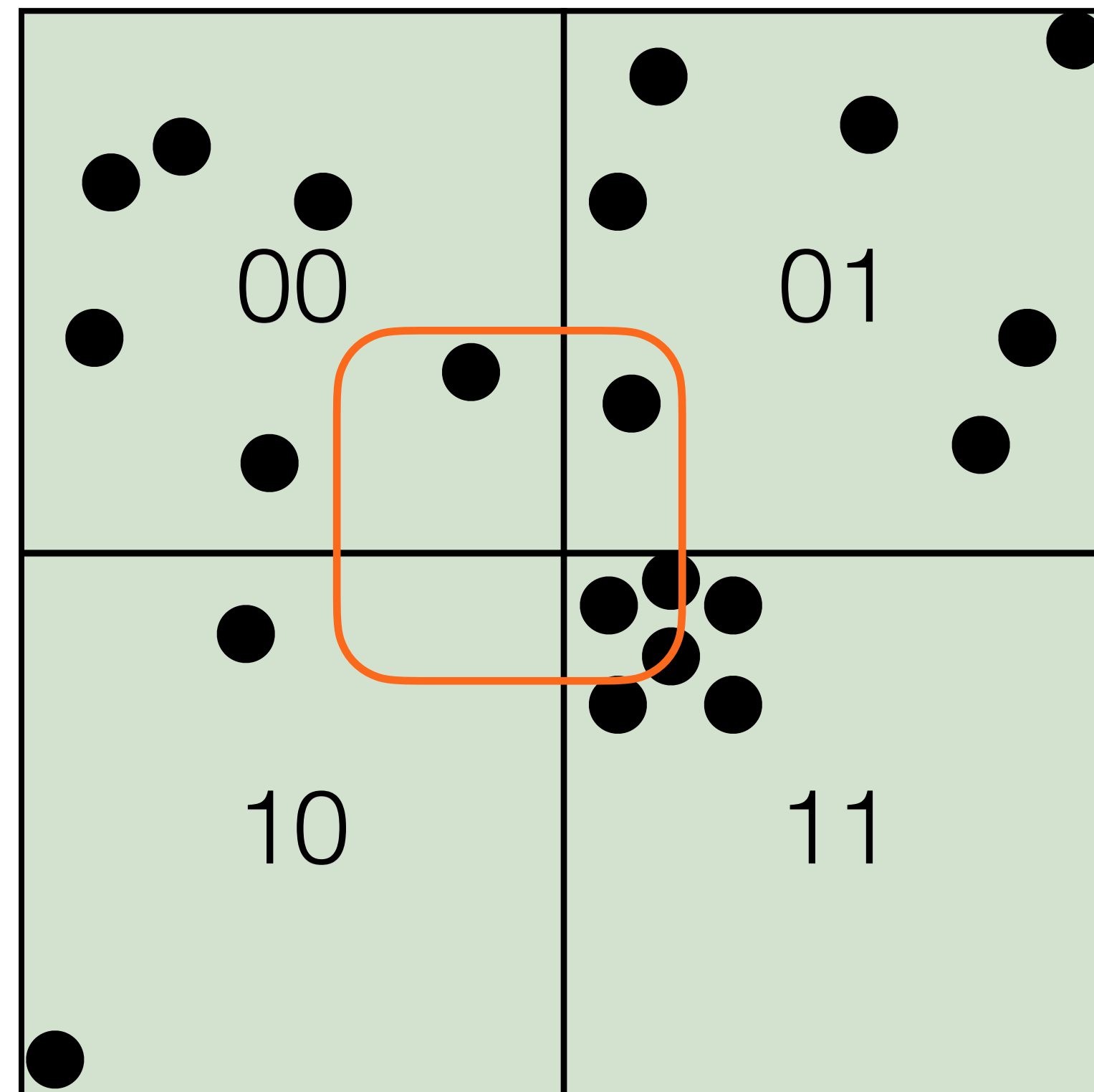
Application to the first problem

Which points are inside the polygon ?

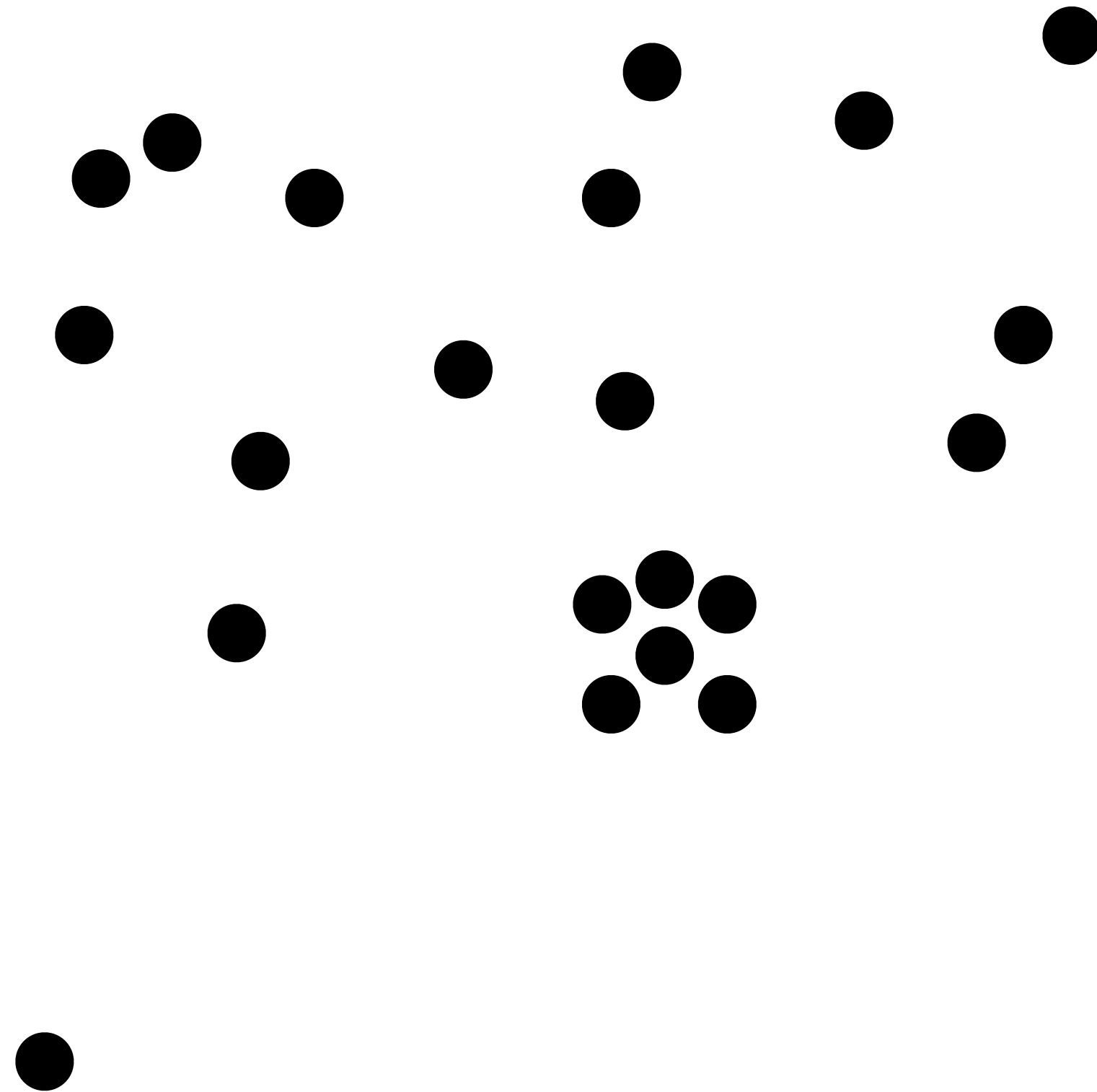


Application to the first problem

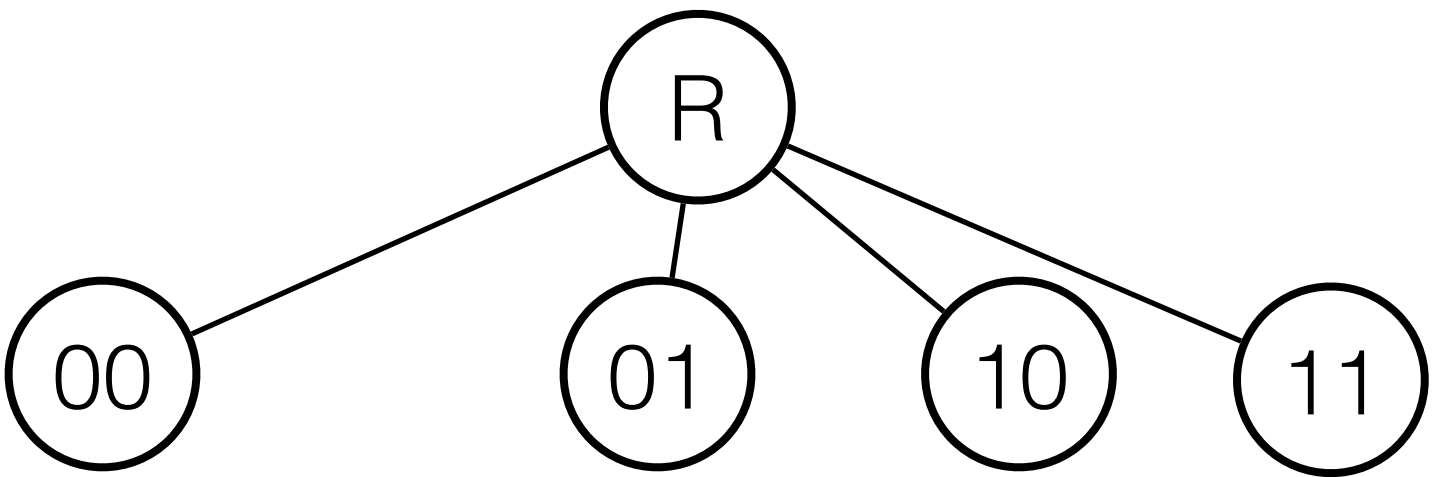
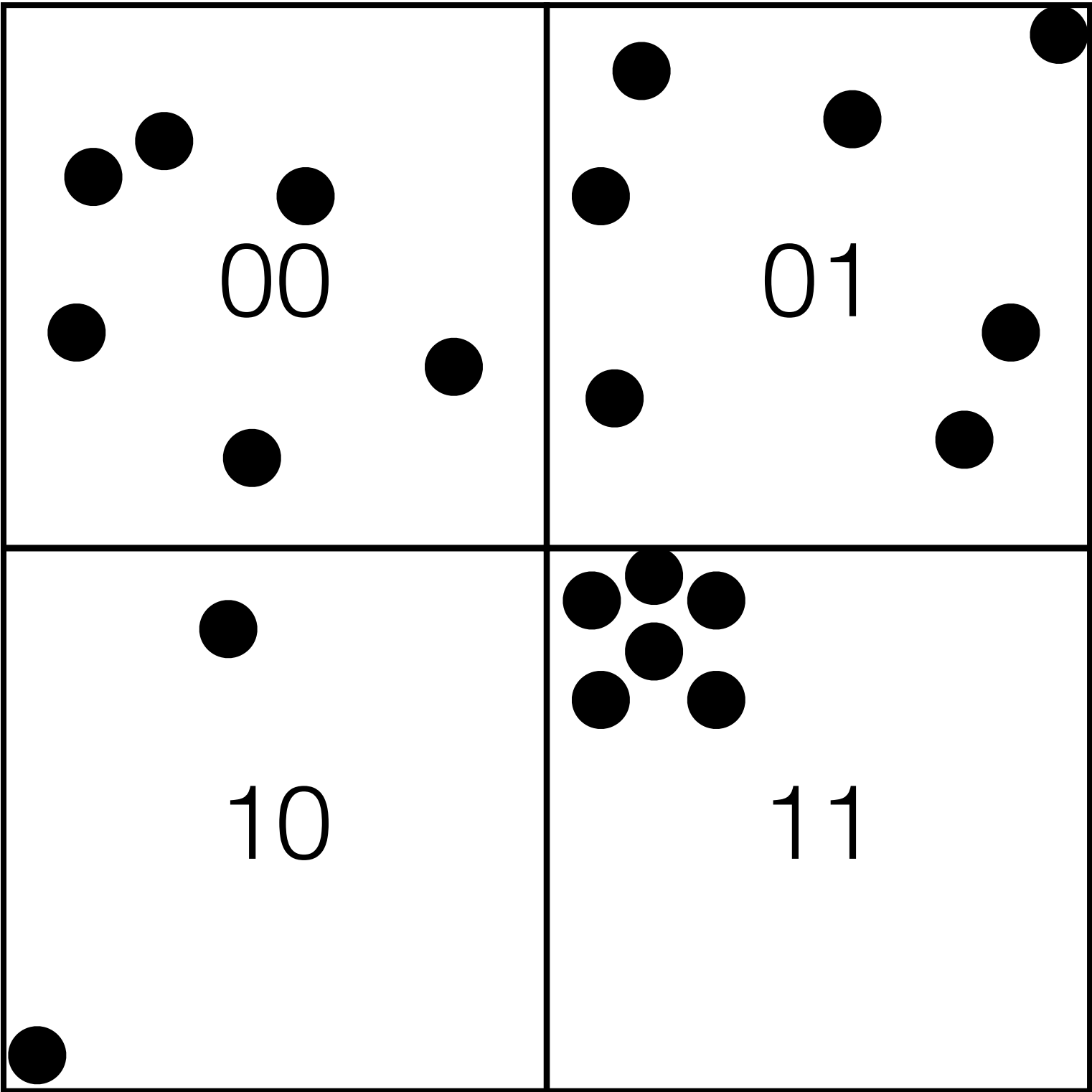
A 2×2 fixed grid index would make us test everything



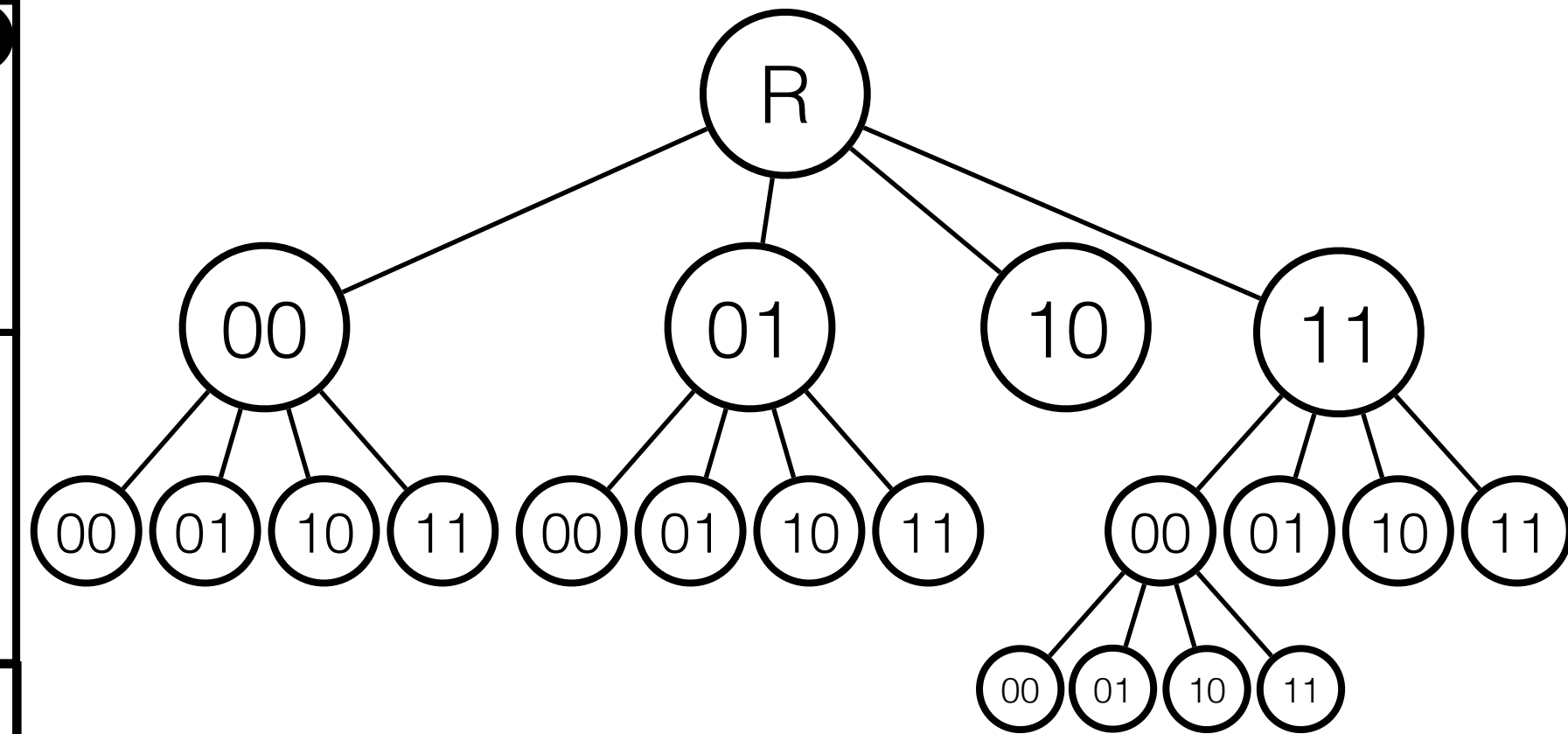
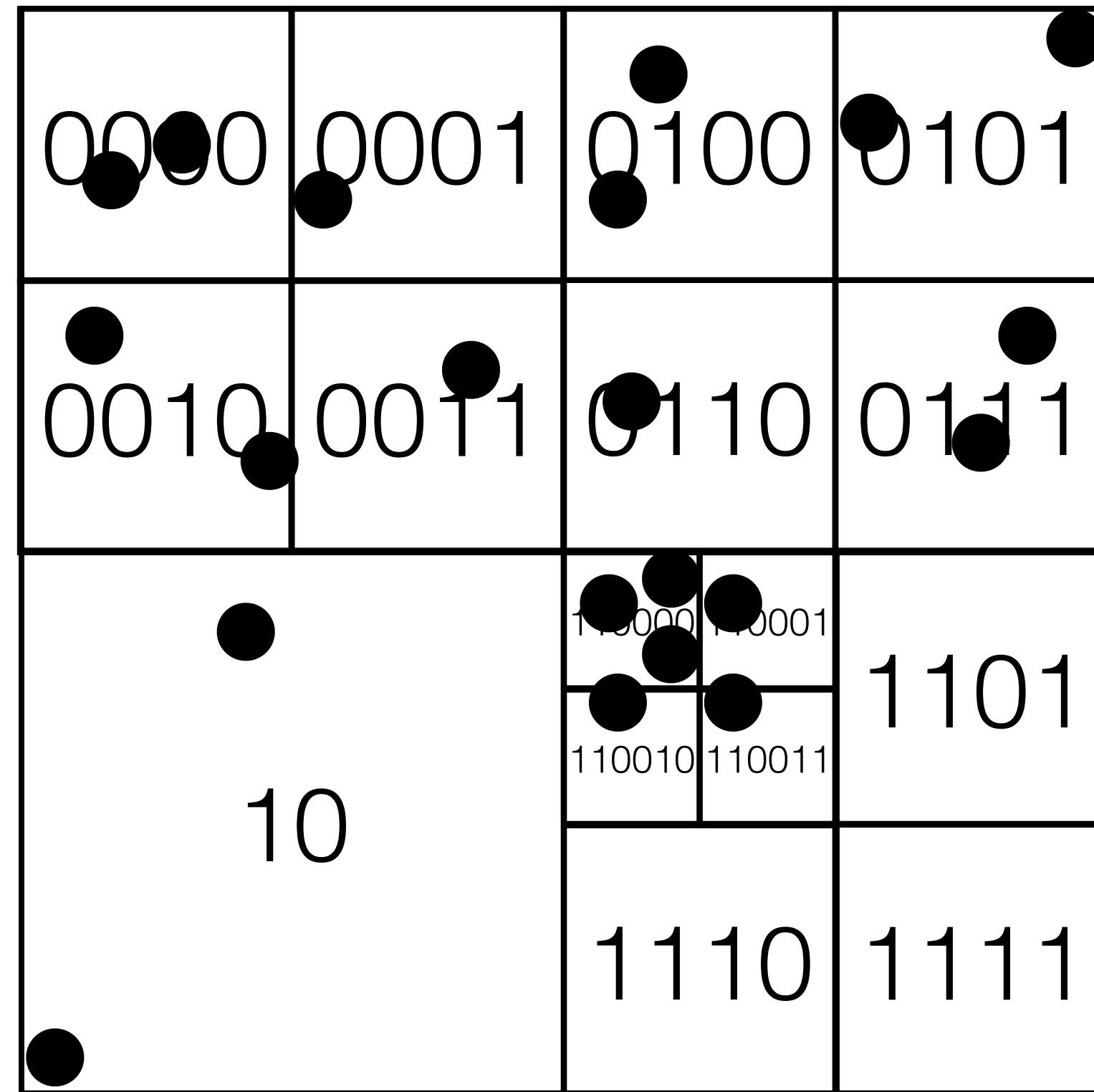
Starting from a list of nodes...



... we divide the space in 4 children **iteratively**...

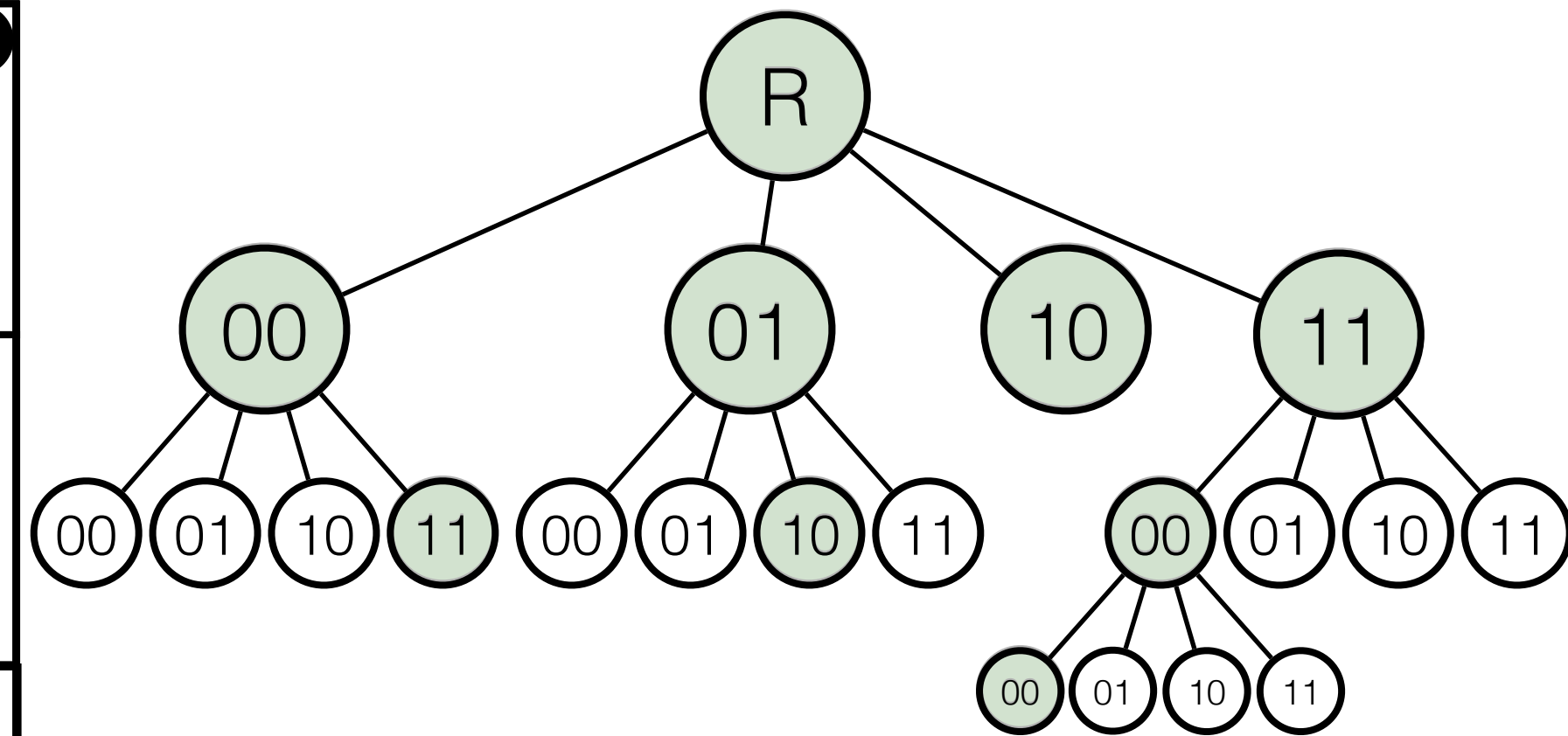
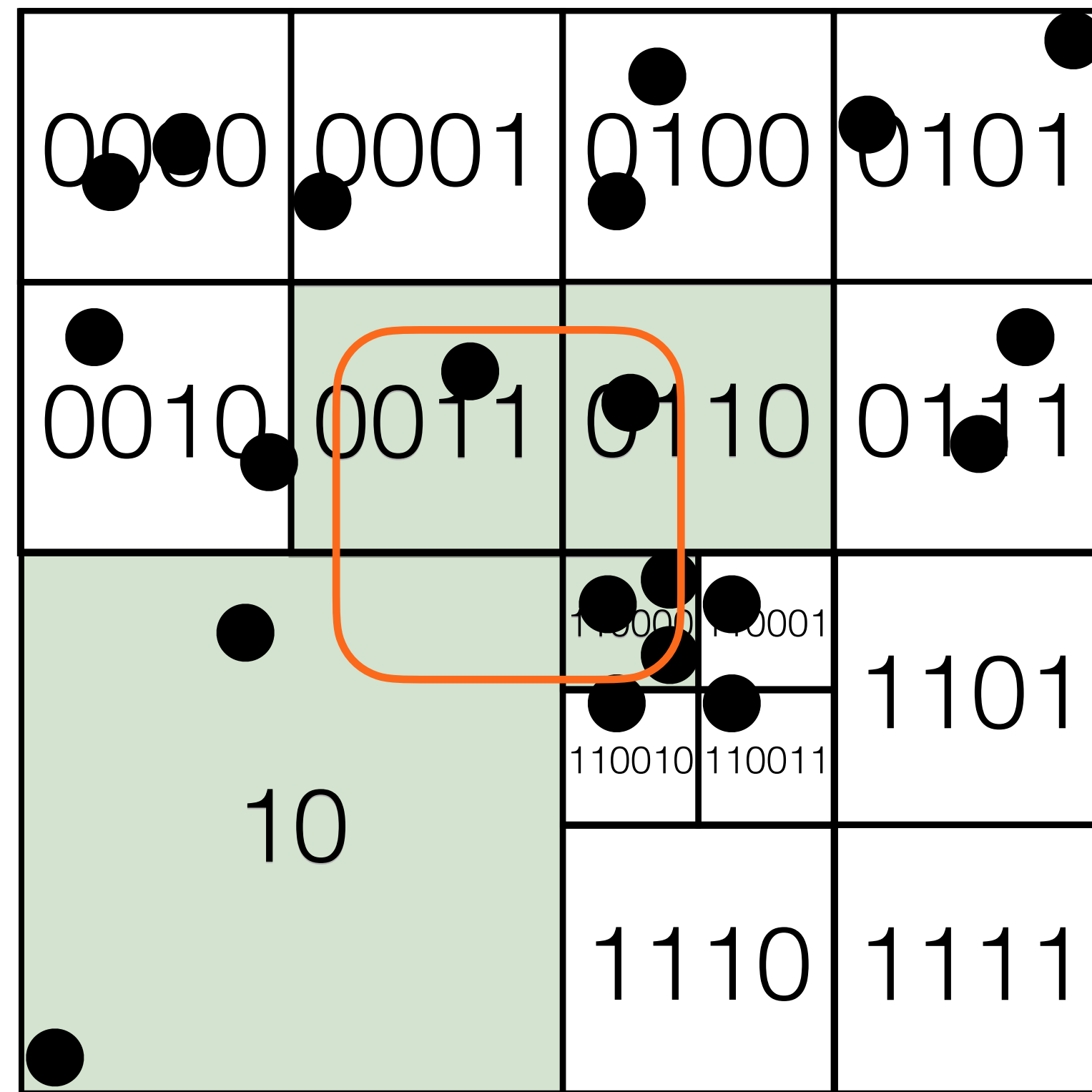


... until there is only a **threshold** of objects in each cell



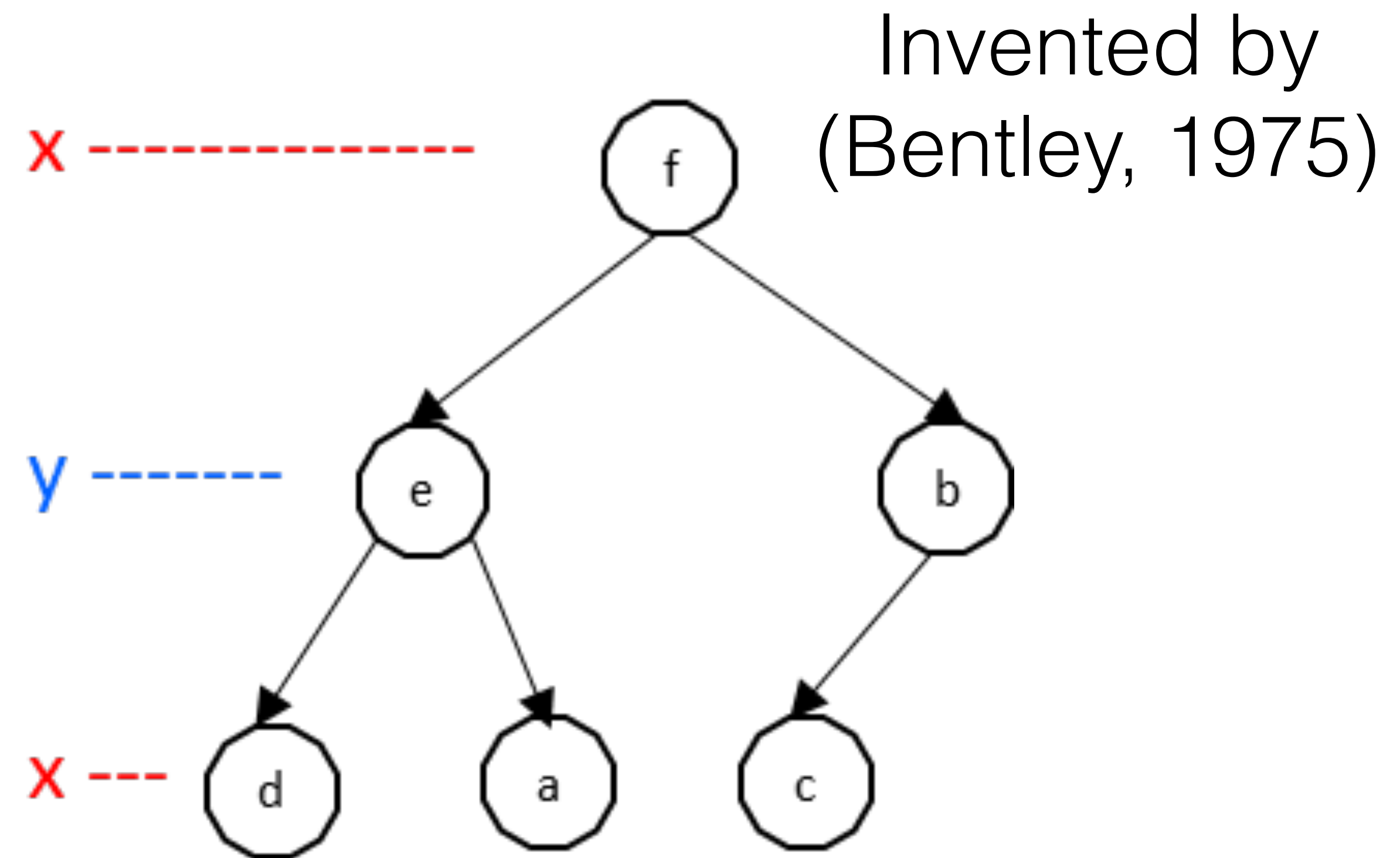
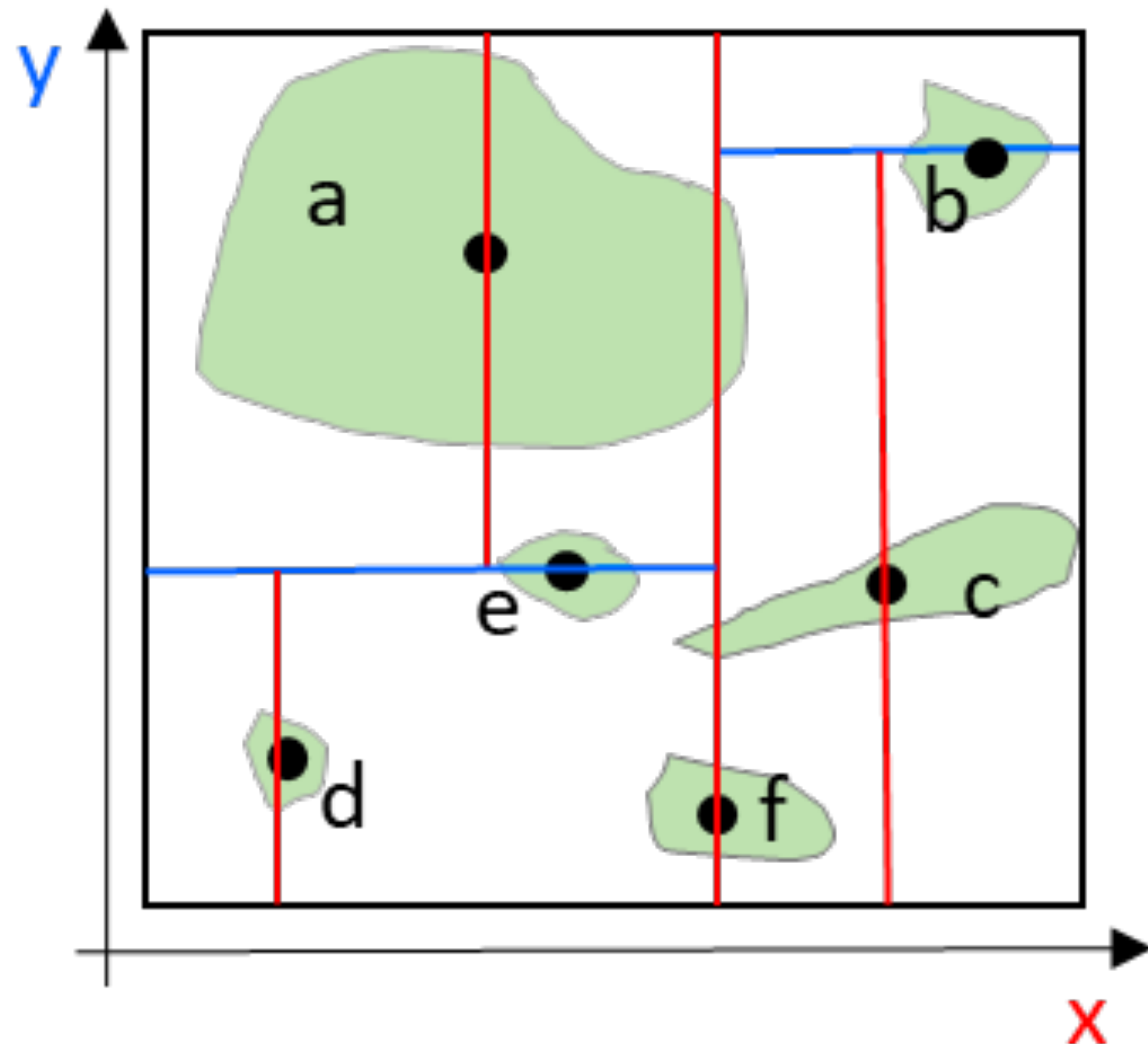
... until there is only a **threshold** of objects in each cell

Test only the points in cells containing the polygon !

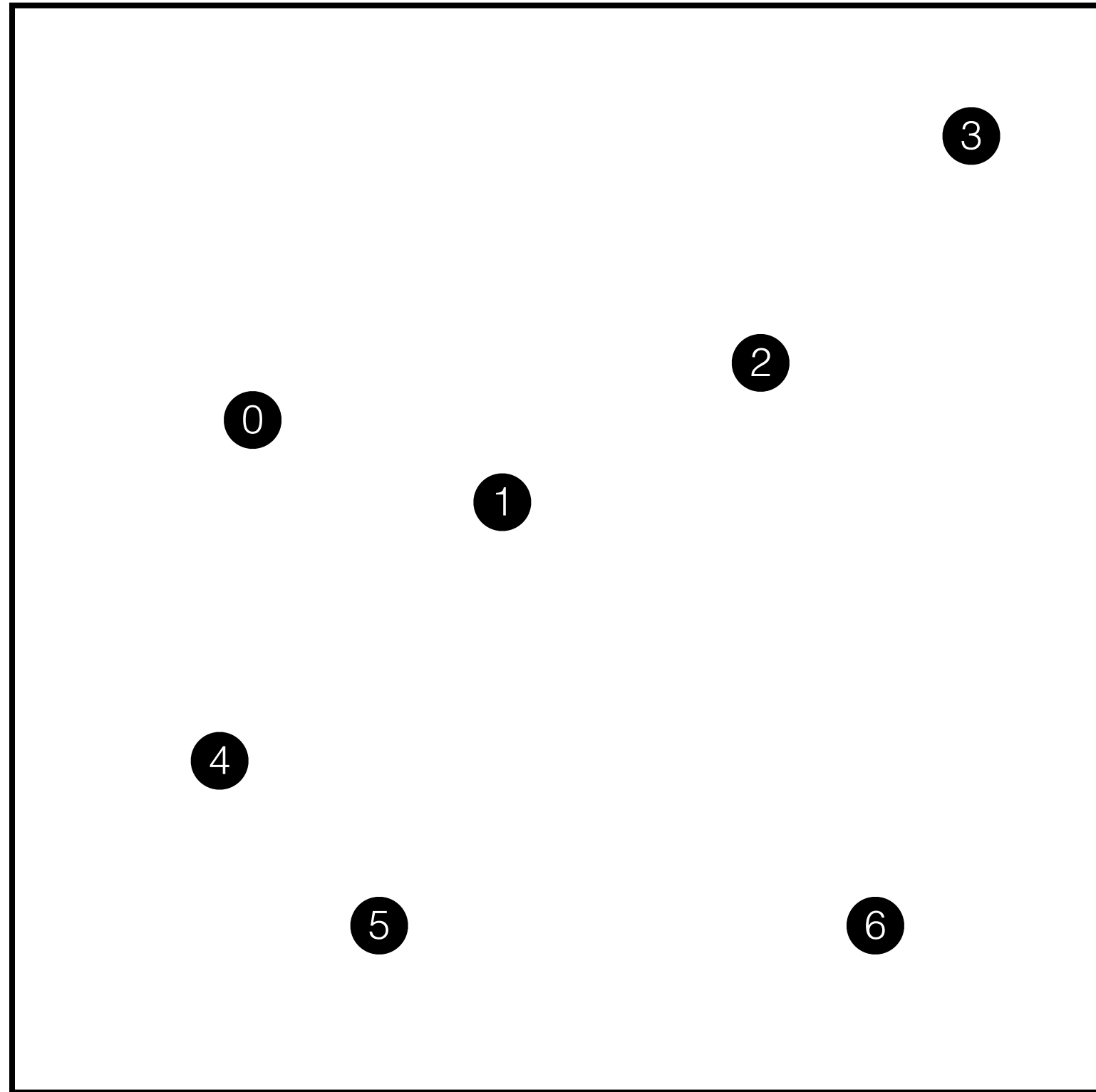


k-d tree is another method to create sub-divisions

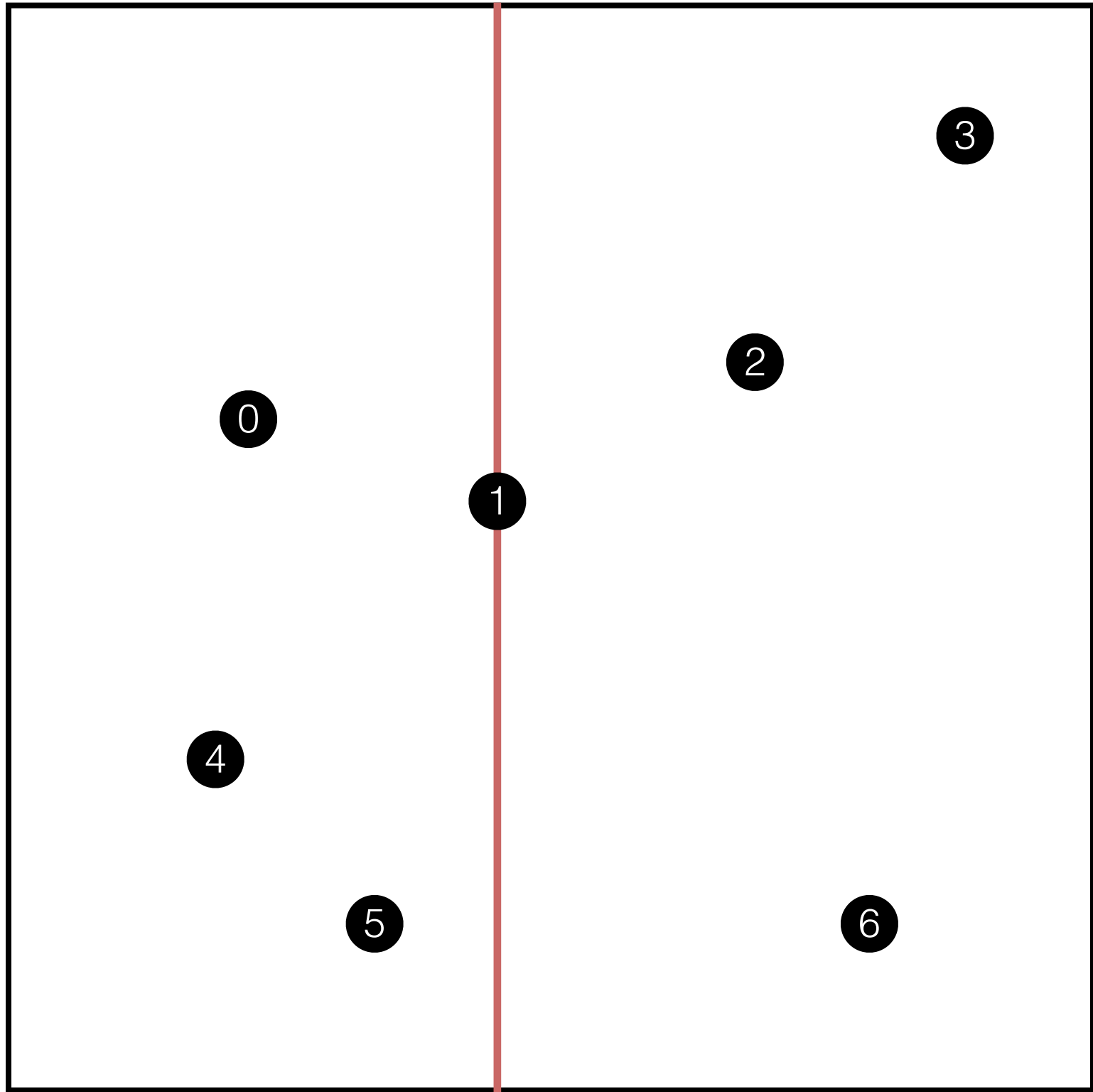
Subdivisions are made from **nodes** and **dimensions**



Taking a list of points...

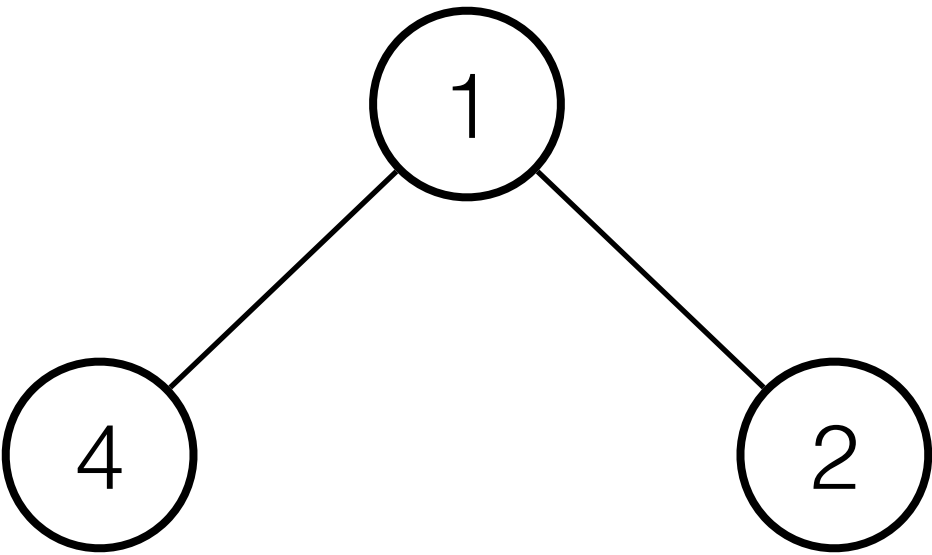
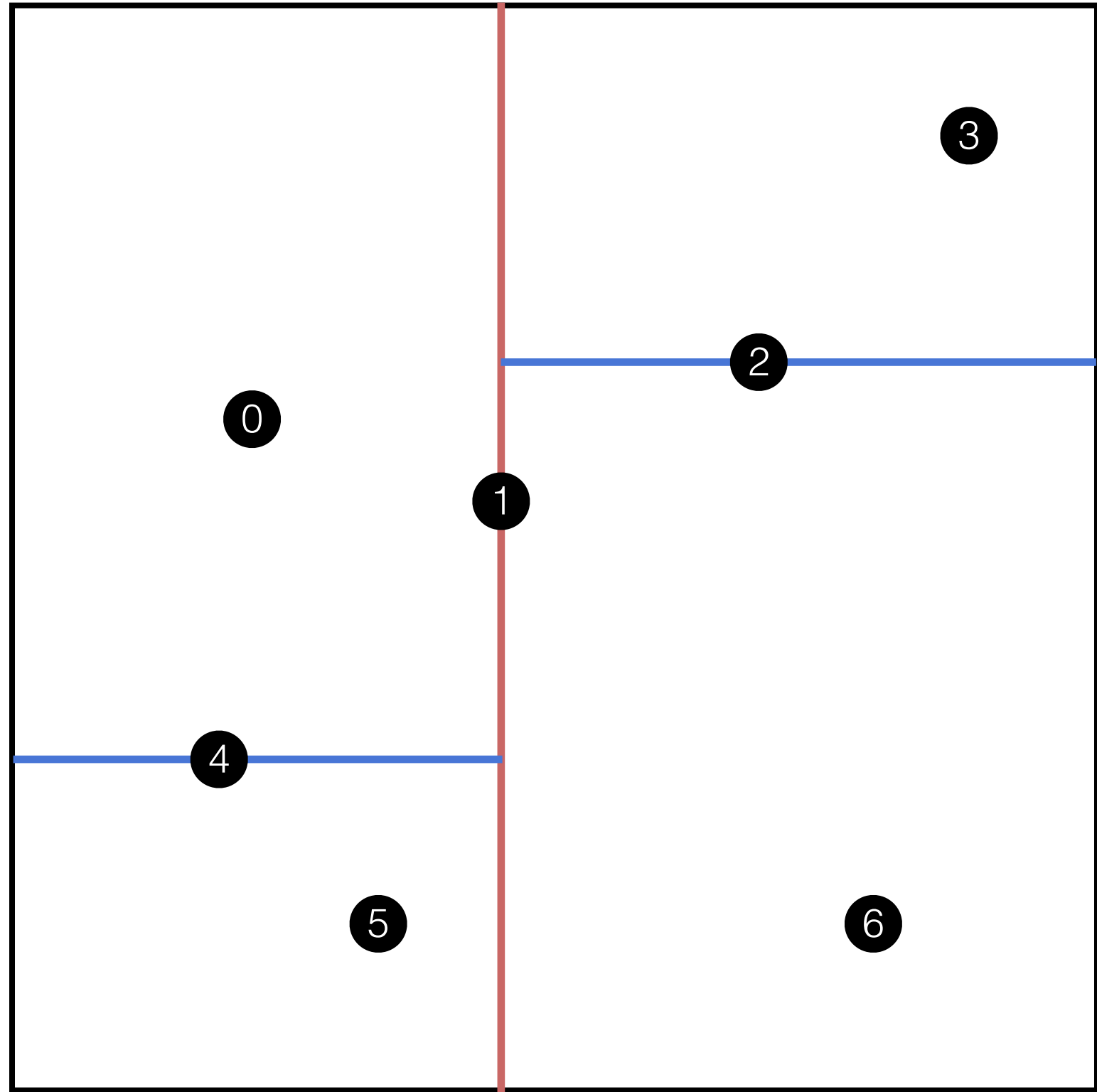


... we divide using the median on a dimension ...

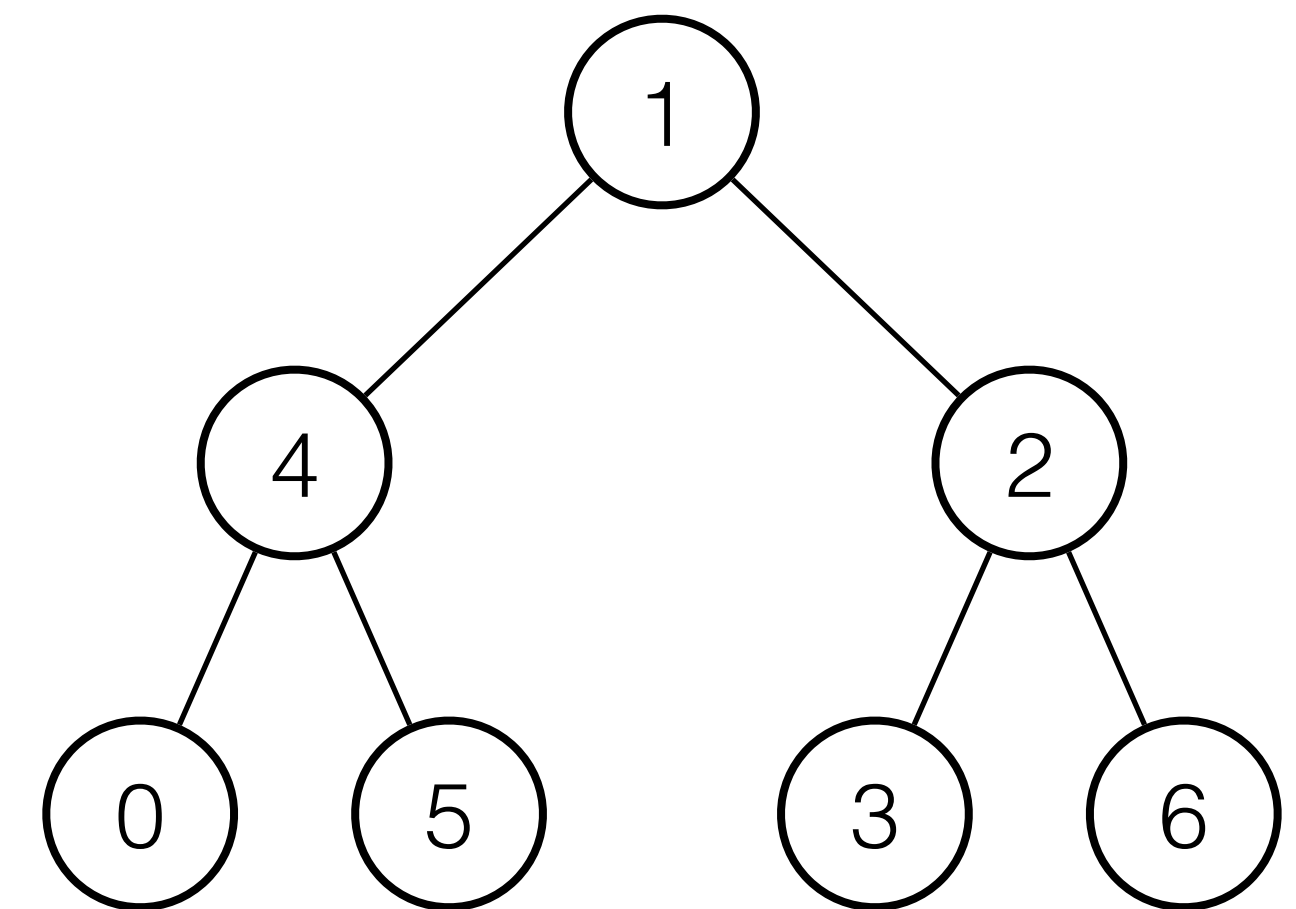
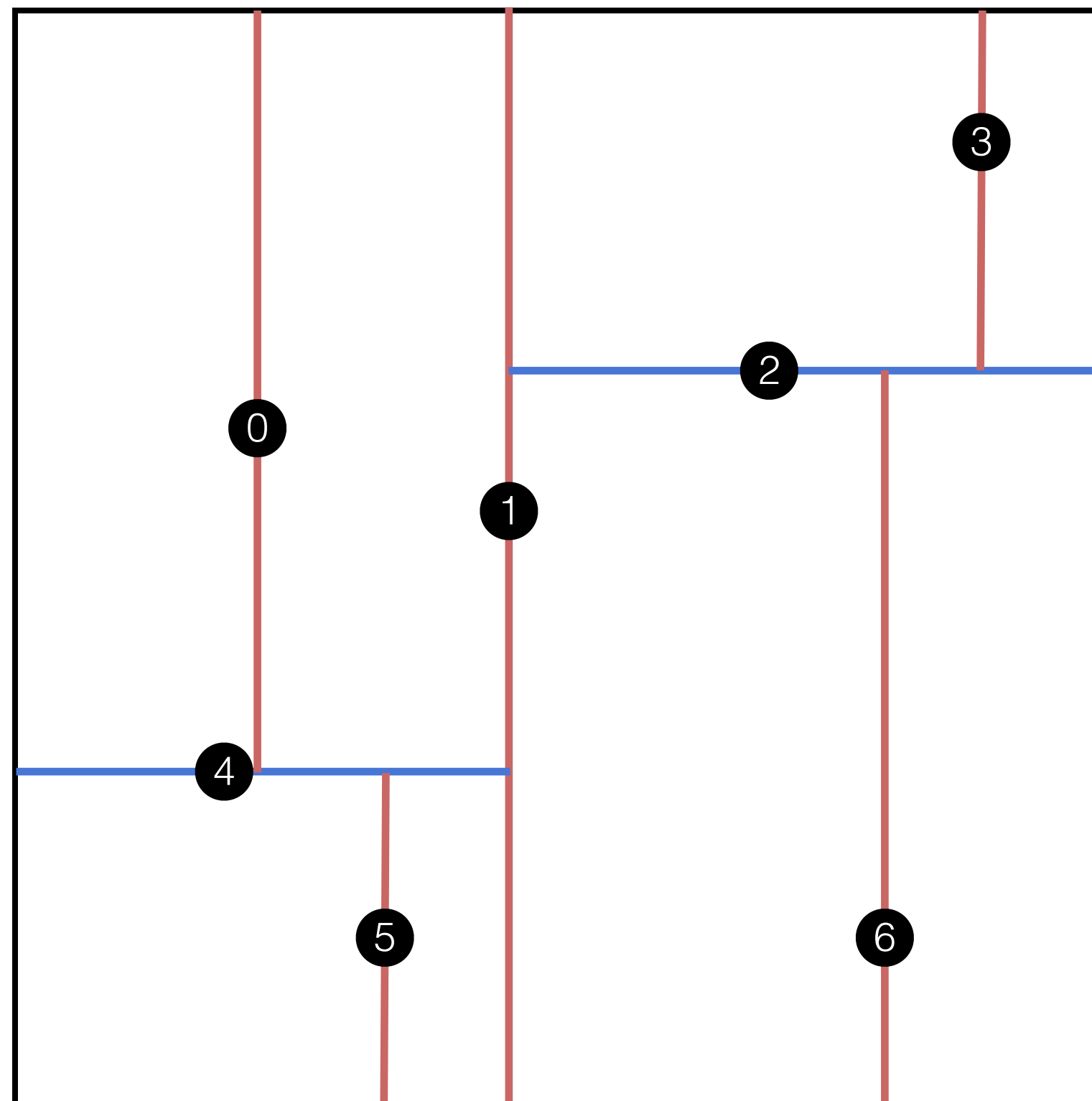


1

... alternating dimension each time...

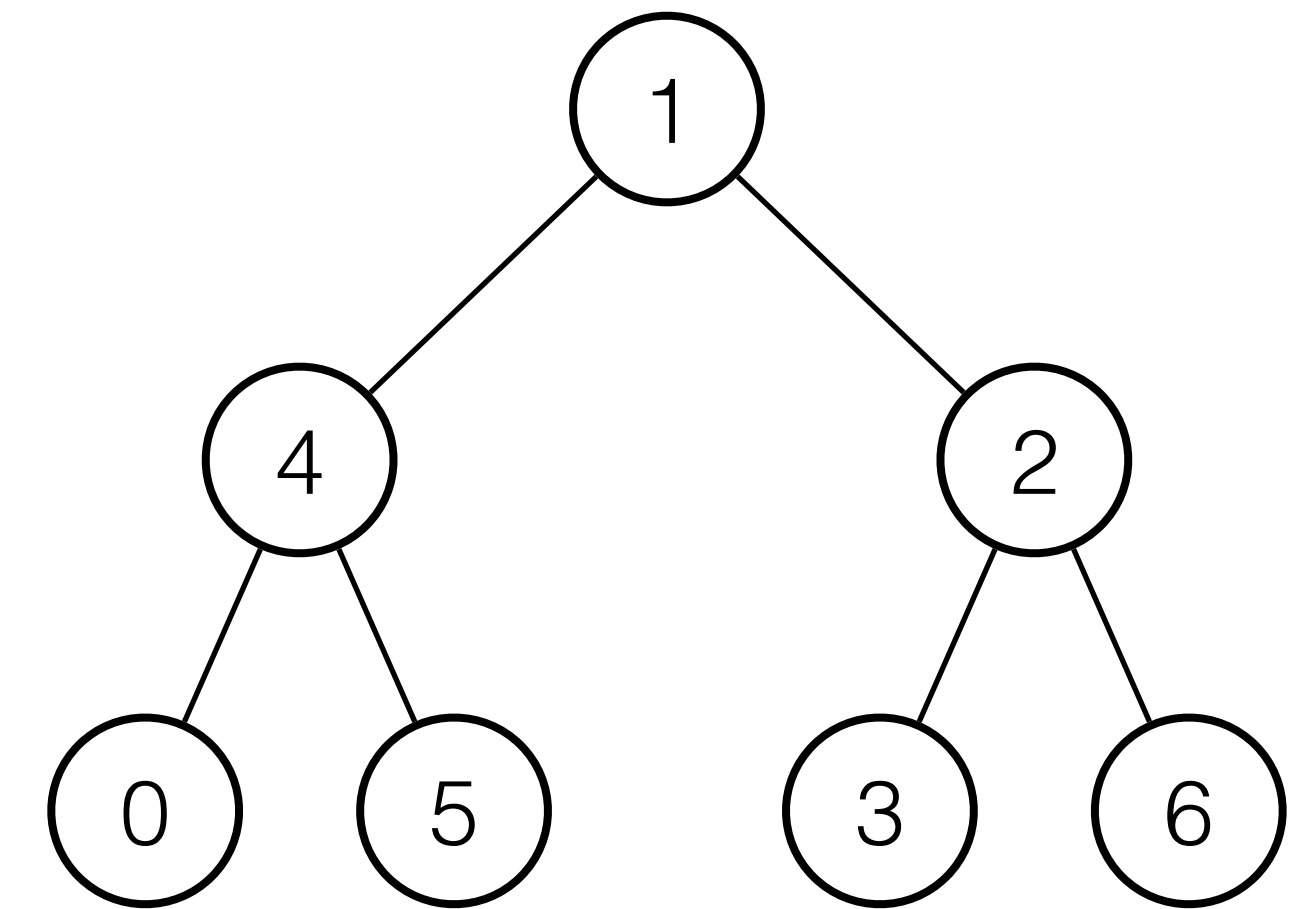
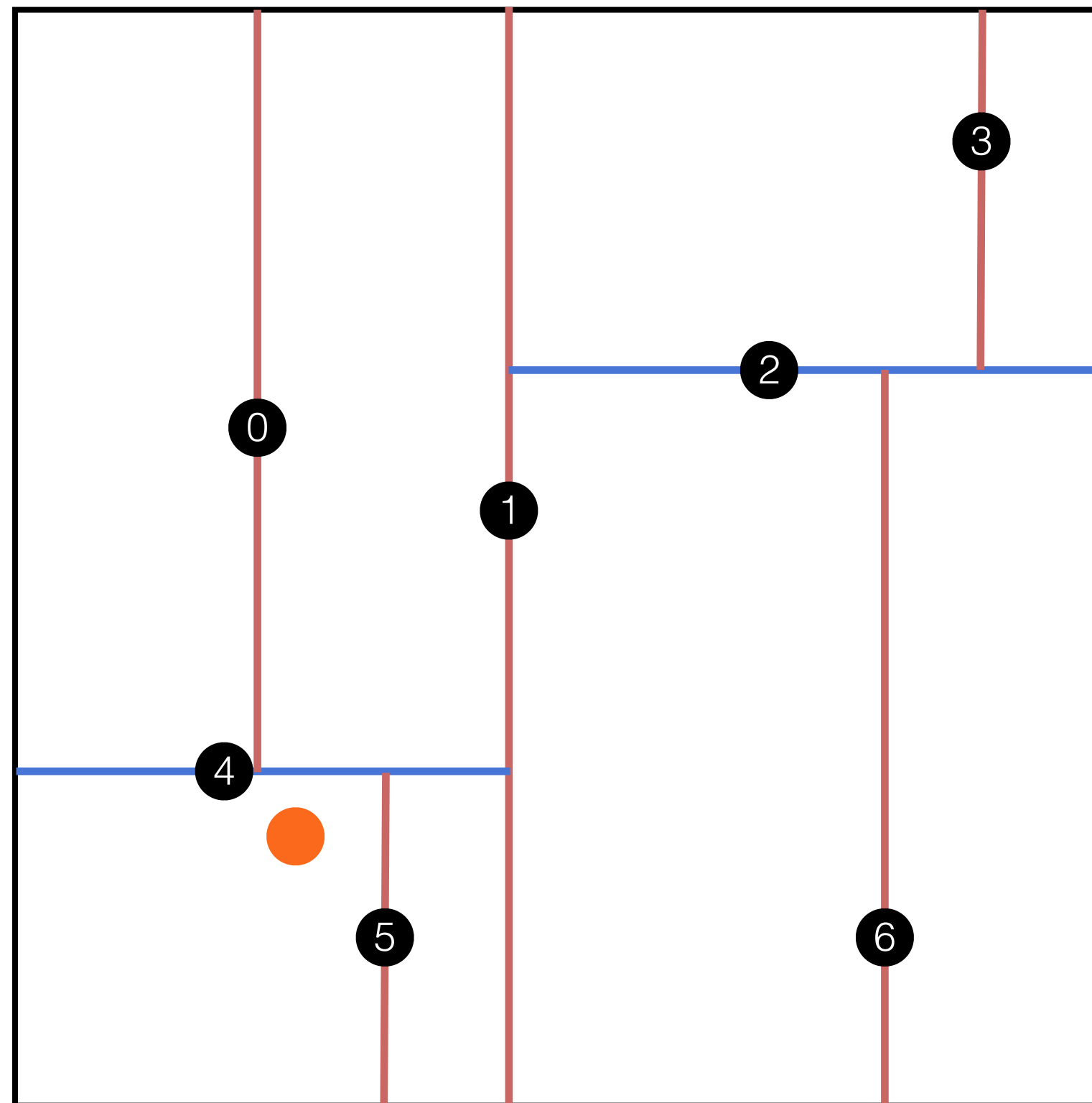


... until each point is in our tree

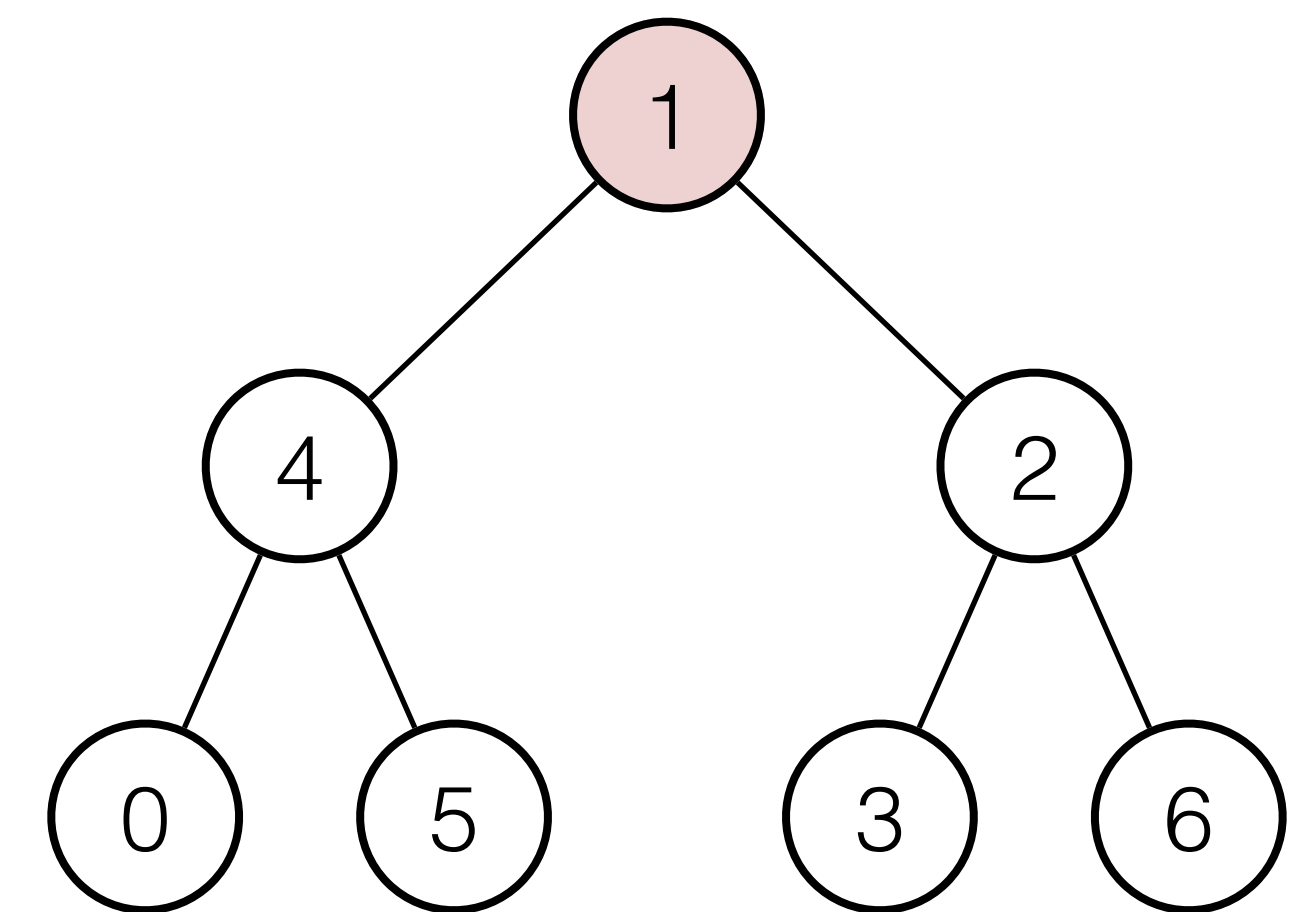
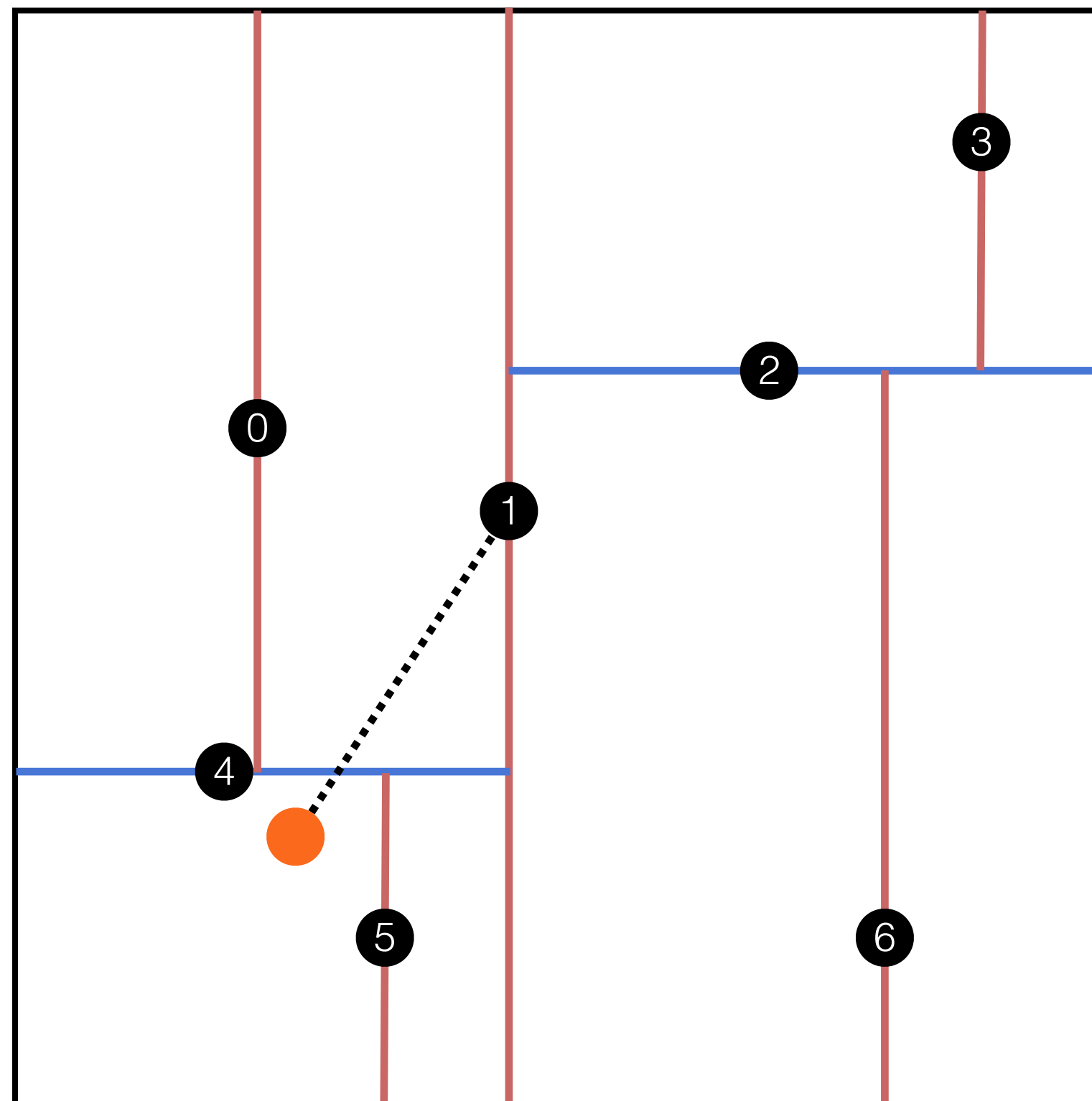


Application to the second problem

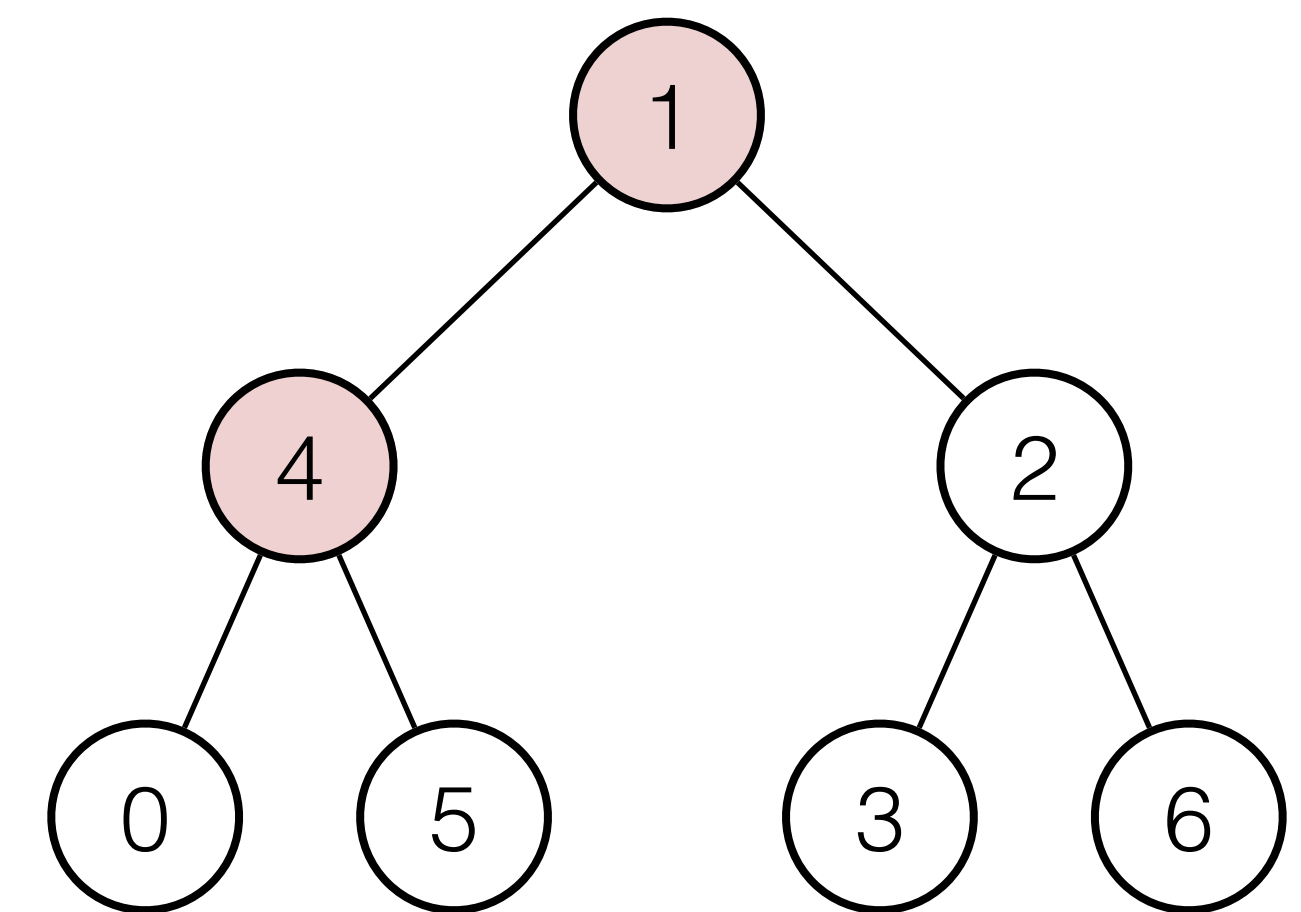
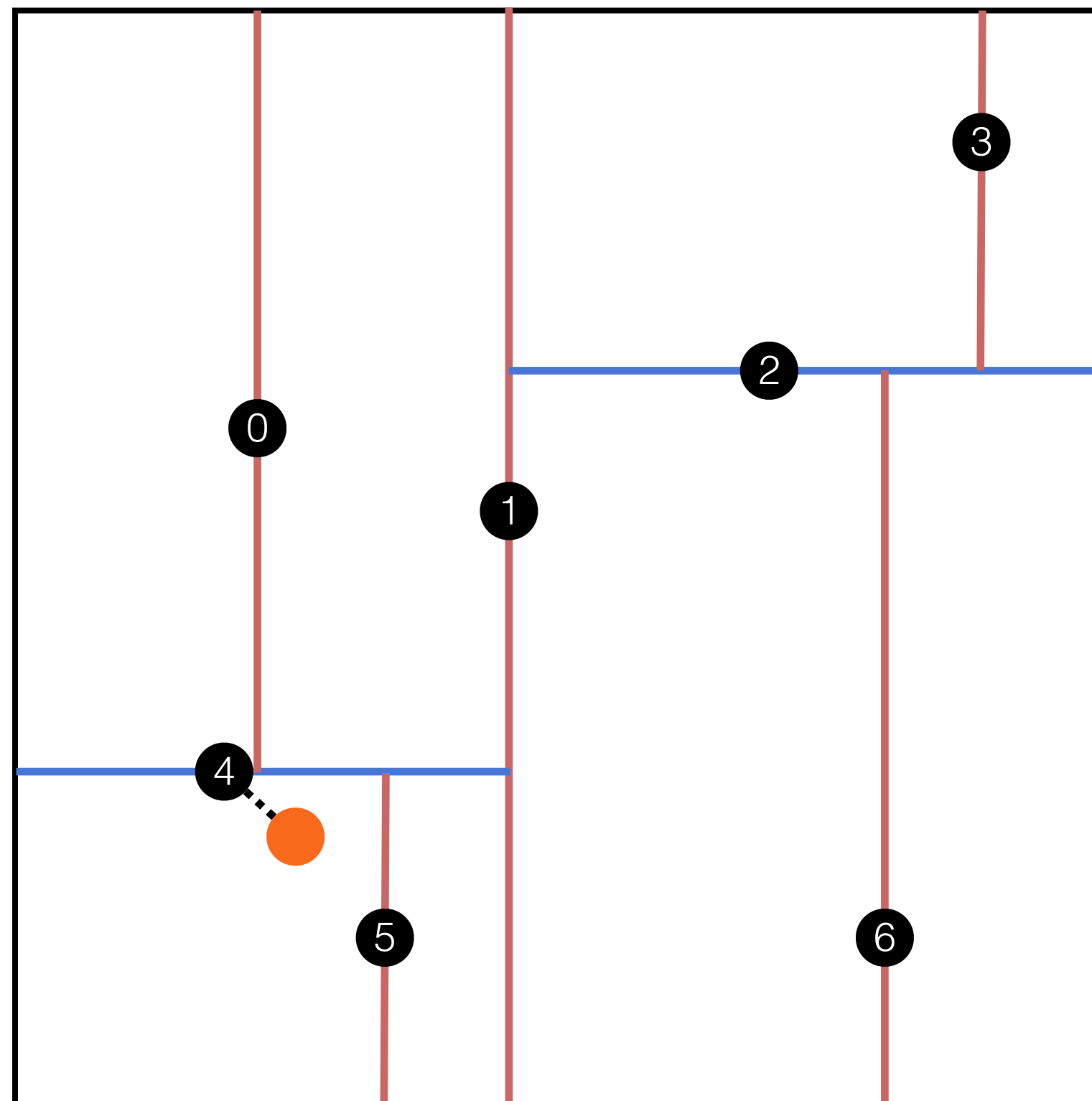
Who is the closest neighbor to the orange point ?



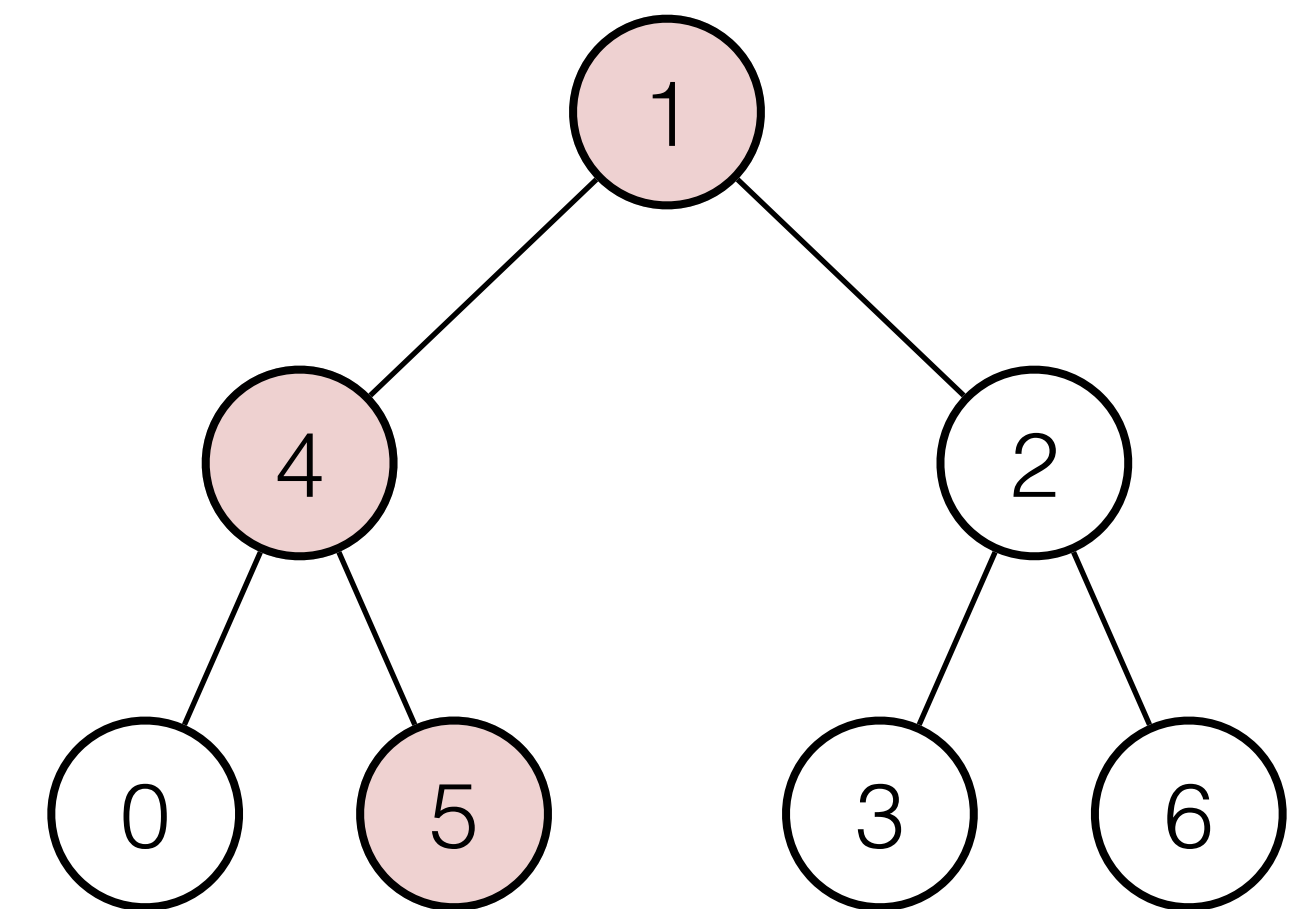
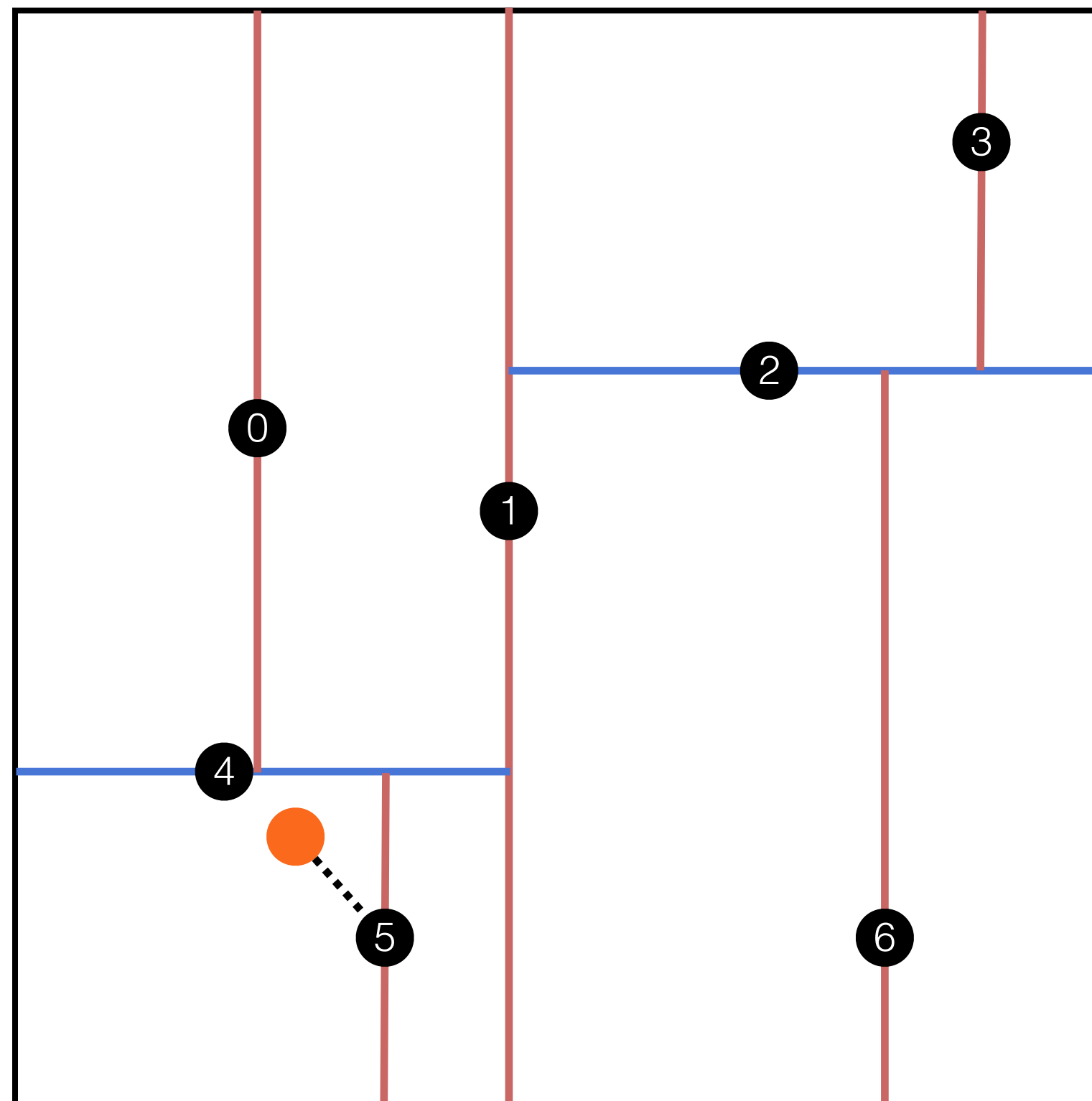
We go down our tree...



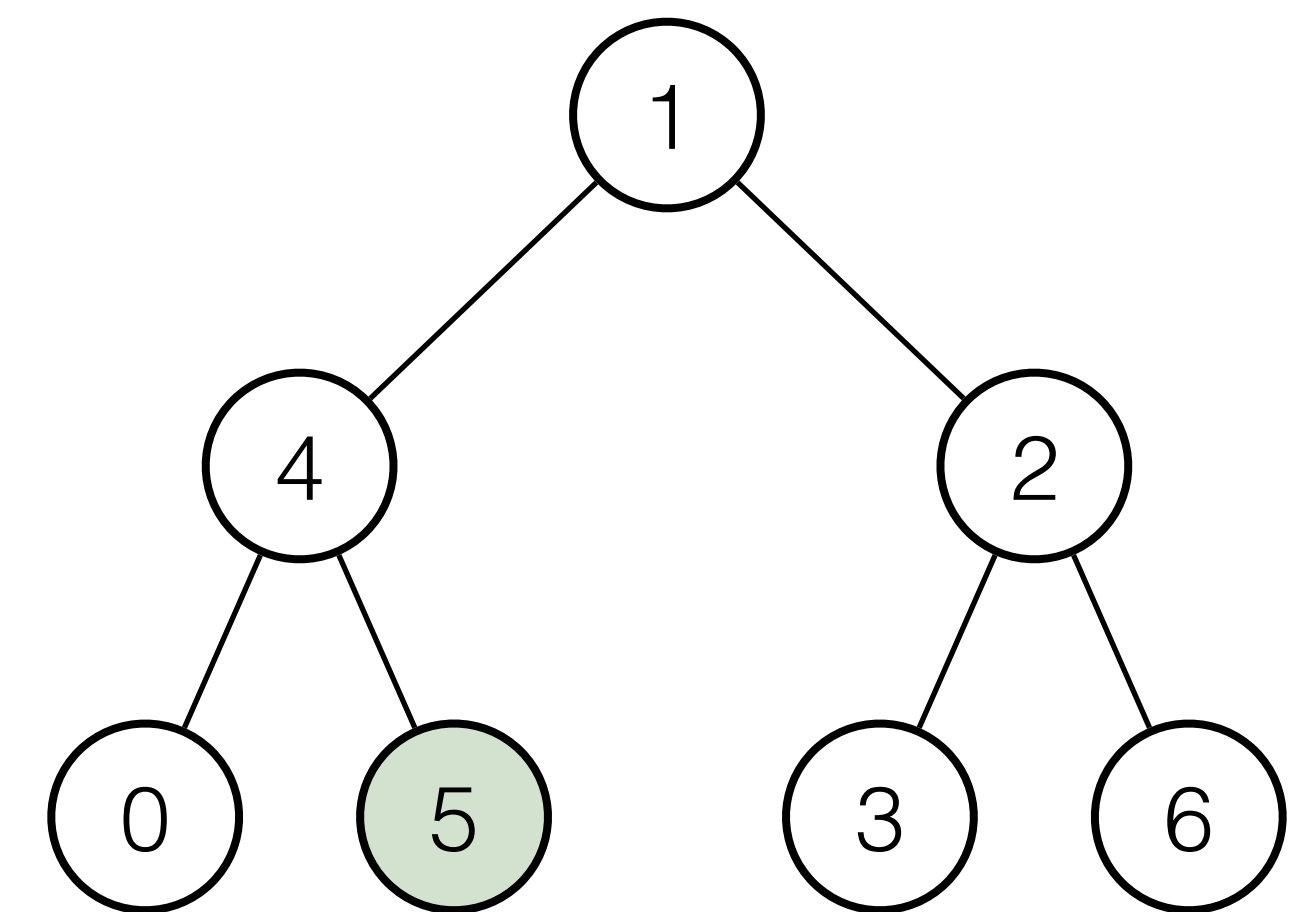
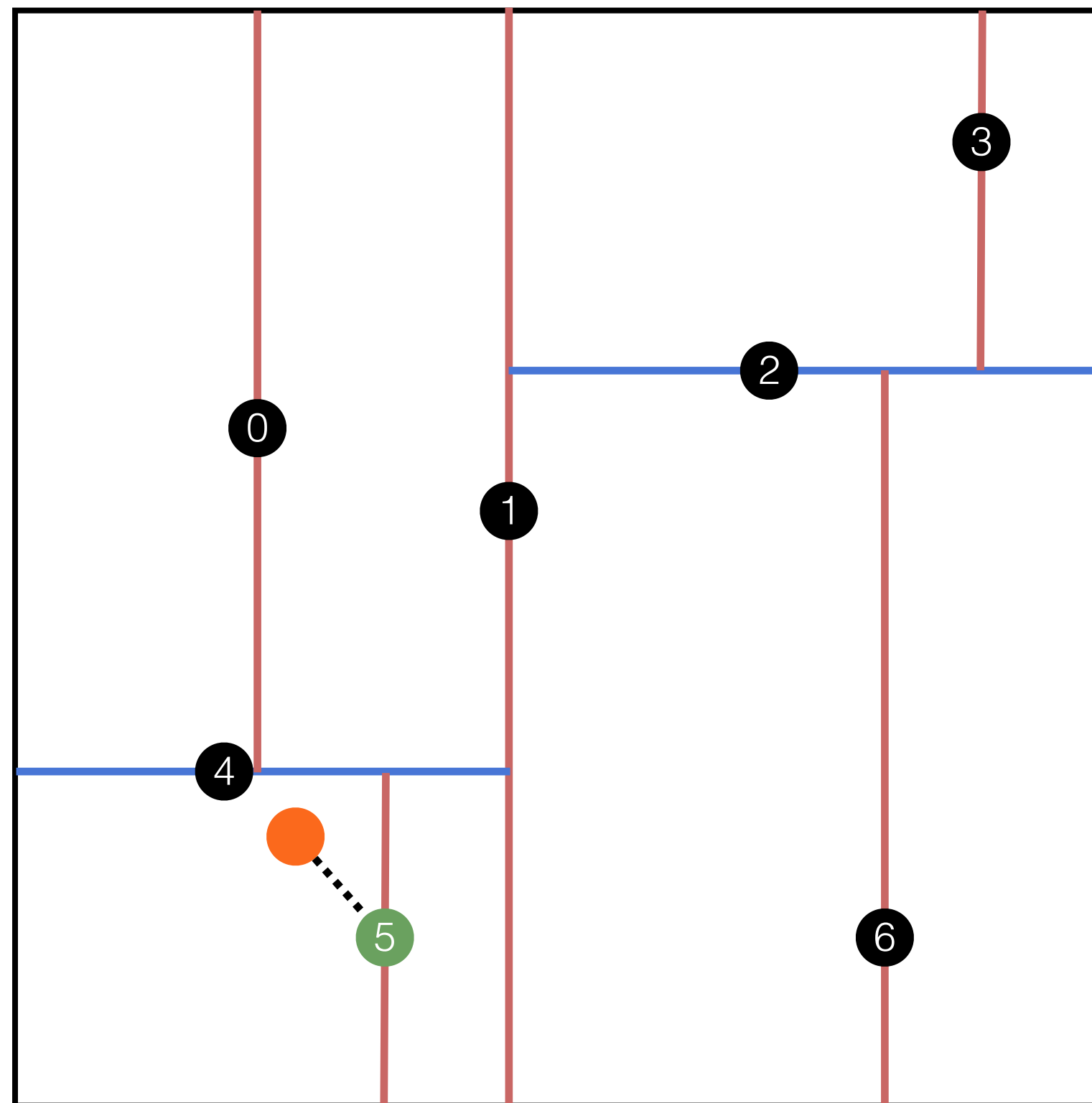
We go down our tree...



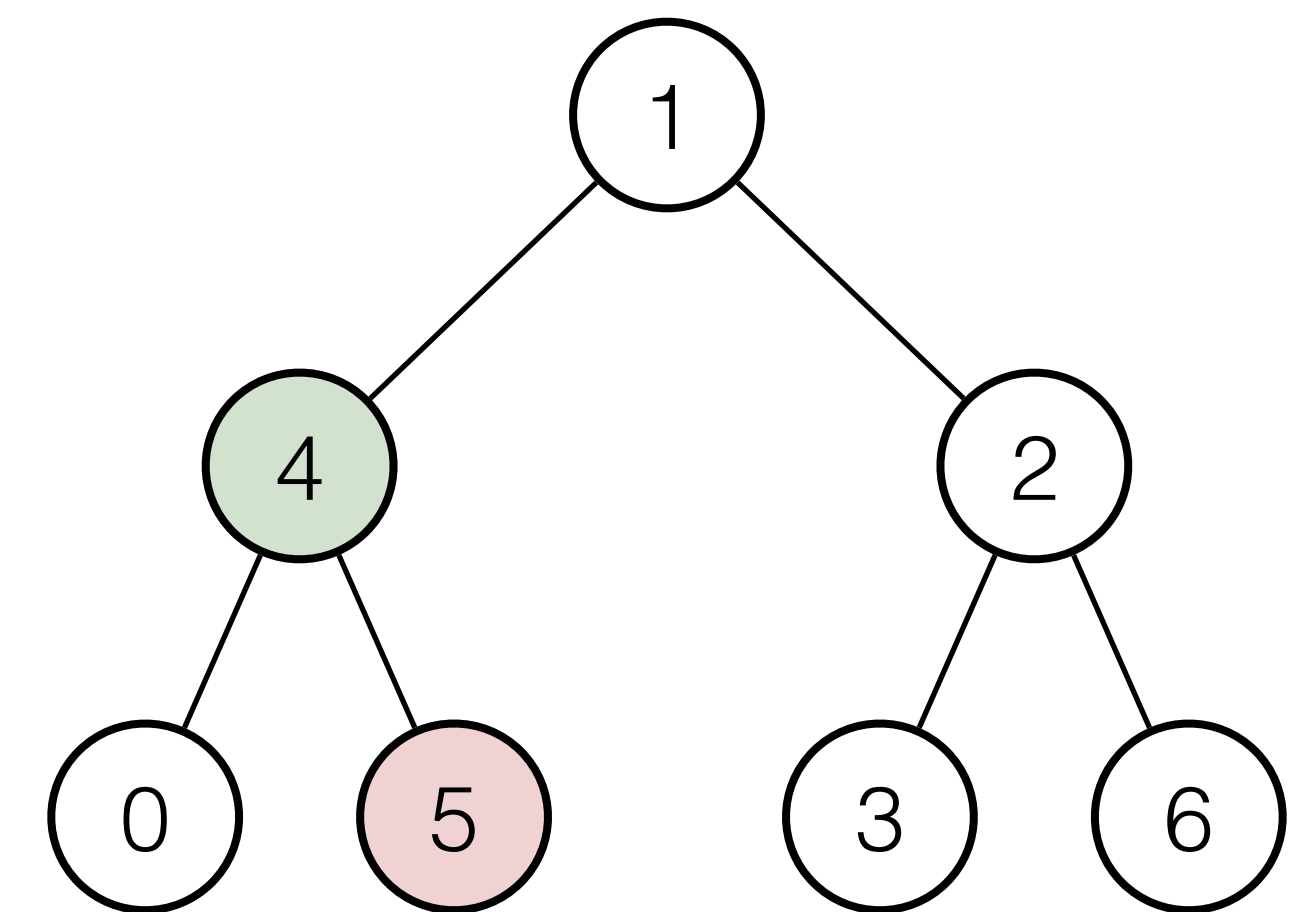
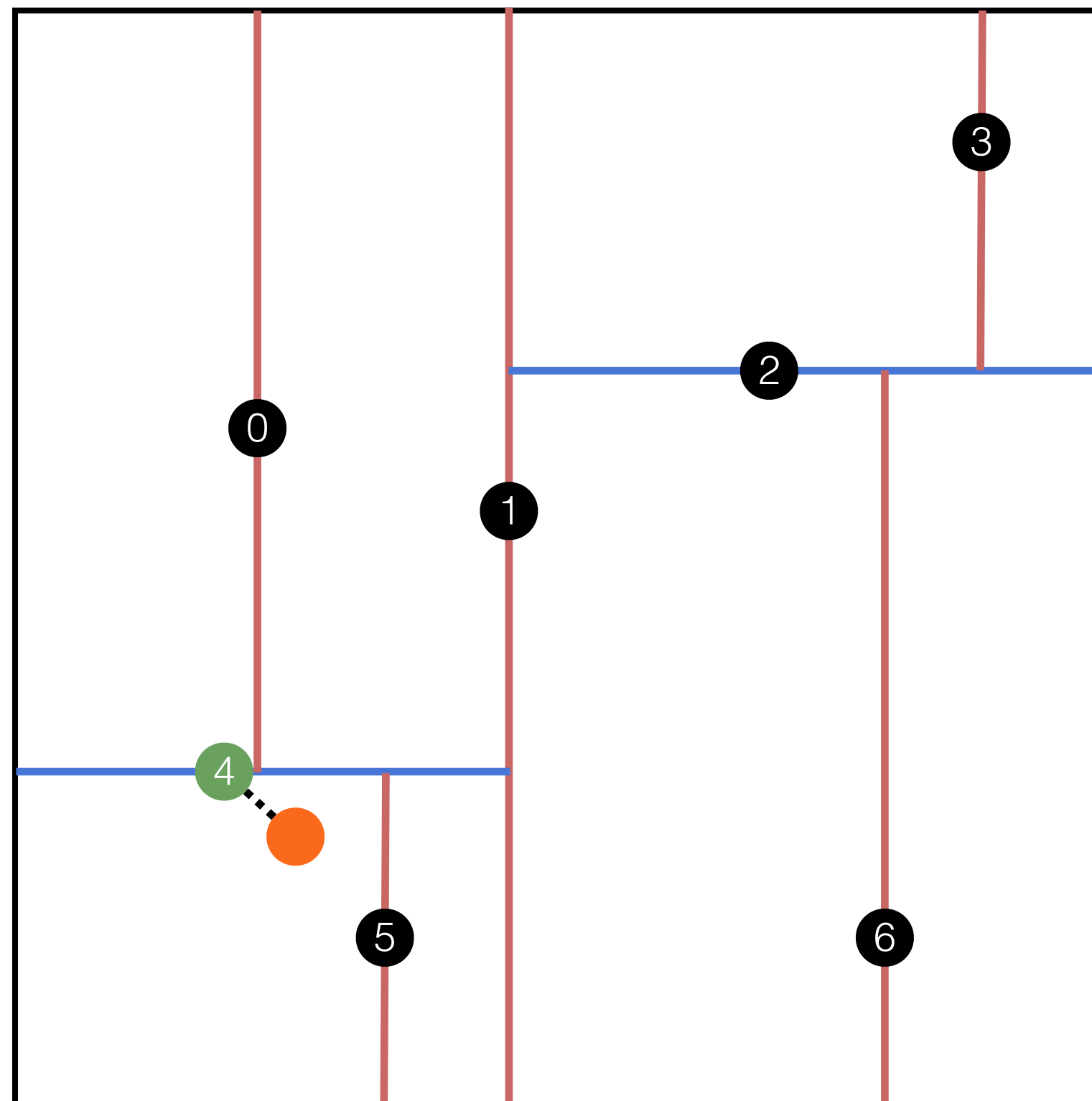
... until the bottom of our tree



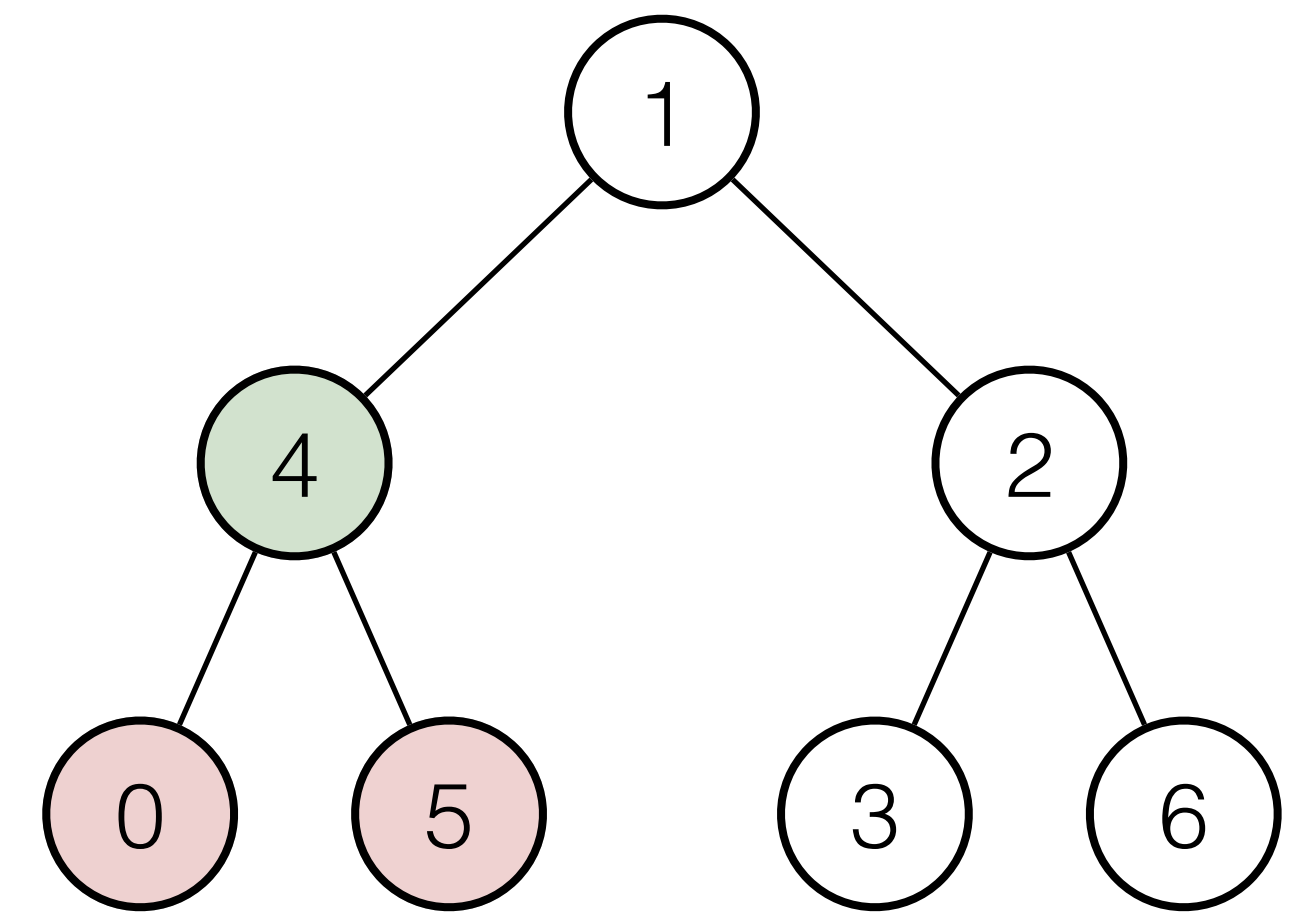
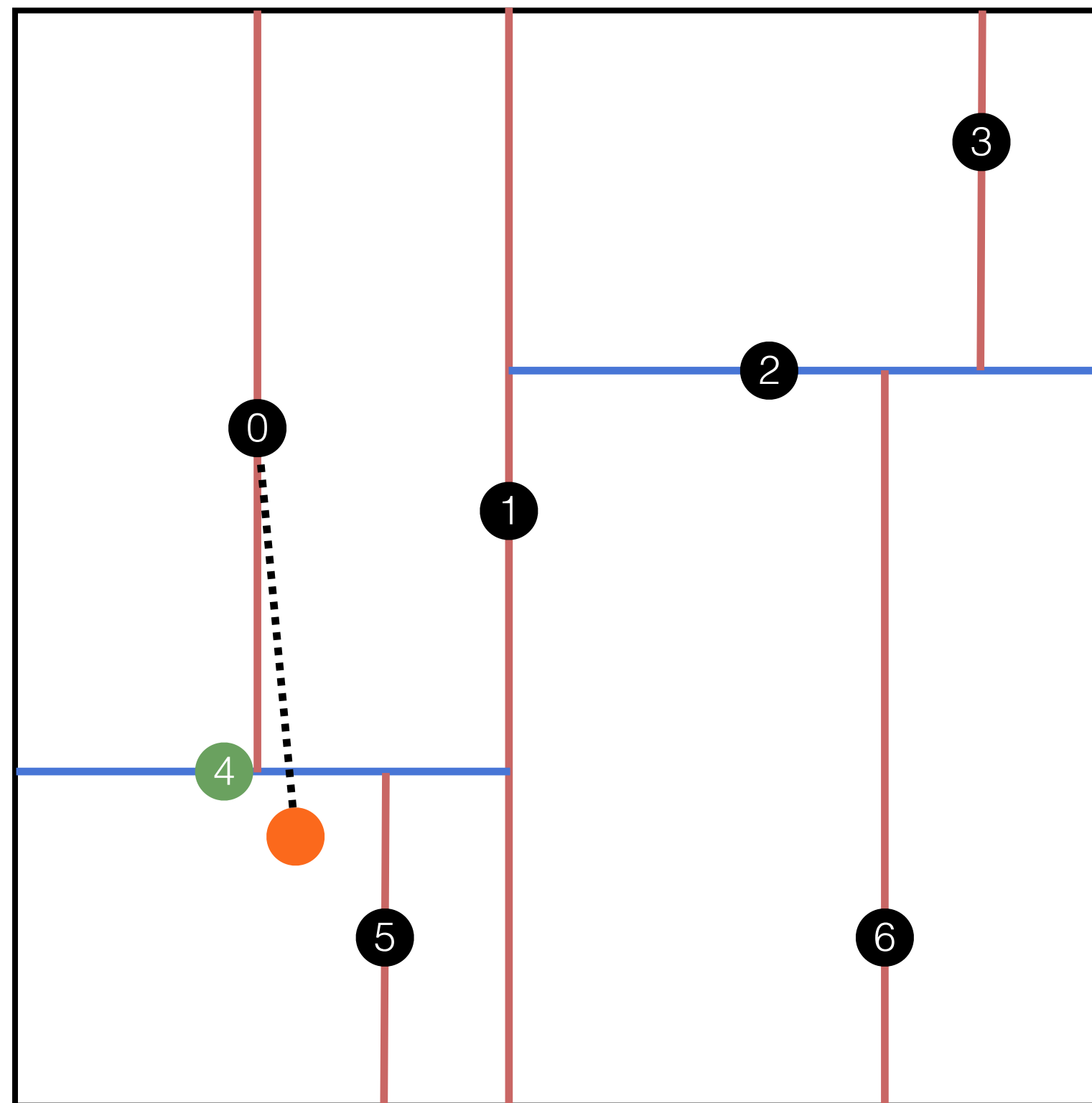
We use that point as our first guess...



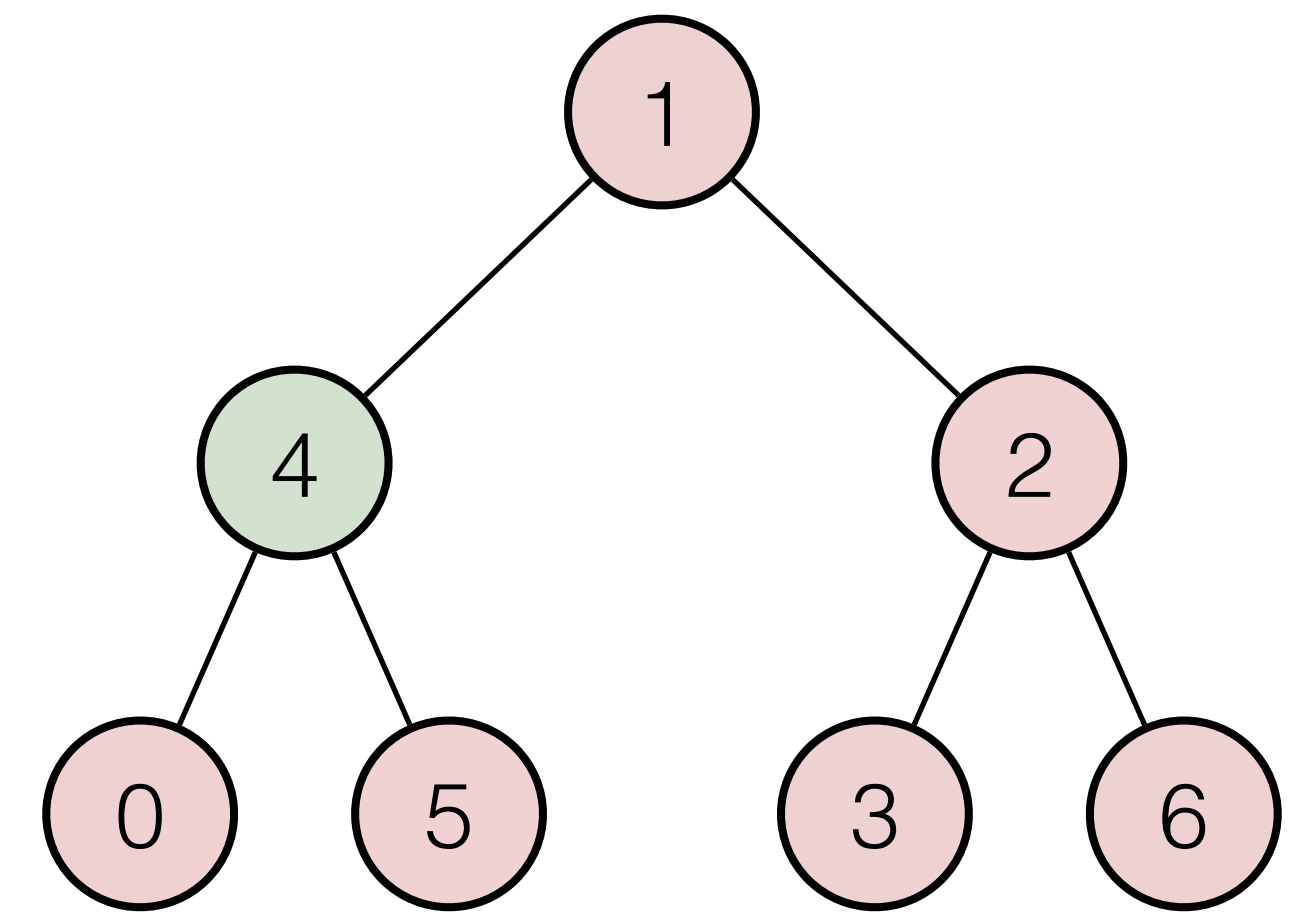
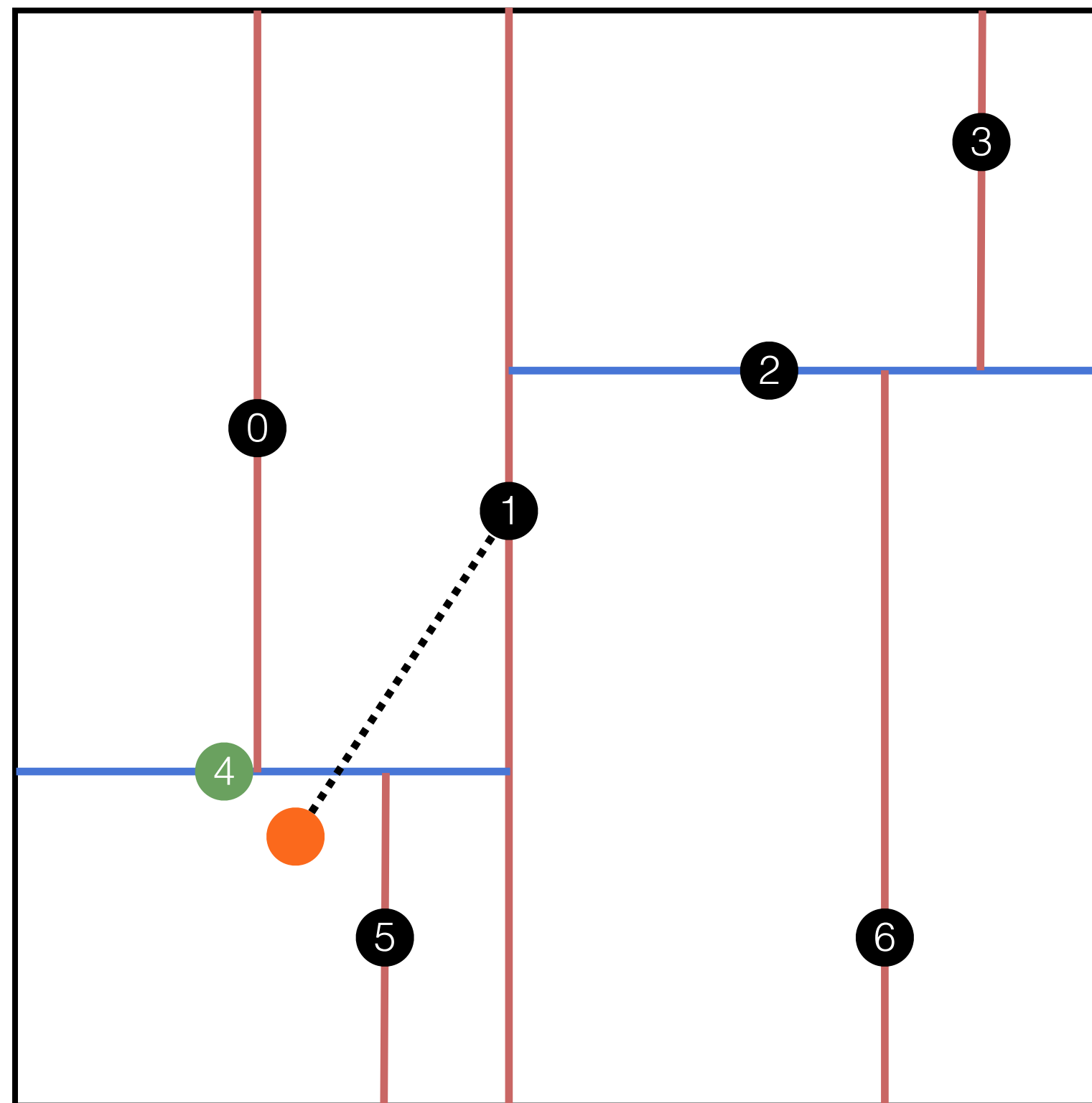
... and go up the tree to test if any point could be closer



We might need to test other branches...



... but we quickly restrict the space



In 2D, both methods can be used

kd-tree is better for **highly clustered** data (and for **higher dimensions**)

Quadtree is more **flexible** (can change objects more easily without a full reindexing)

Recap, and breather 🥱

- A space-driven spatial index uses a **partitioning strategy** to decompose a 2D plane into cells
- The most common methods are with are a **fixed grid index**, a **quadtree**, and a **k-d tree**
- Cells are **contiguous** and **not overlapping**

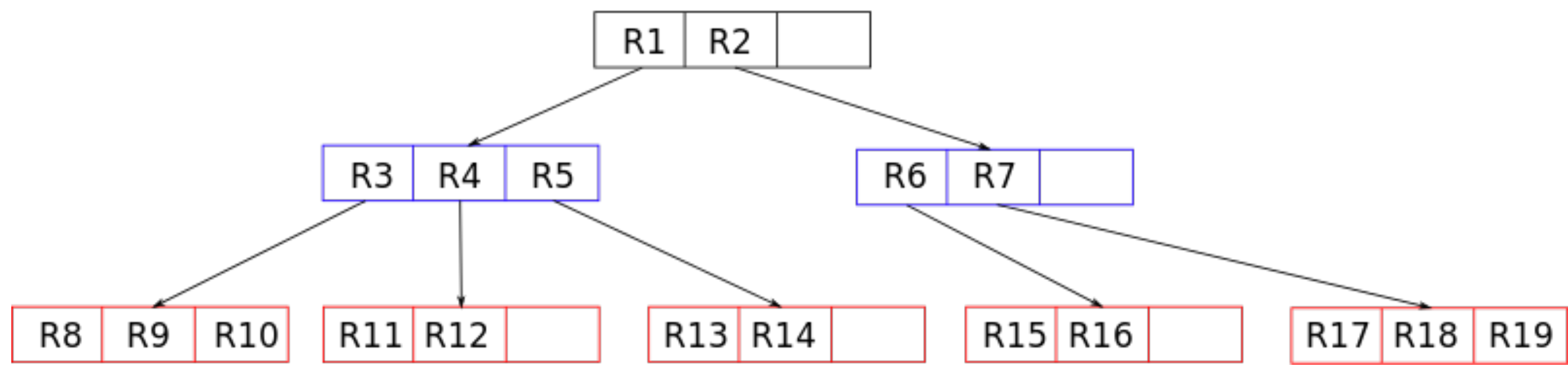
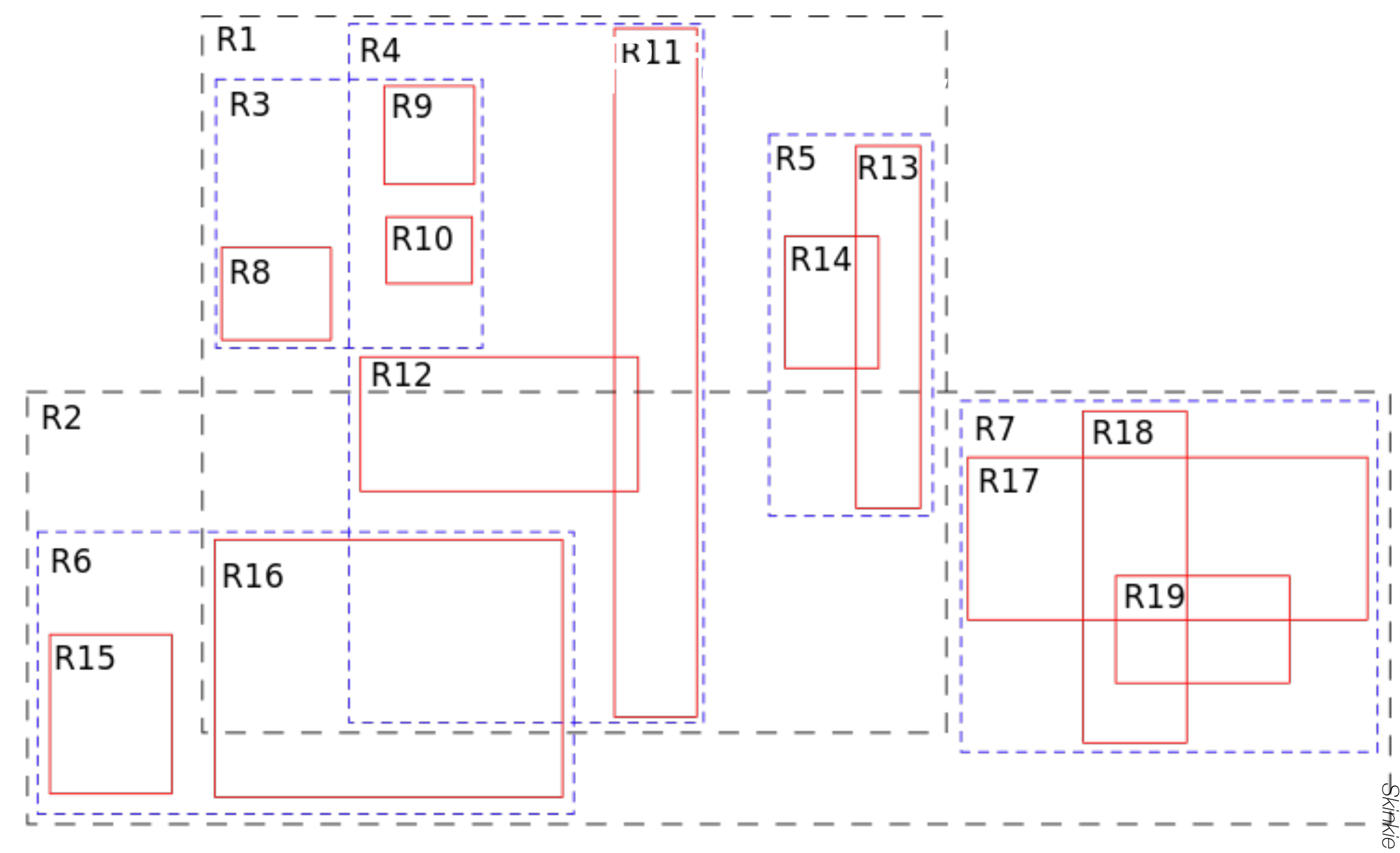
Data-driven Spatial Indexes

...uses spatial containment relationships based on hierarchies of minimum bounding boxes.

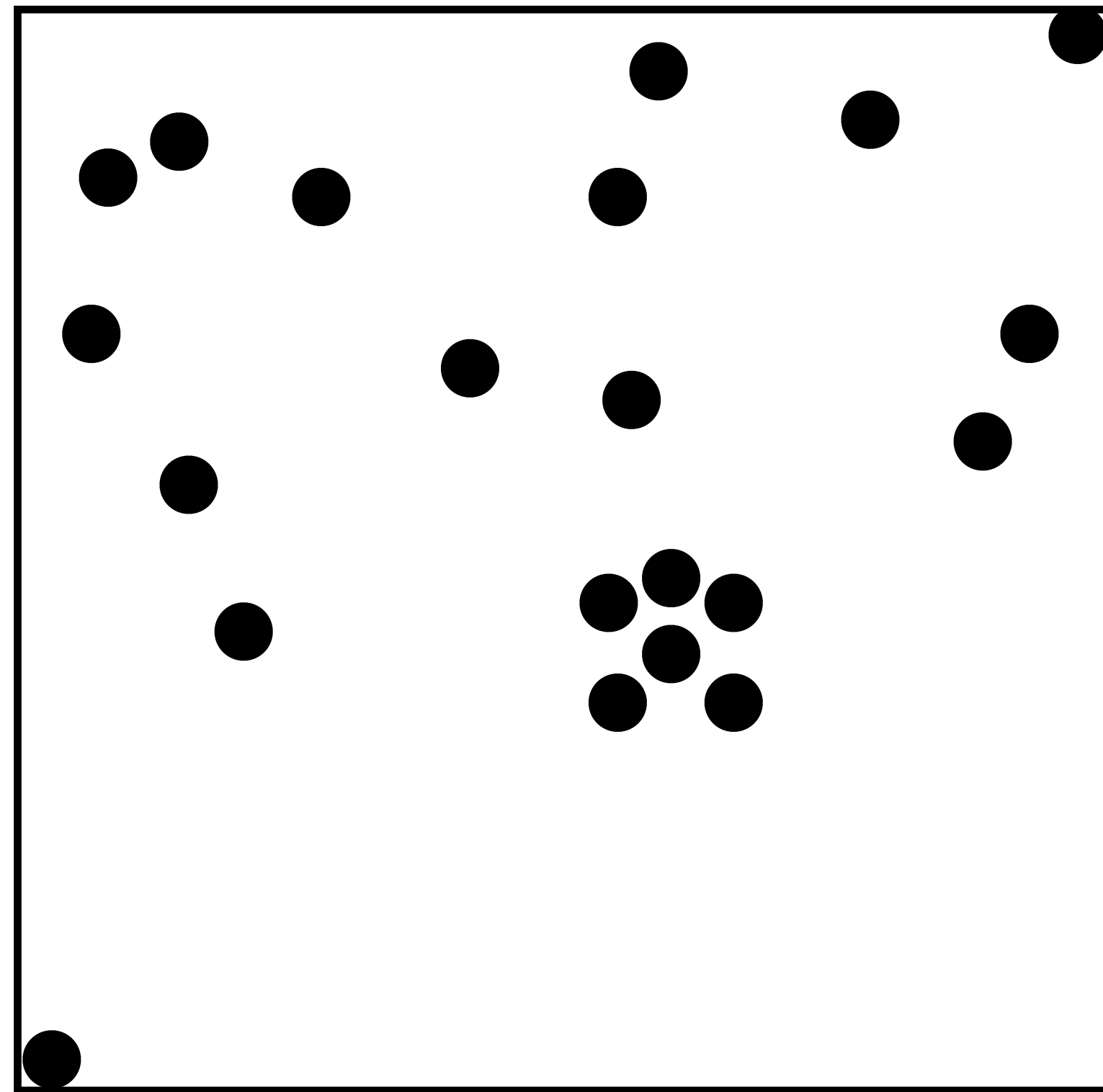
What is a **r-tree** ?

Data structure invented
by (Guttman, 1984)
grouping objects in
their **Minimum
Bounding Box**

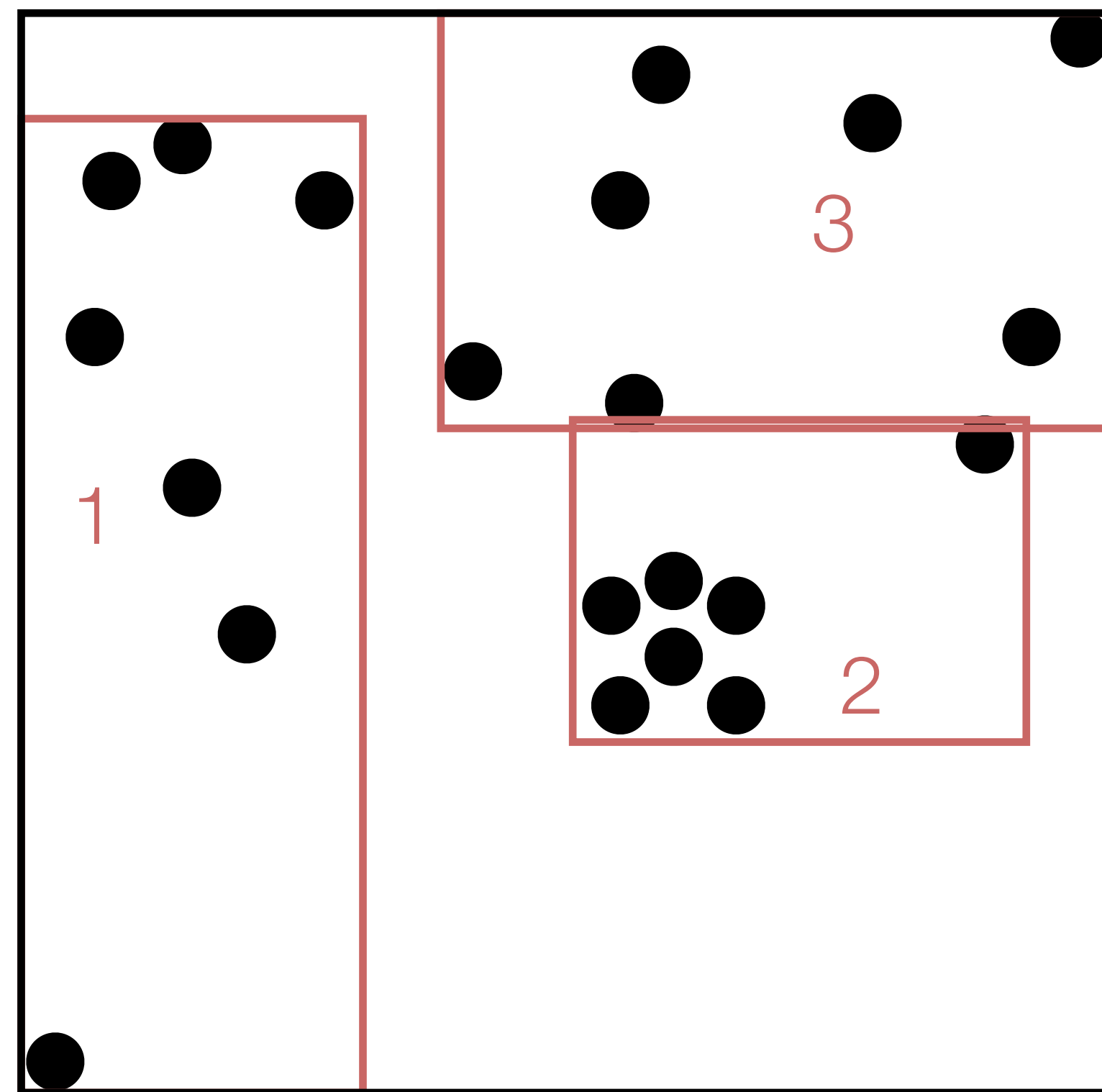
r → rectangle



We divide the space in MBBs...

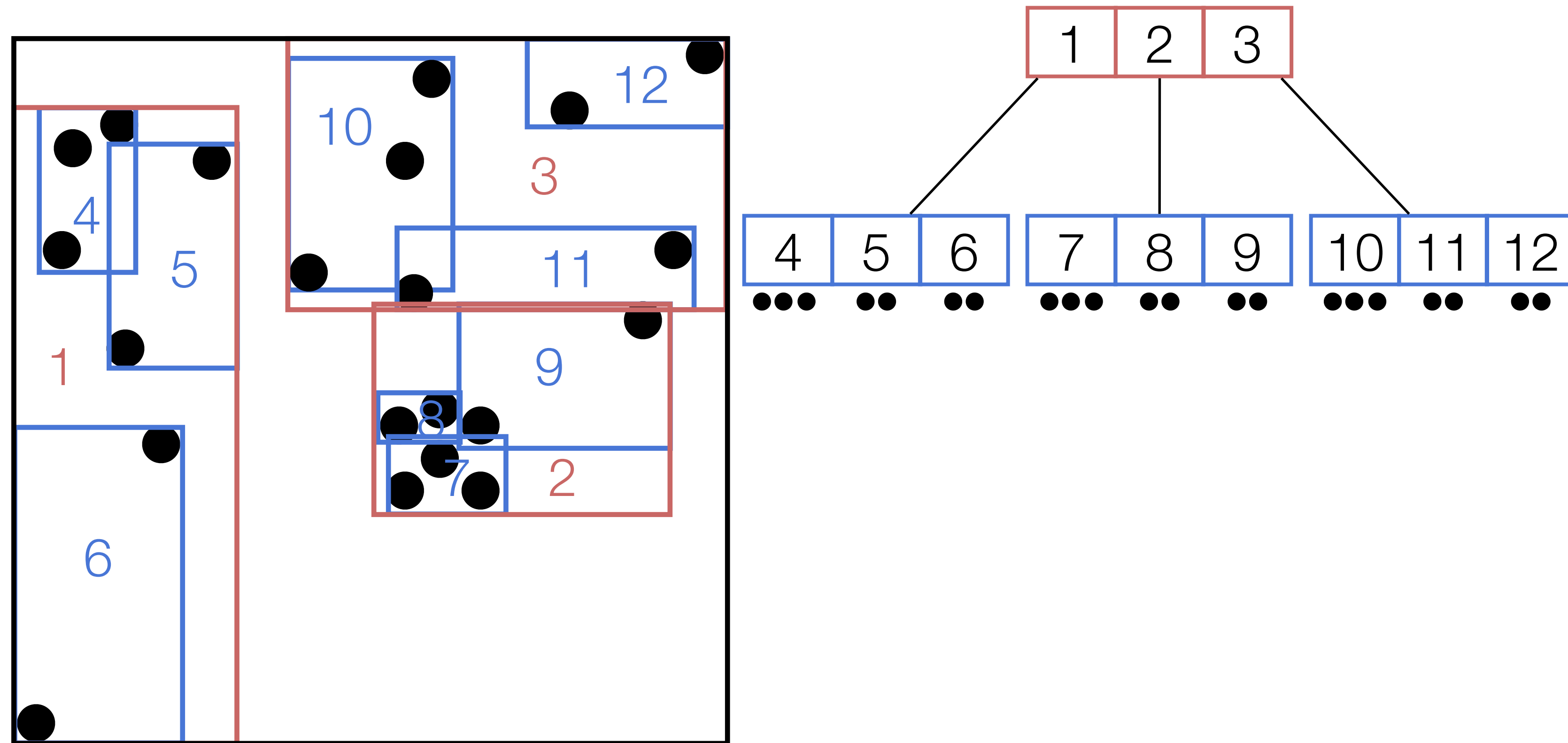


We divide the space in MBBs...

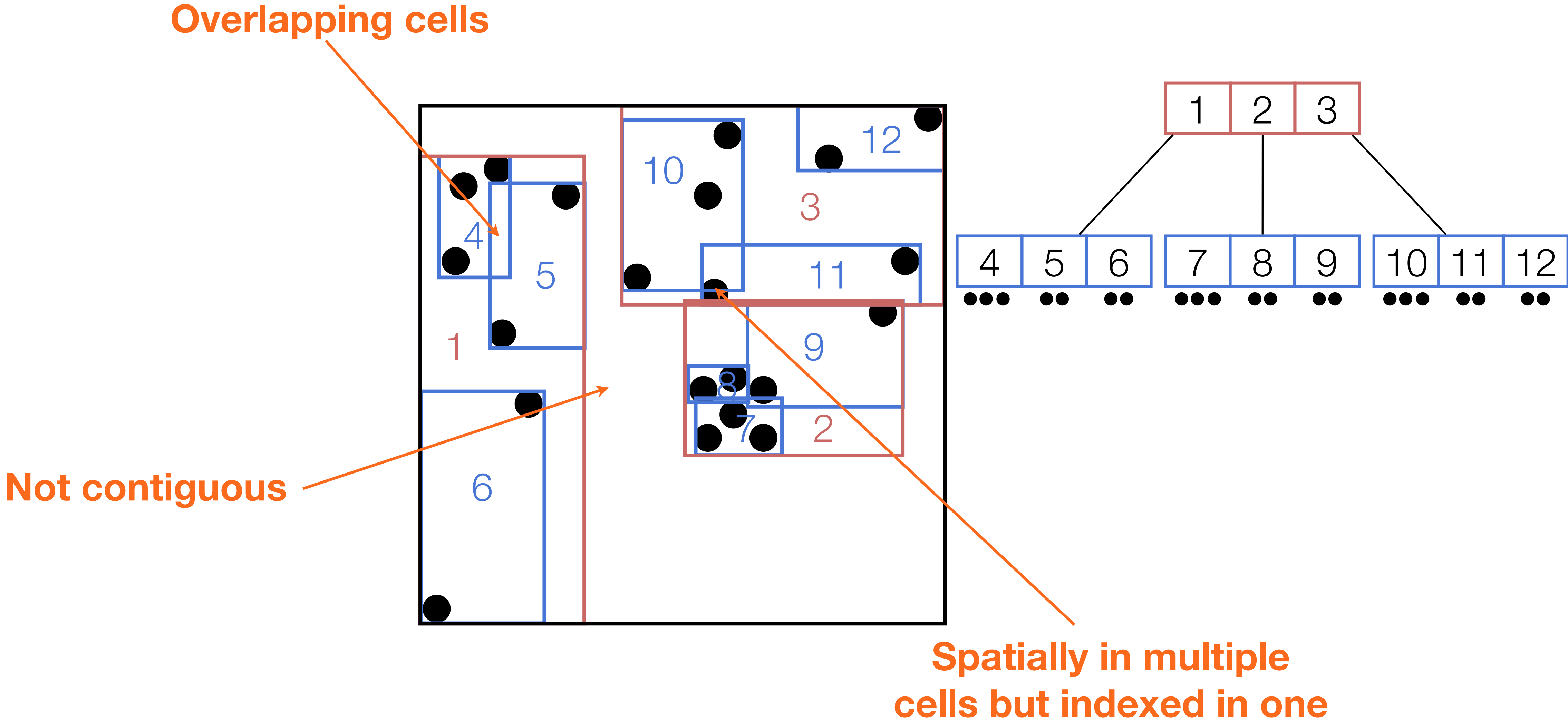


1	2	3
---	---	---

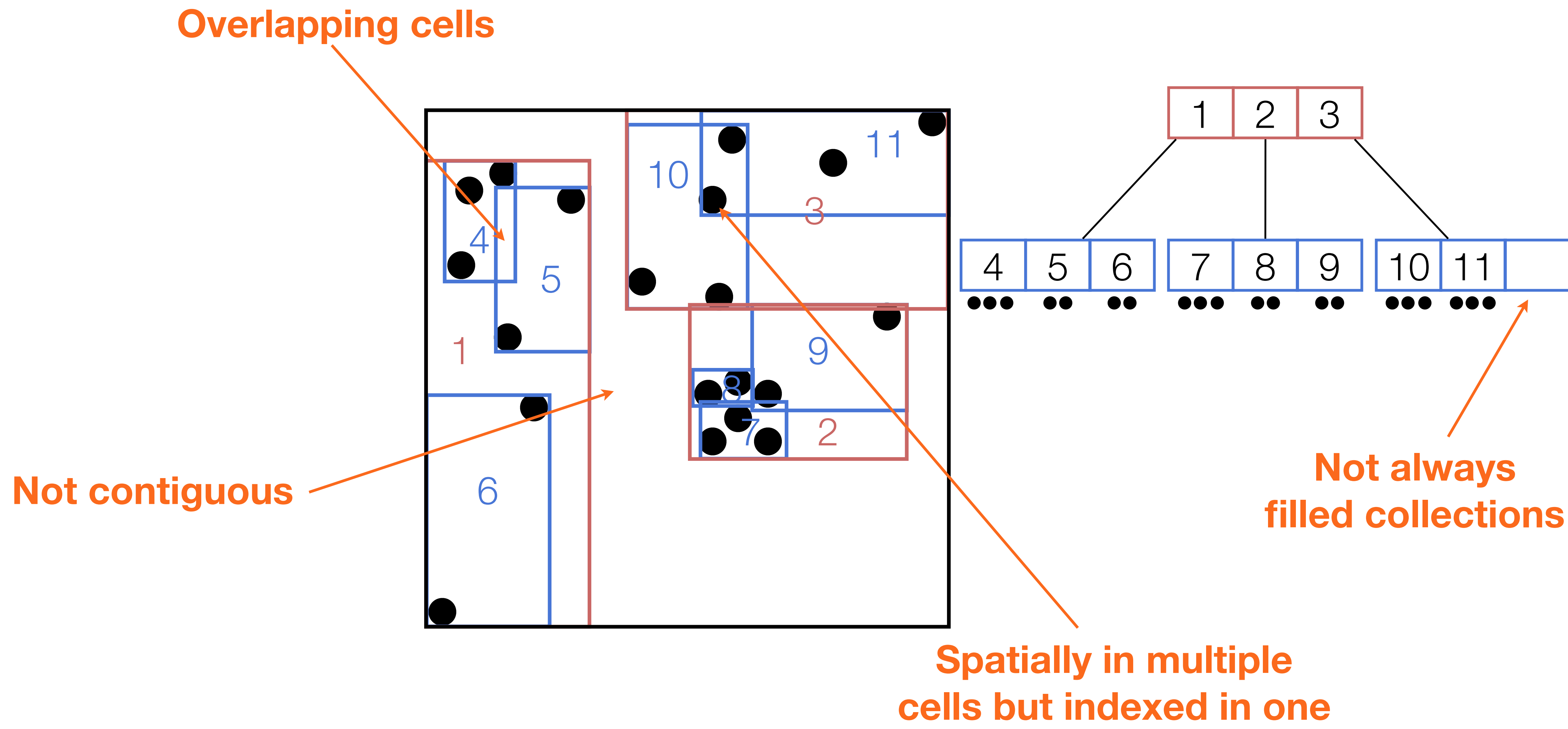
... until there is only a **threshold** of objects in each MBBs



This is not space-driven !

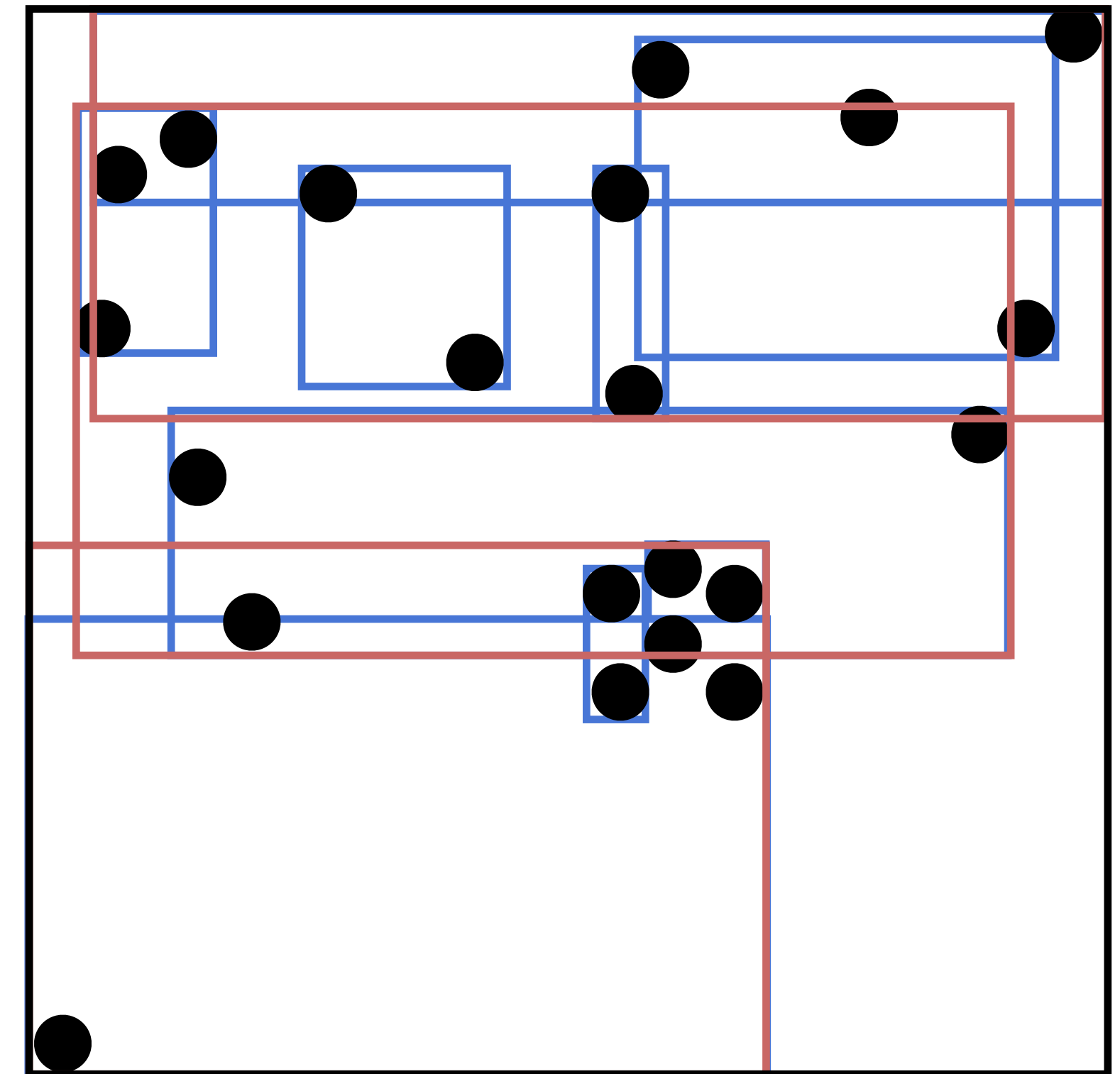
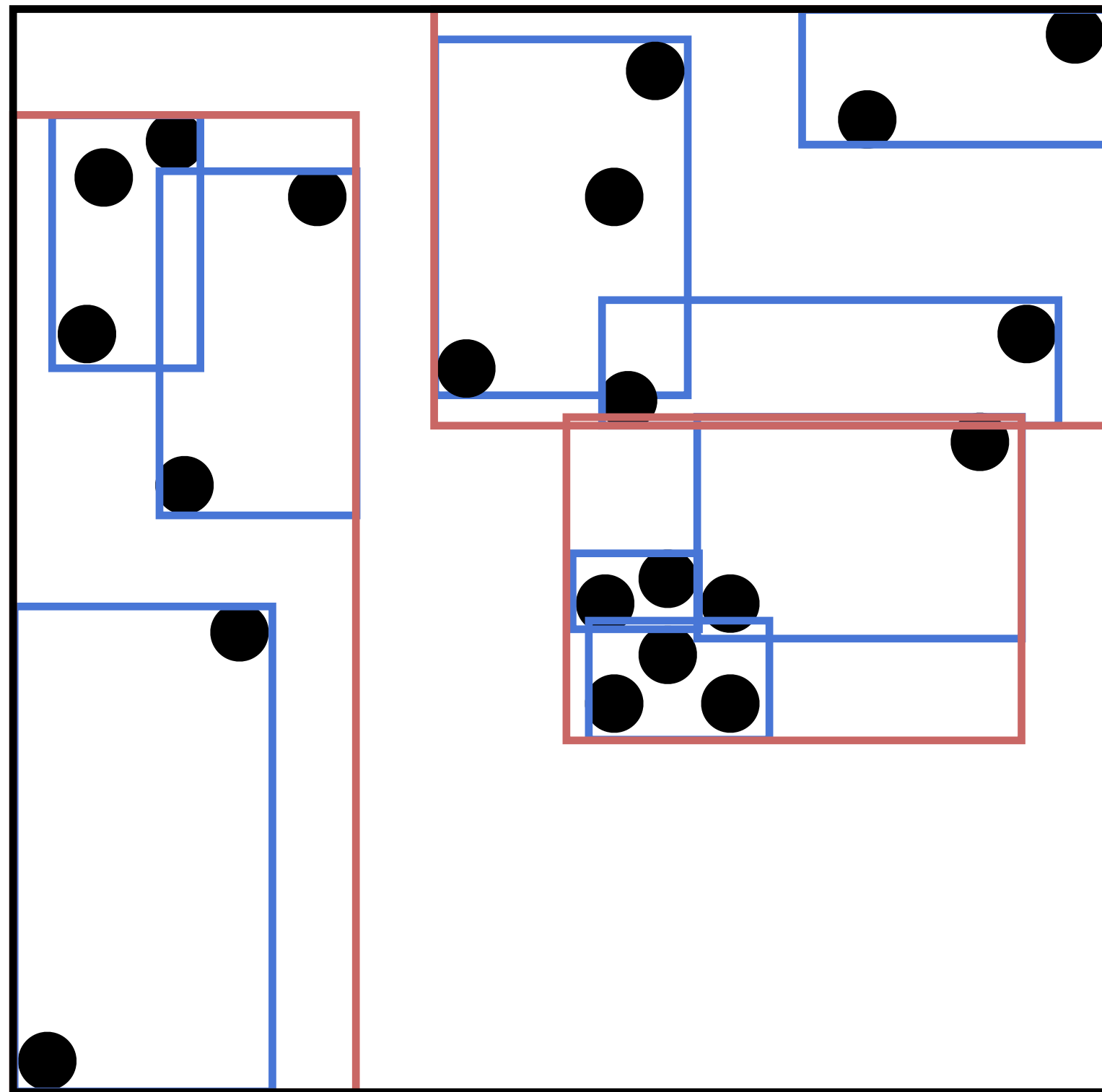


This is not space-driven !

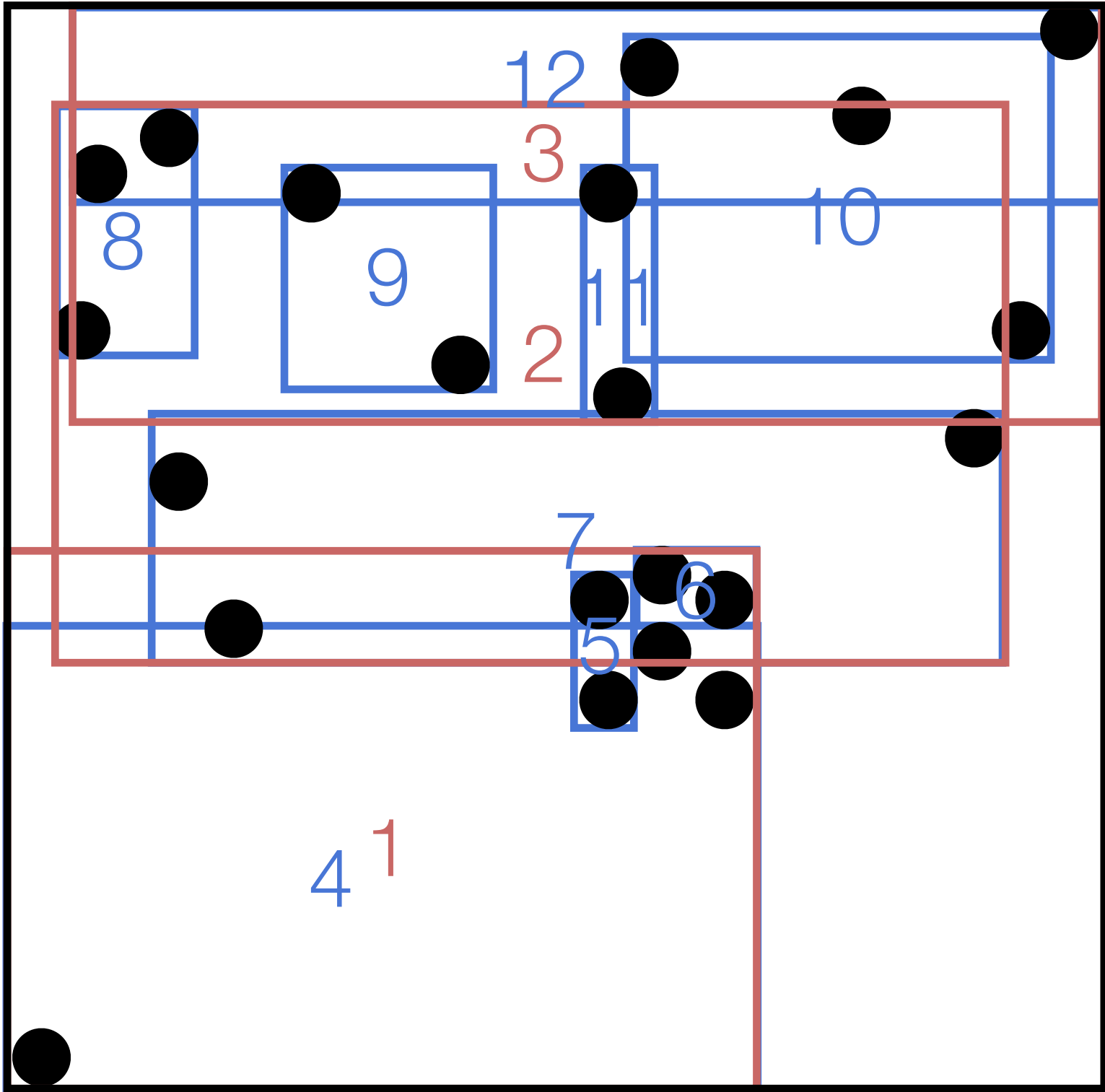
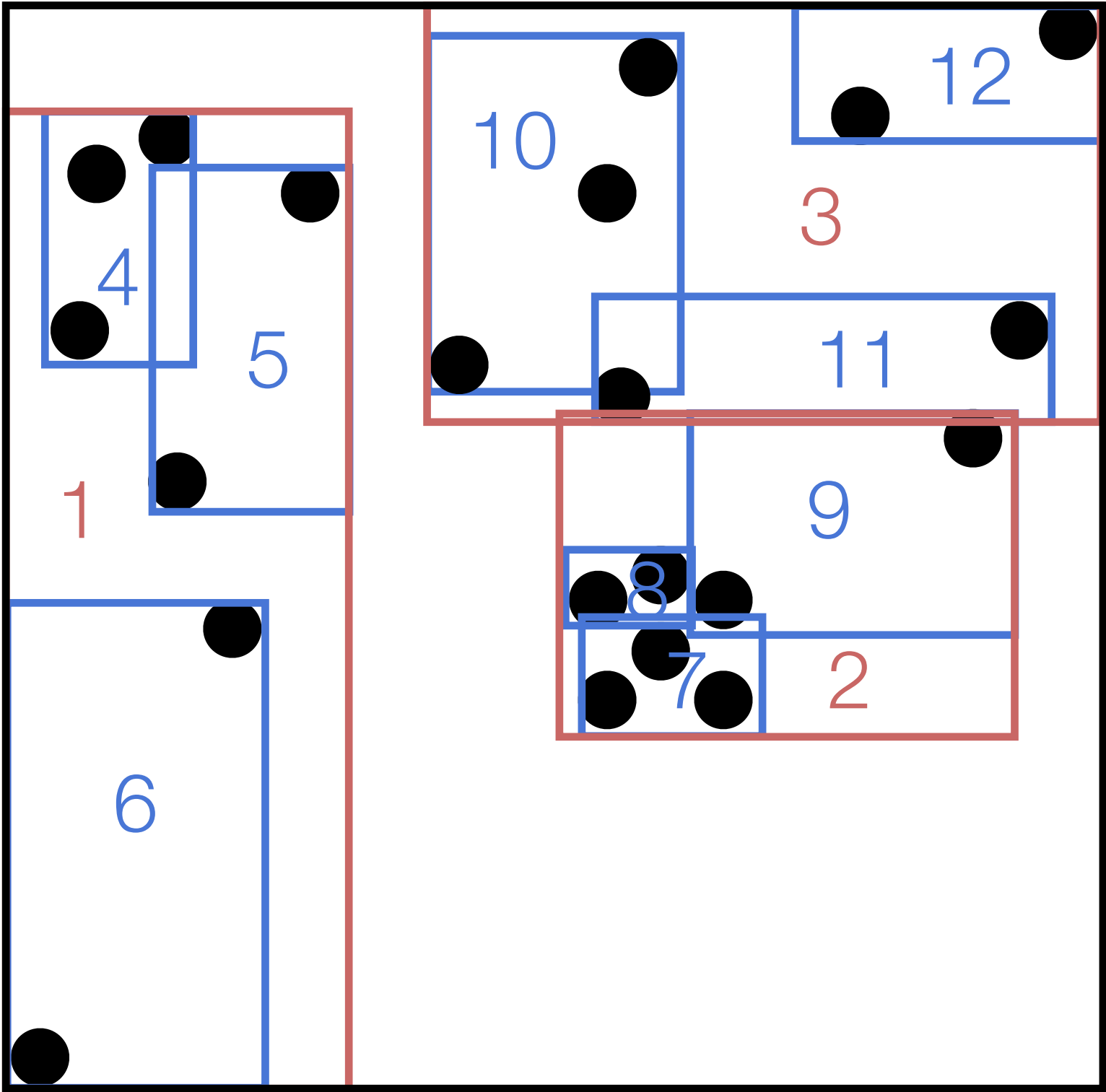
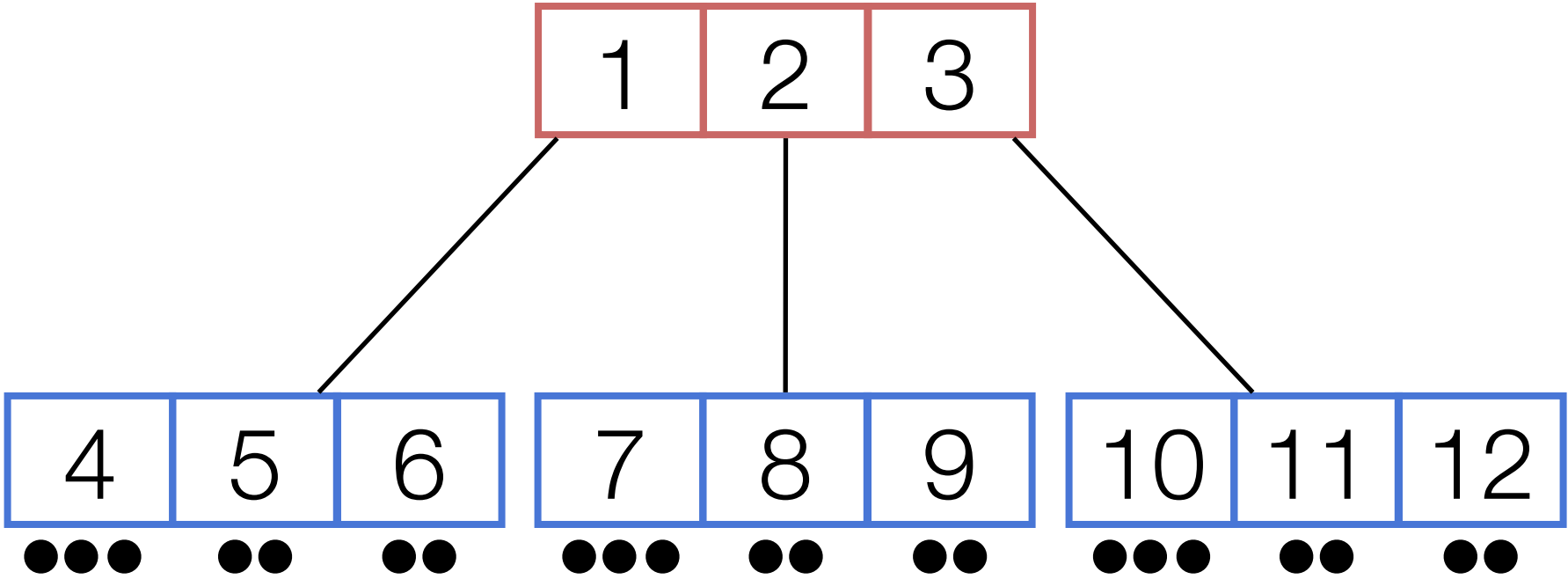


An r-tree can hold **arbitrary** MBBs...

Both are respecting the same conditions,
only the MBB packing is different

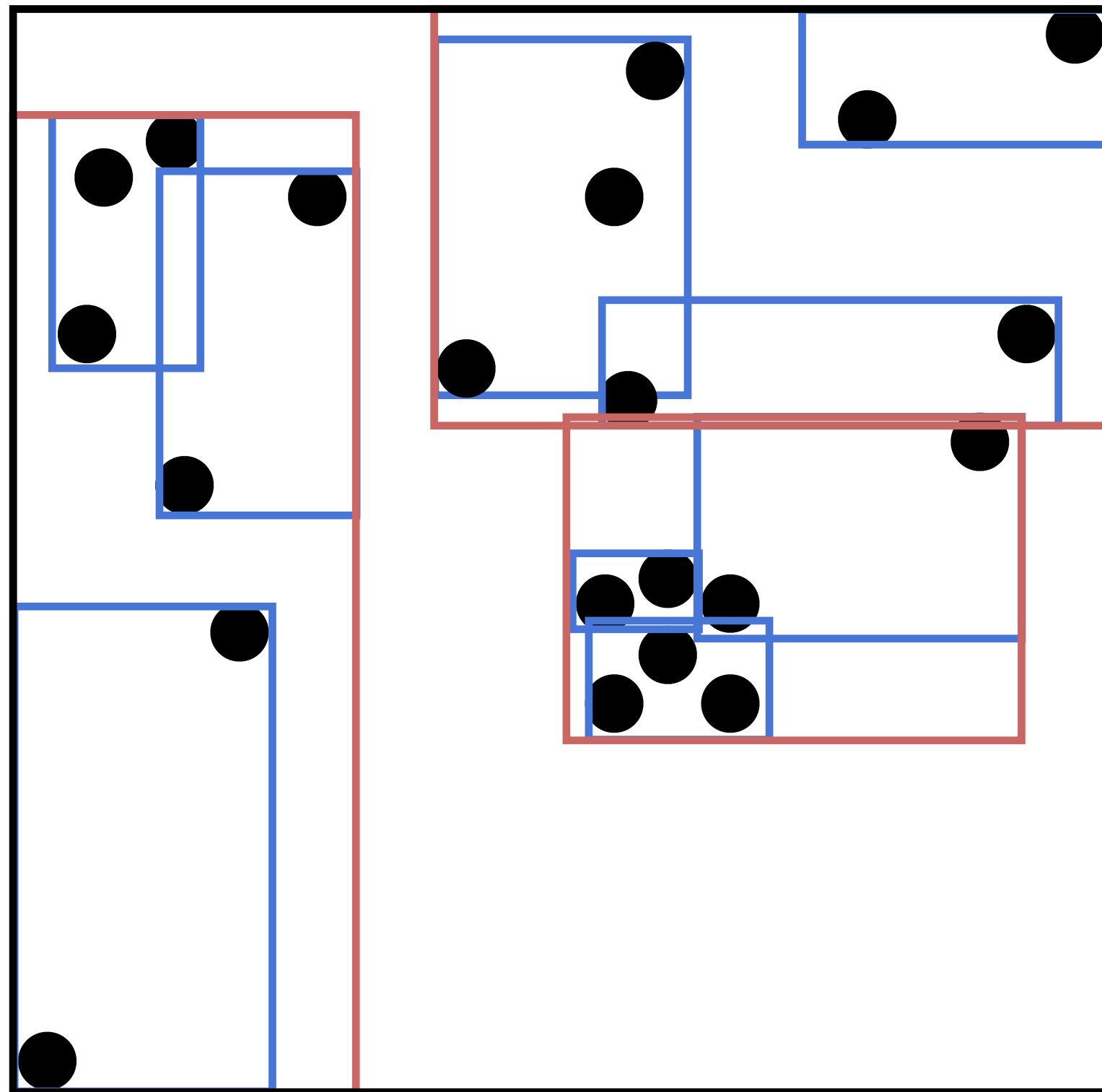


An r-tree can hold **arbitrary** MBBs...

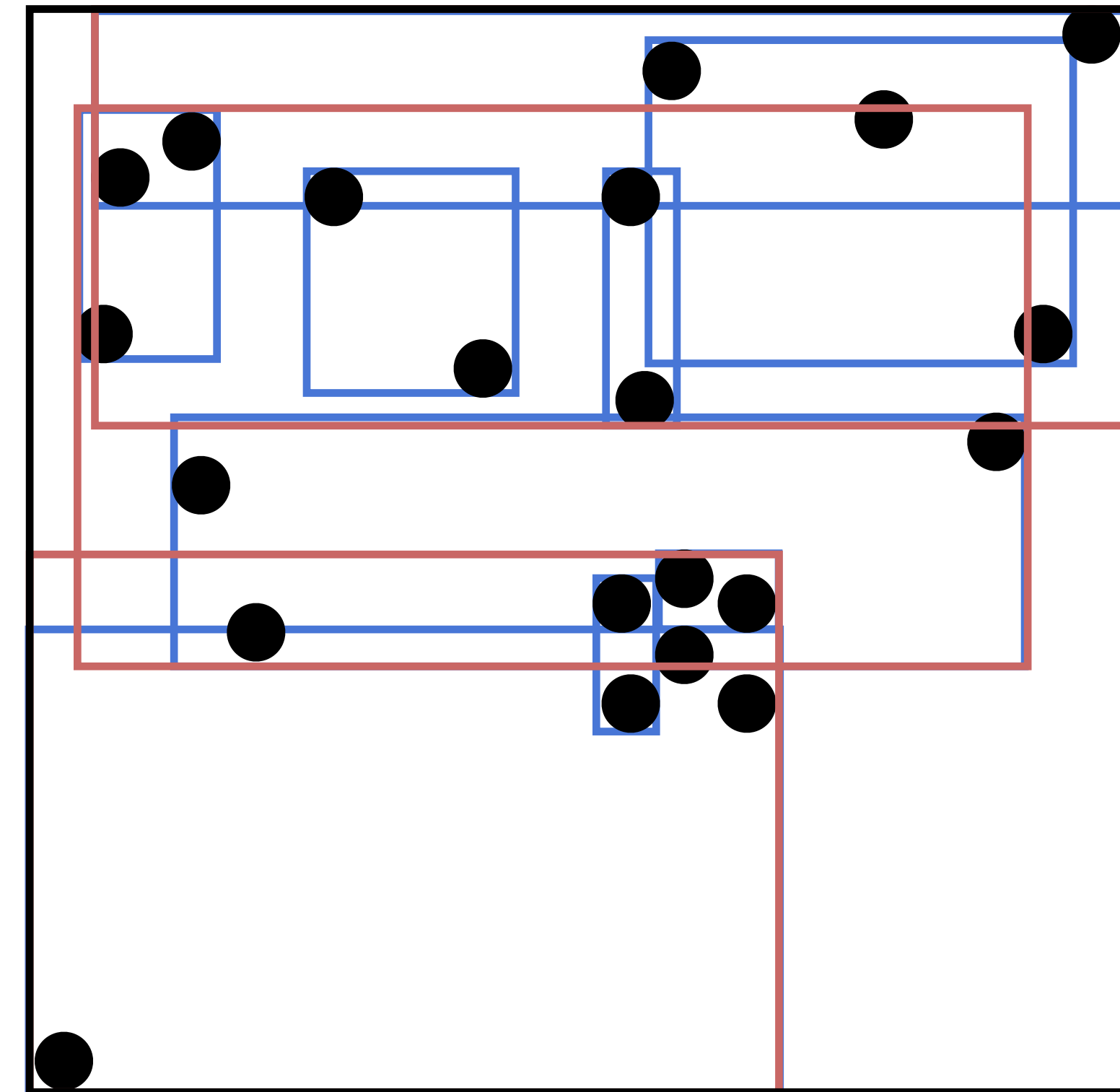


... but some packing algorithm are better than others

A good heuristic: the **lower the sum of perimeters** is, the **better the packing** usually is



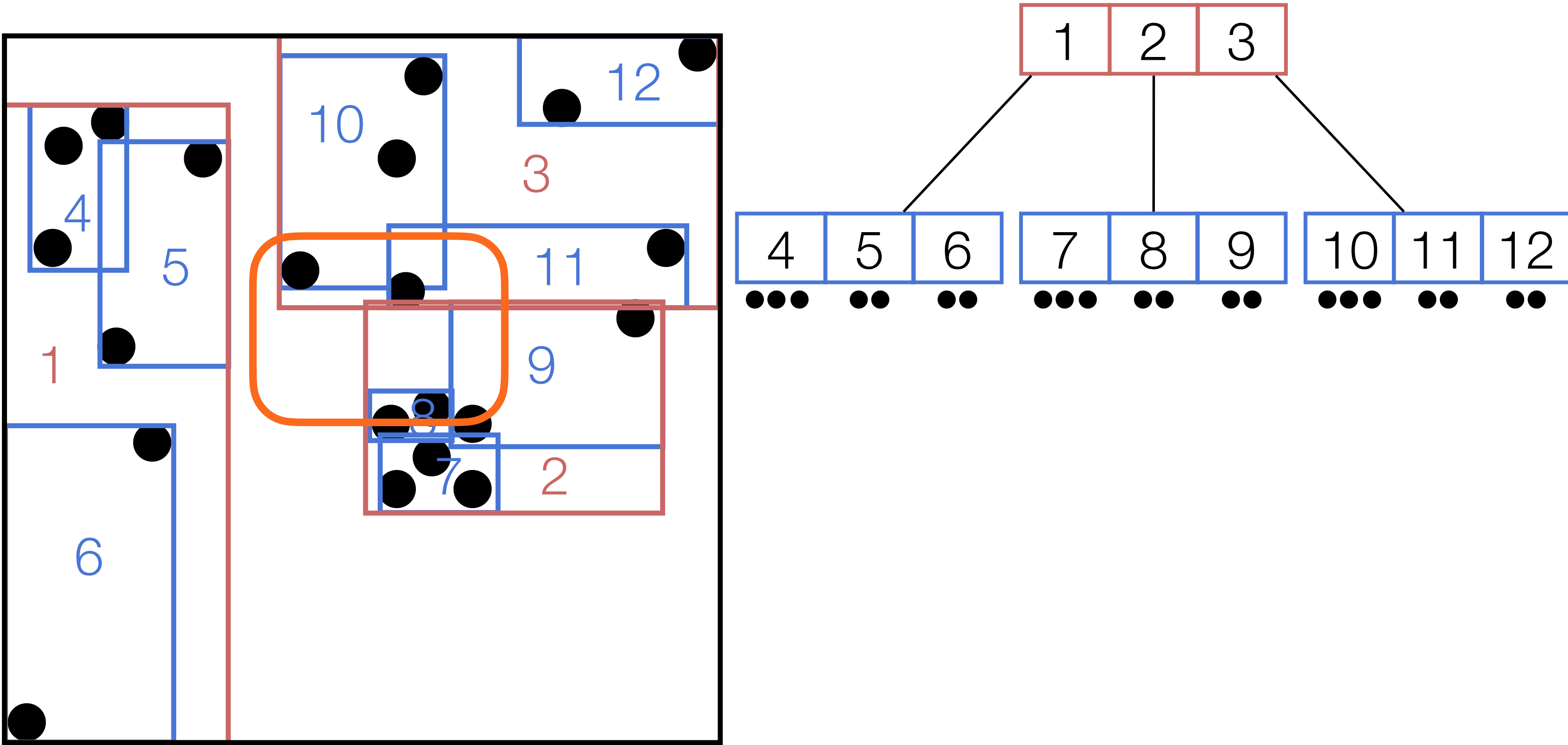
“Good” packing



“Bad” packing

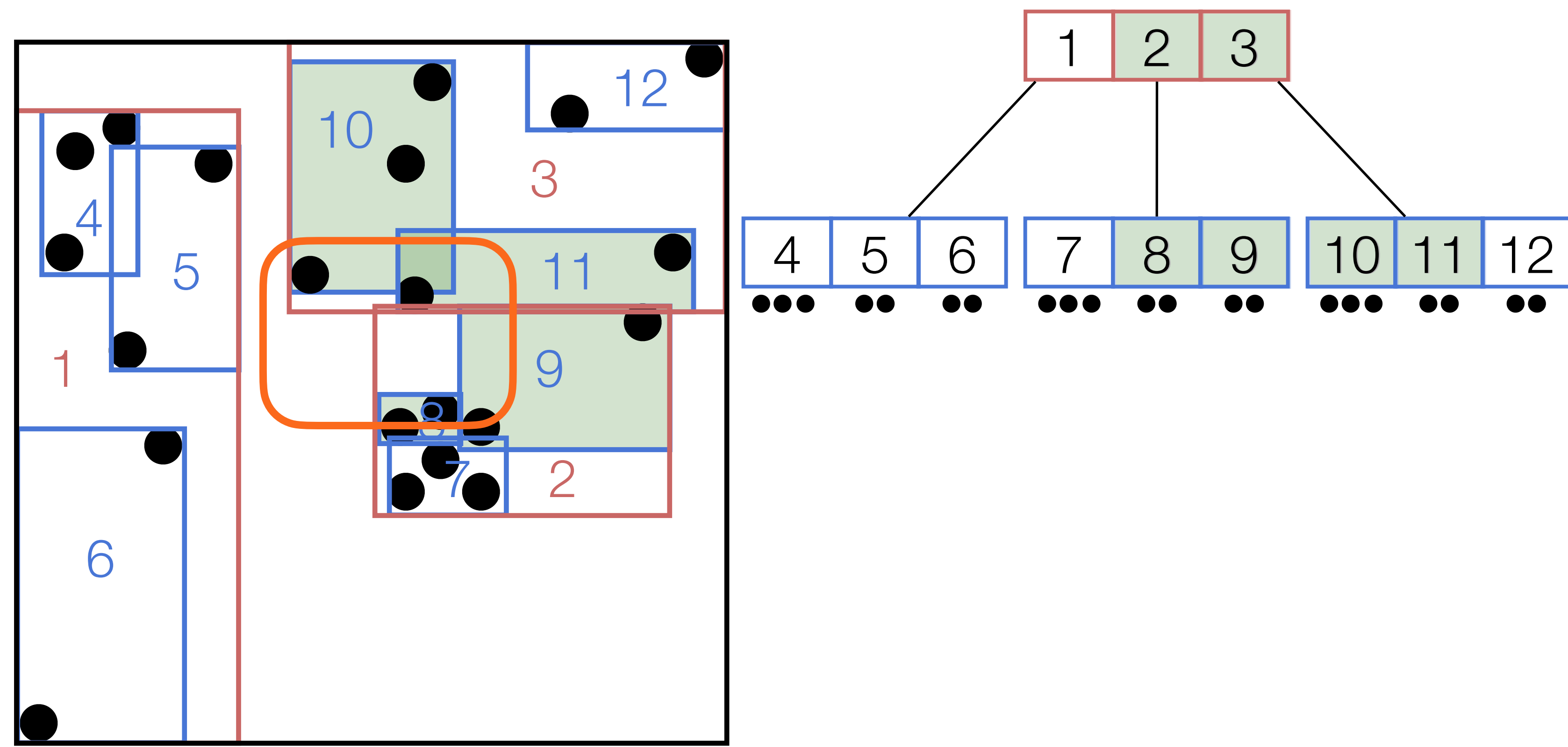
Application to the first problem

Which points are inside the polygon ?

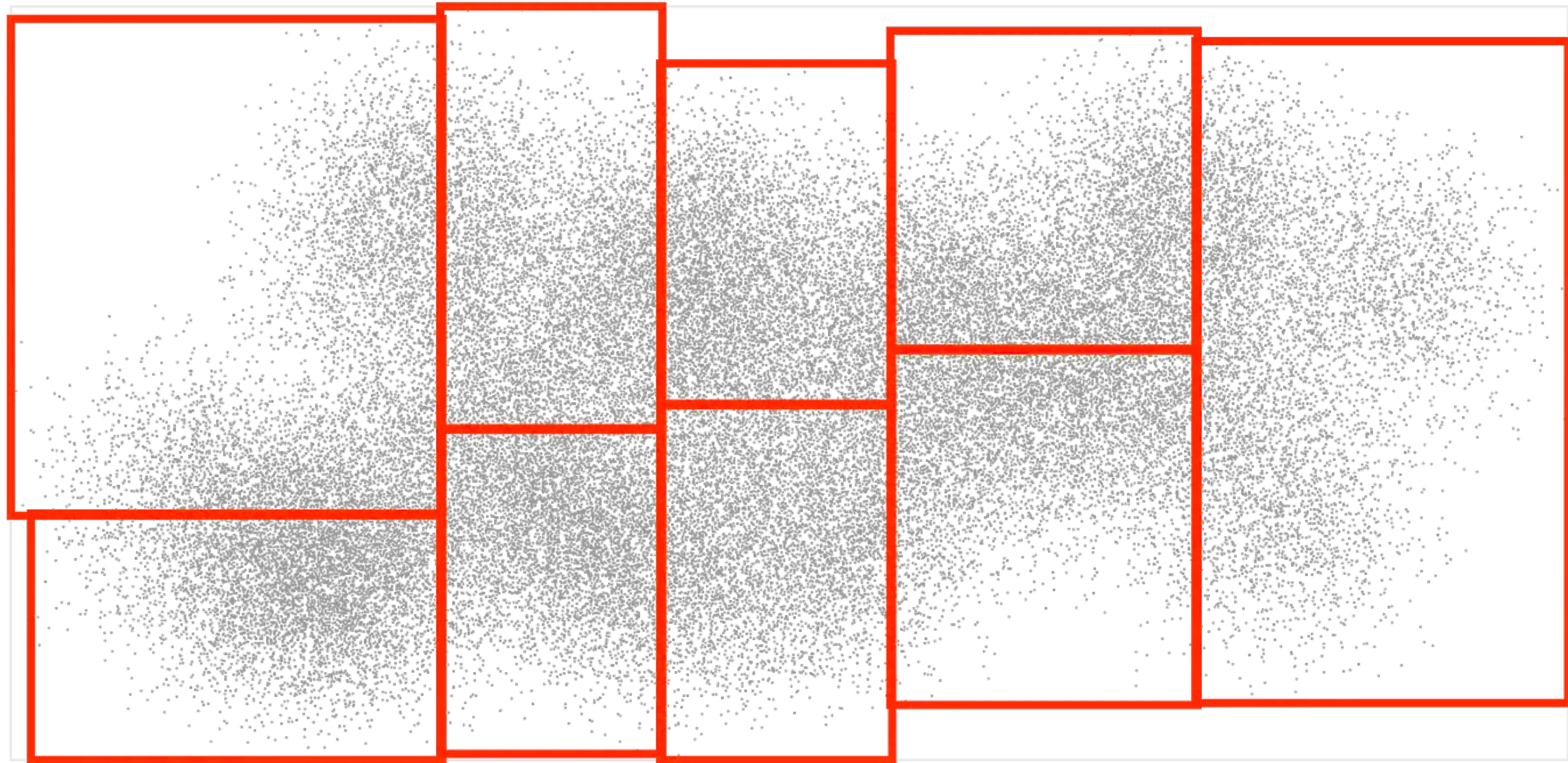


Application to the first problem

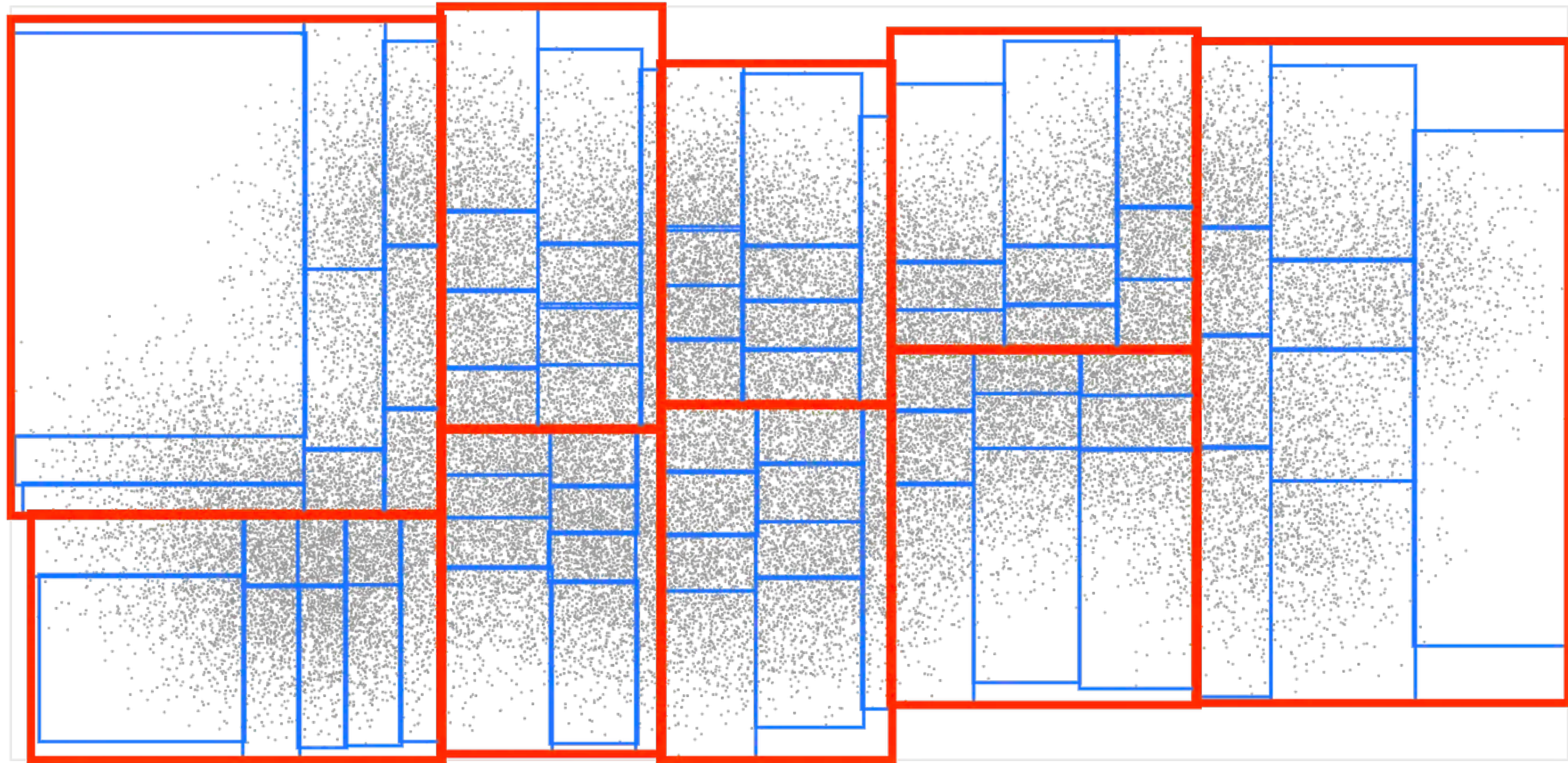
Test only the points in cells containing the polygon !



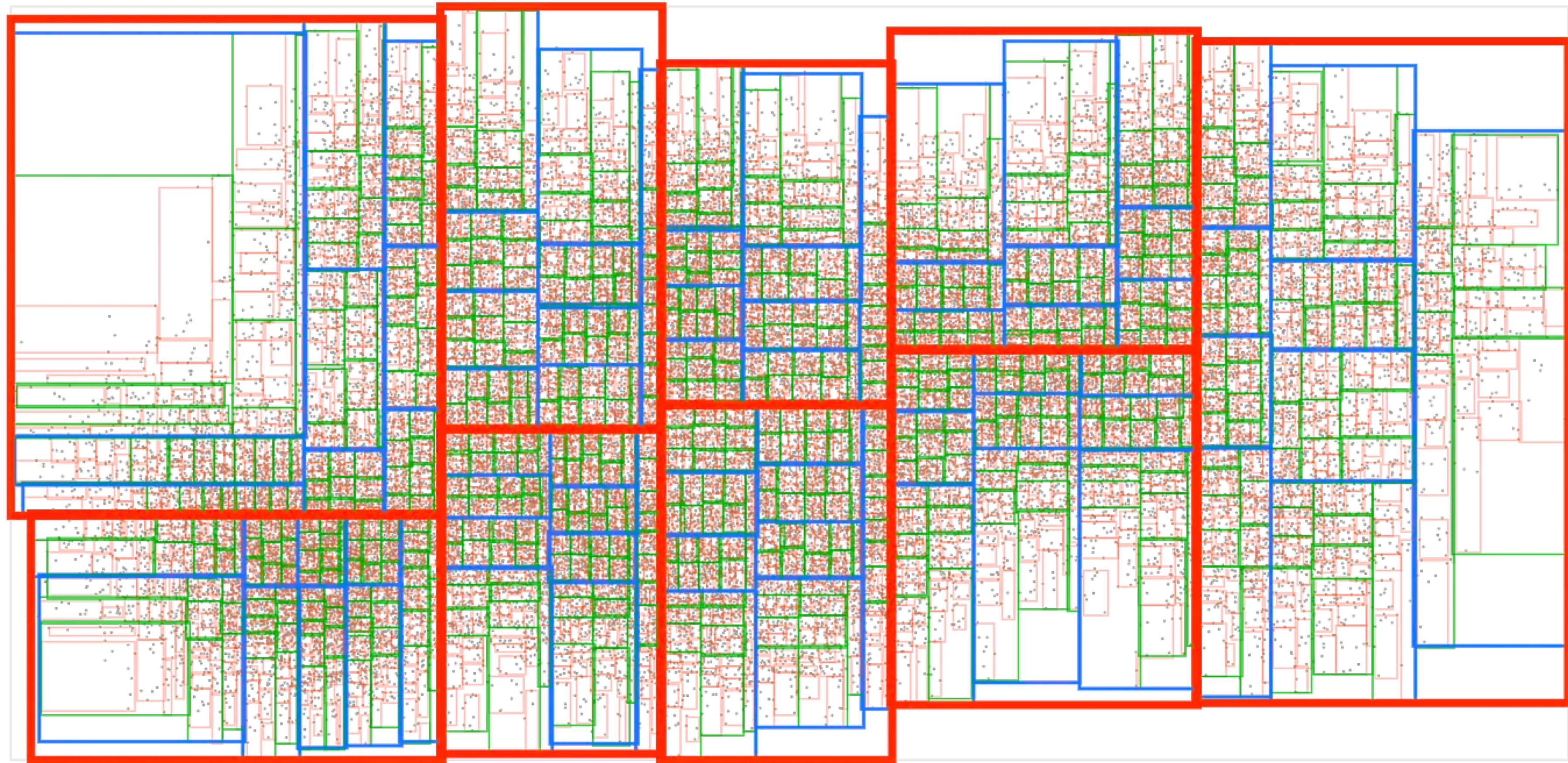
Here is a more realistic depiction



Here is a more realistic depiction



Here is a more realistic depiction



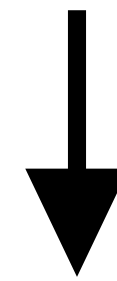
How to choose your spatial index ?

Consider:

- Geometry type: storing/searching points or polygons ?
- Distribution: clustered or sparse ?
- Dimensions: 2D or more ?
- Query types: Nearest Neighbor or Intersection ?
- Update frequency: Should you reindex frequently ?

What is available by default in our tools ?

```
geopandas.GeoSeries.sindex
```



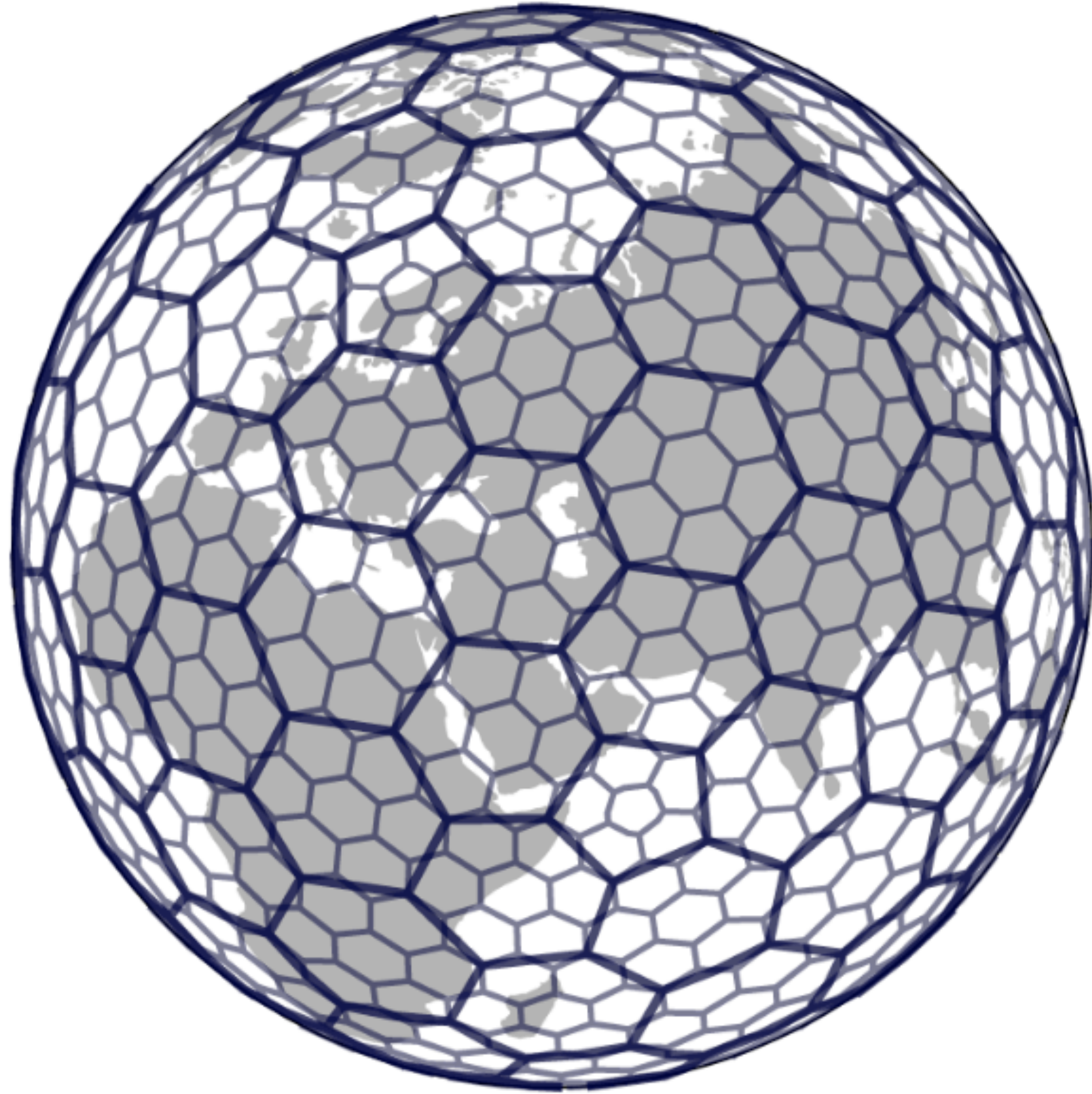
(For latest versions)

```
shapely.STRTree
```

Only the r-tree, with the STR algorithm
by (Leutenegger et al., 1997)

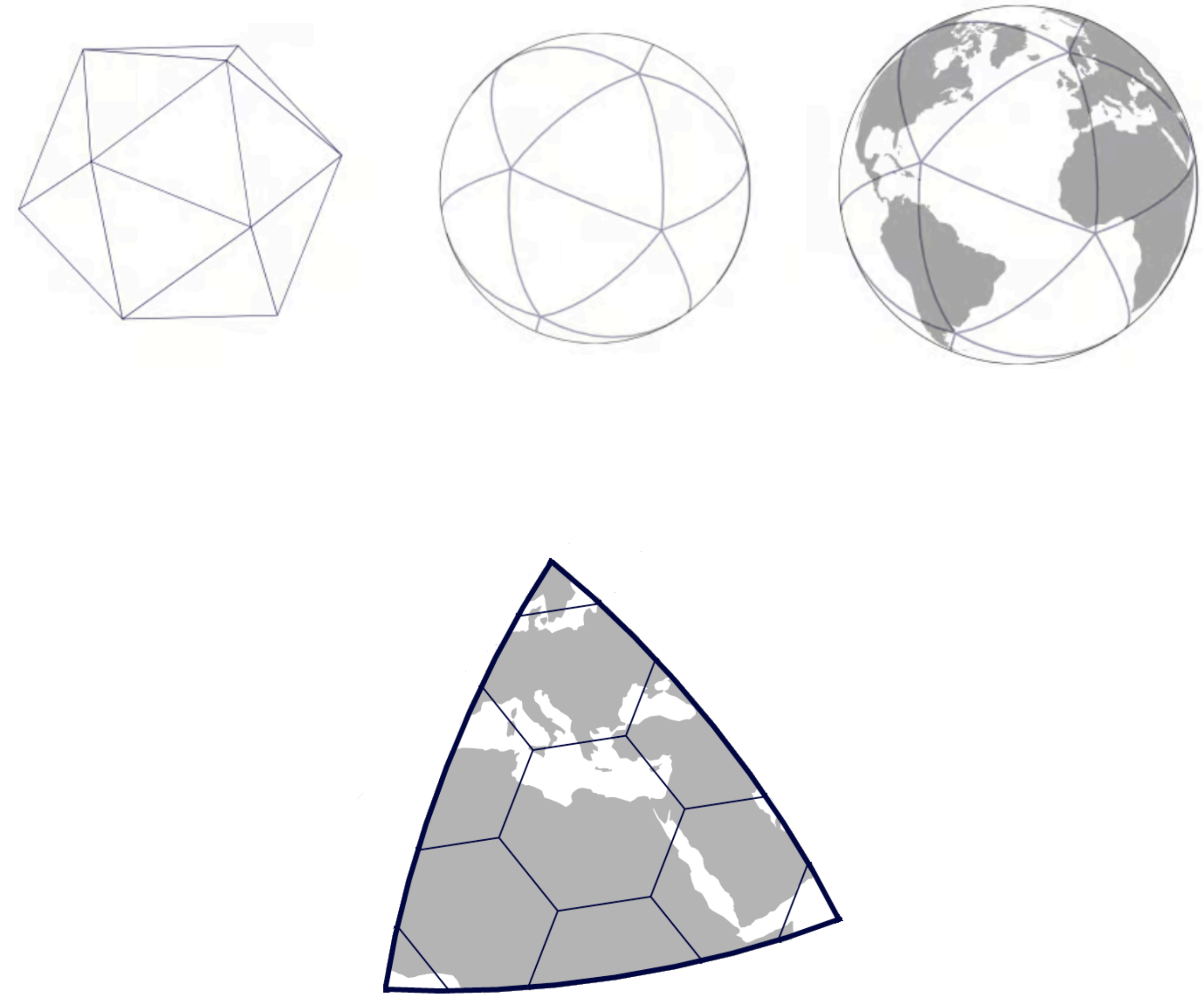
Tools for **BIG** spatial data

What is **h3** ?

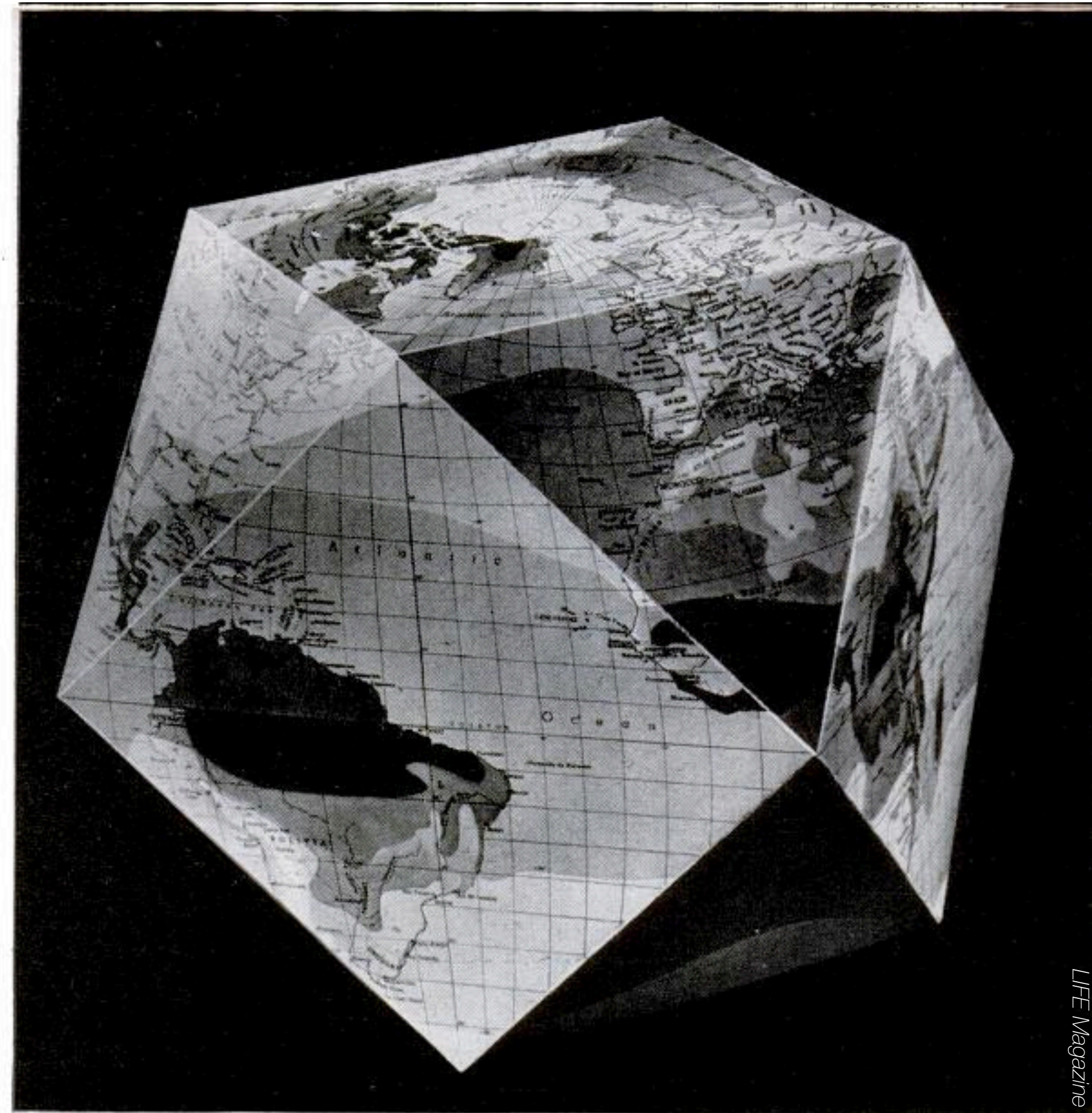


- Originally developed by Uber
- **Hexagonal* global hierarchical geospatial indexing system**
- 16 different resolutions
- for spatial indexing, data aggregation, data joining, flow analysis, etc.

It creates a model based on a **icosahedron**...

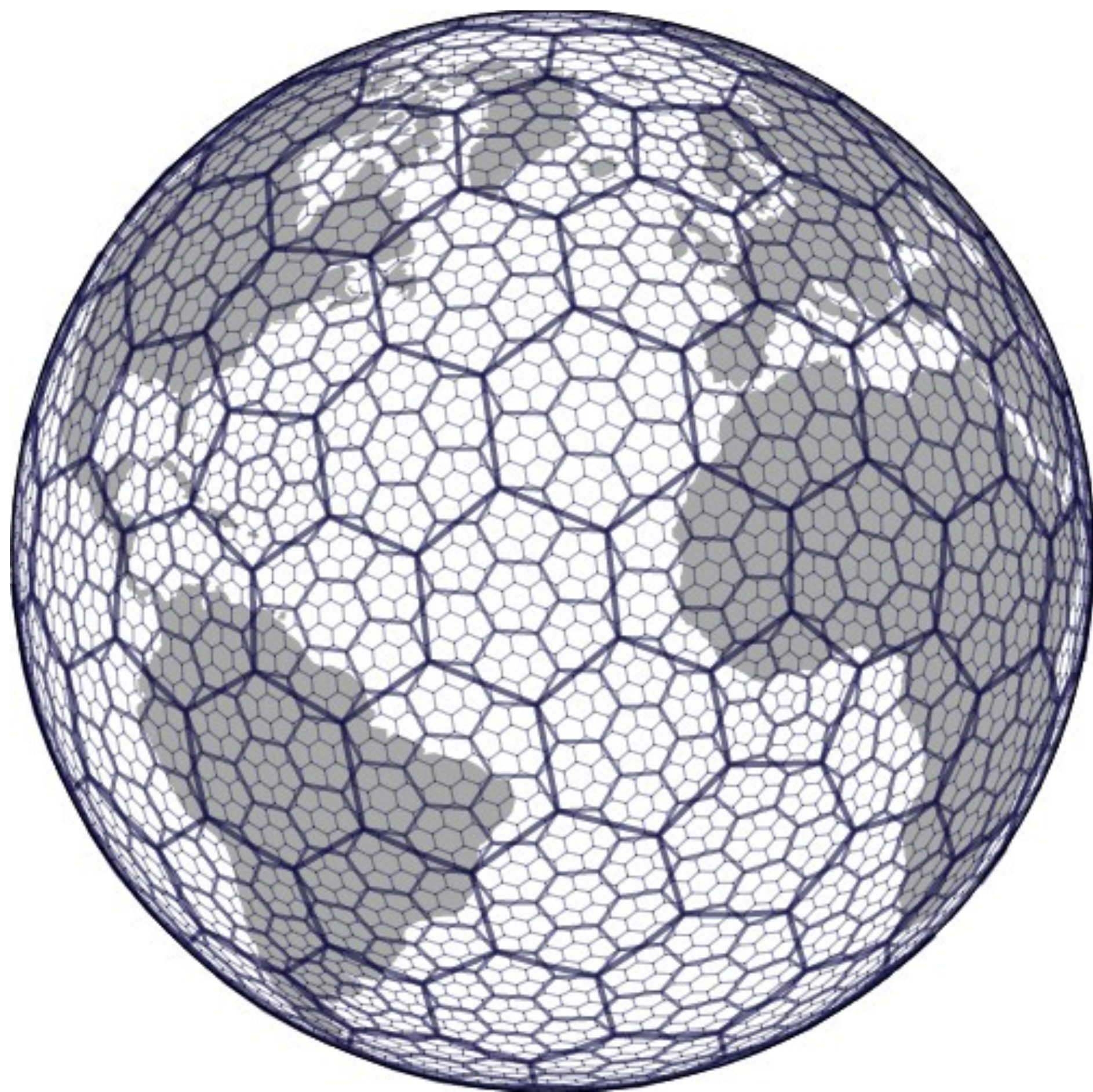


... just like the **Dymaxion** cut of the Earth !

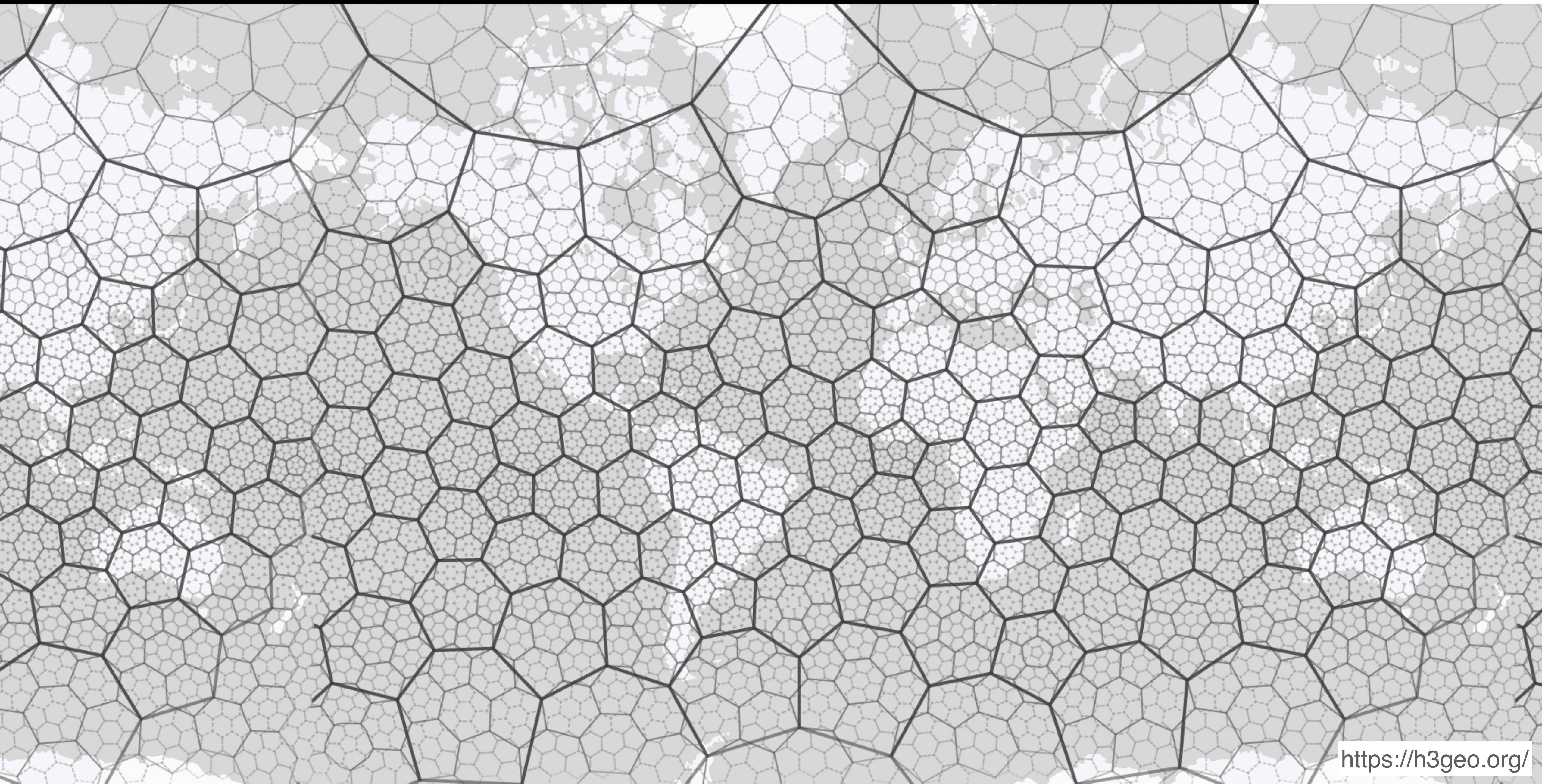


DYMAXION GLOBE, a cube with its corners cut off, has shape of irregular solid first constructed by Archimedes. Its scale, constant on edges of segments, is similar to that of a 12-in. globe.

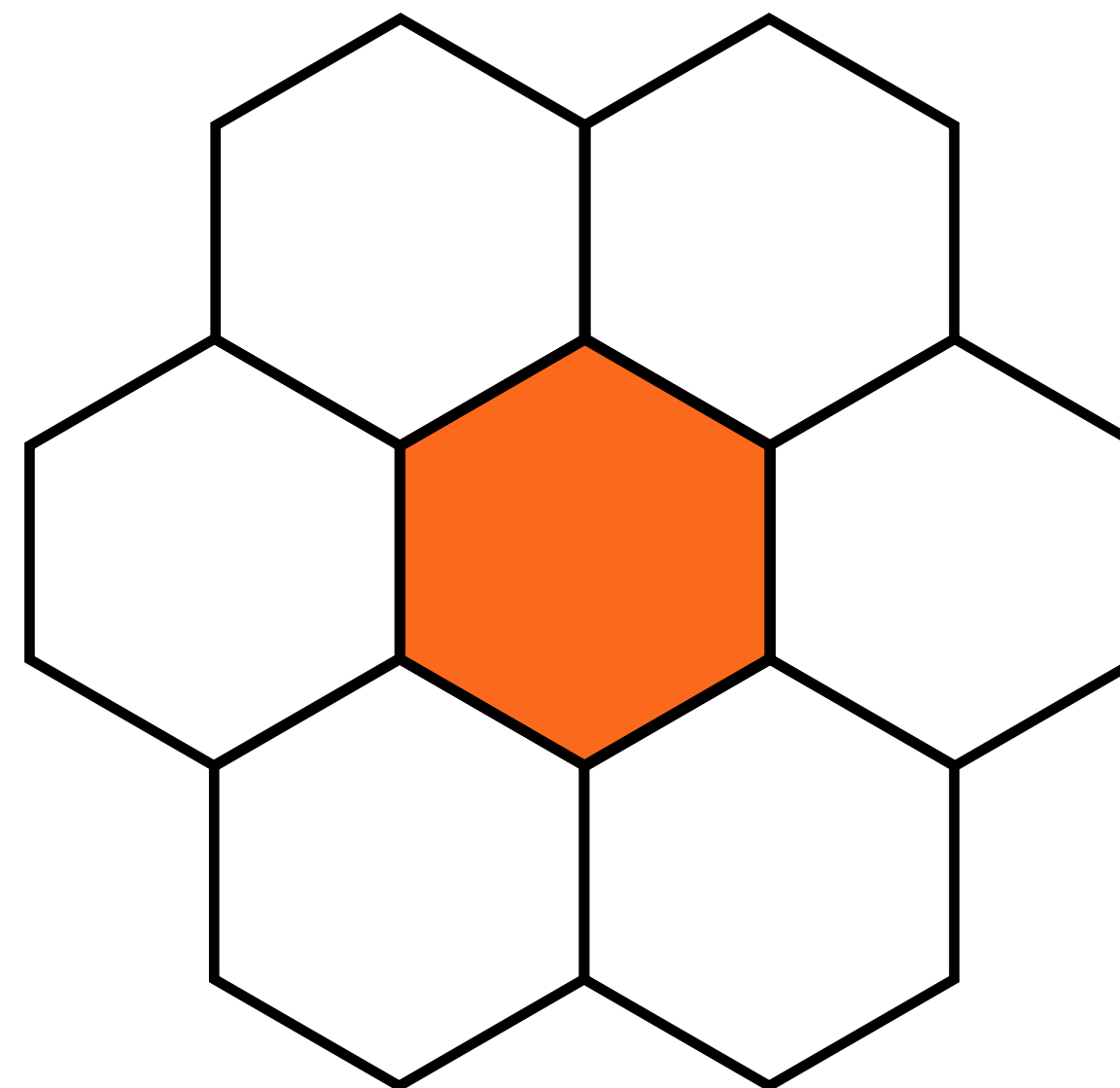
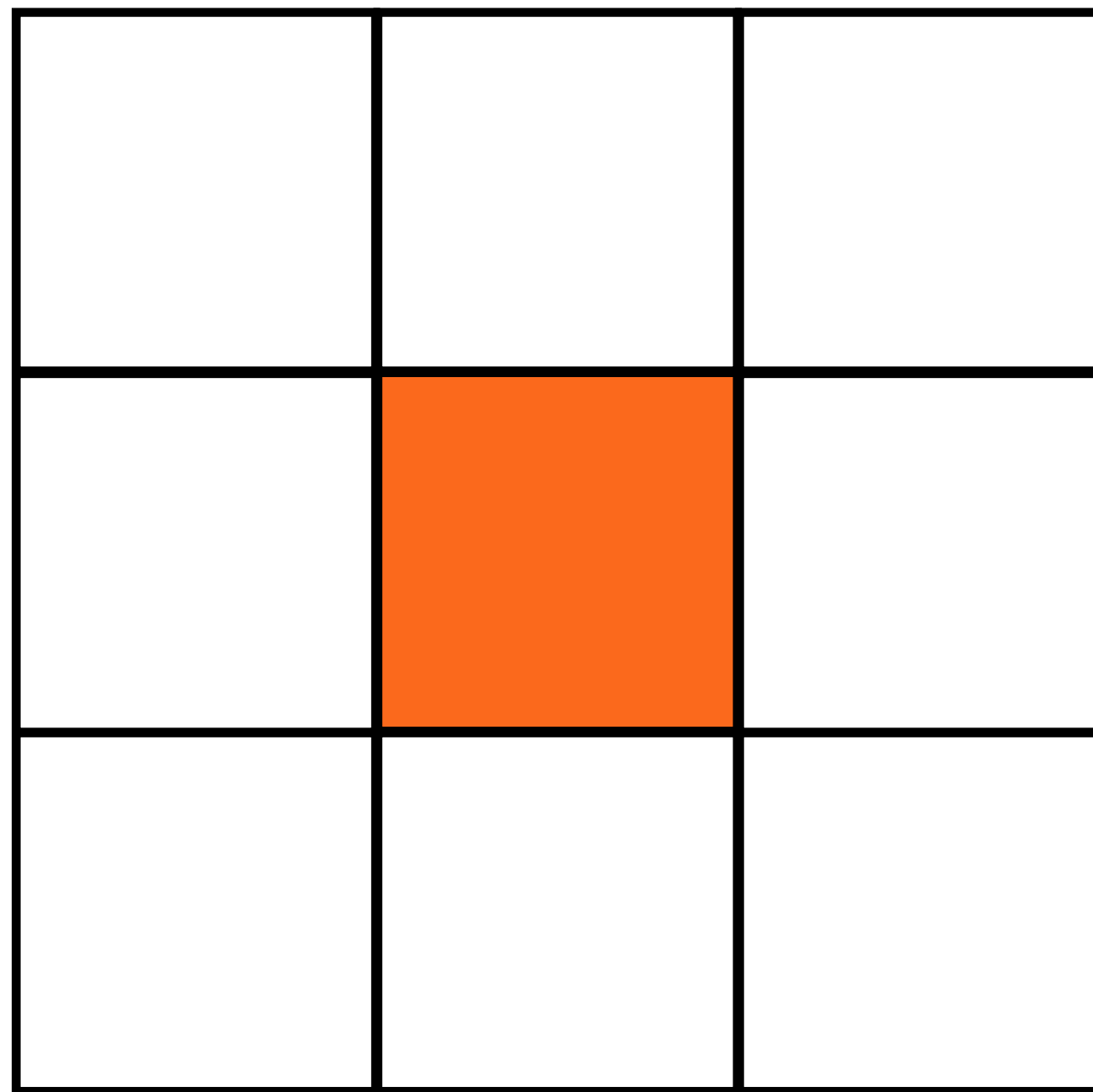
Hexagons **partially overlap** at different scales...



Hexagons **partially overlap** at different scales...

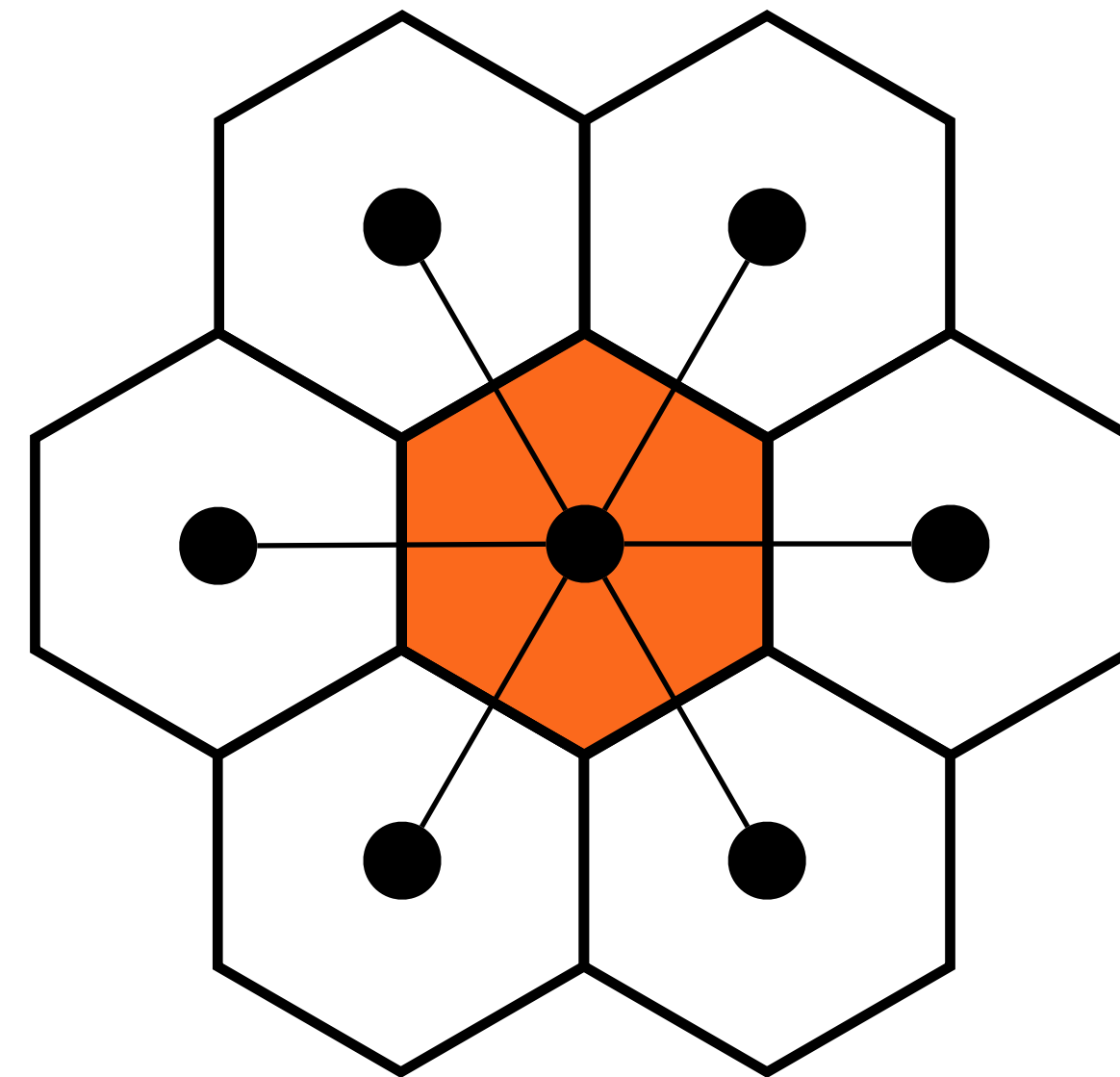
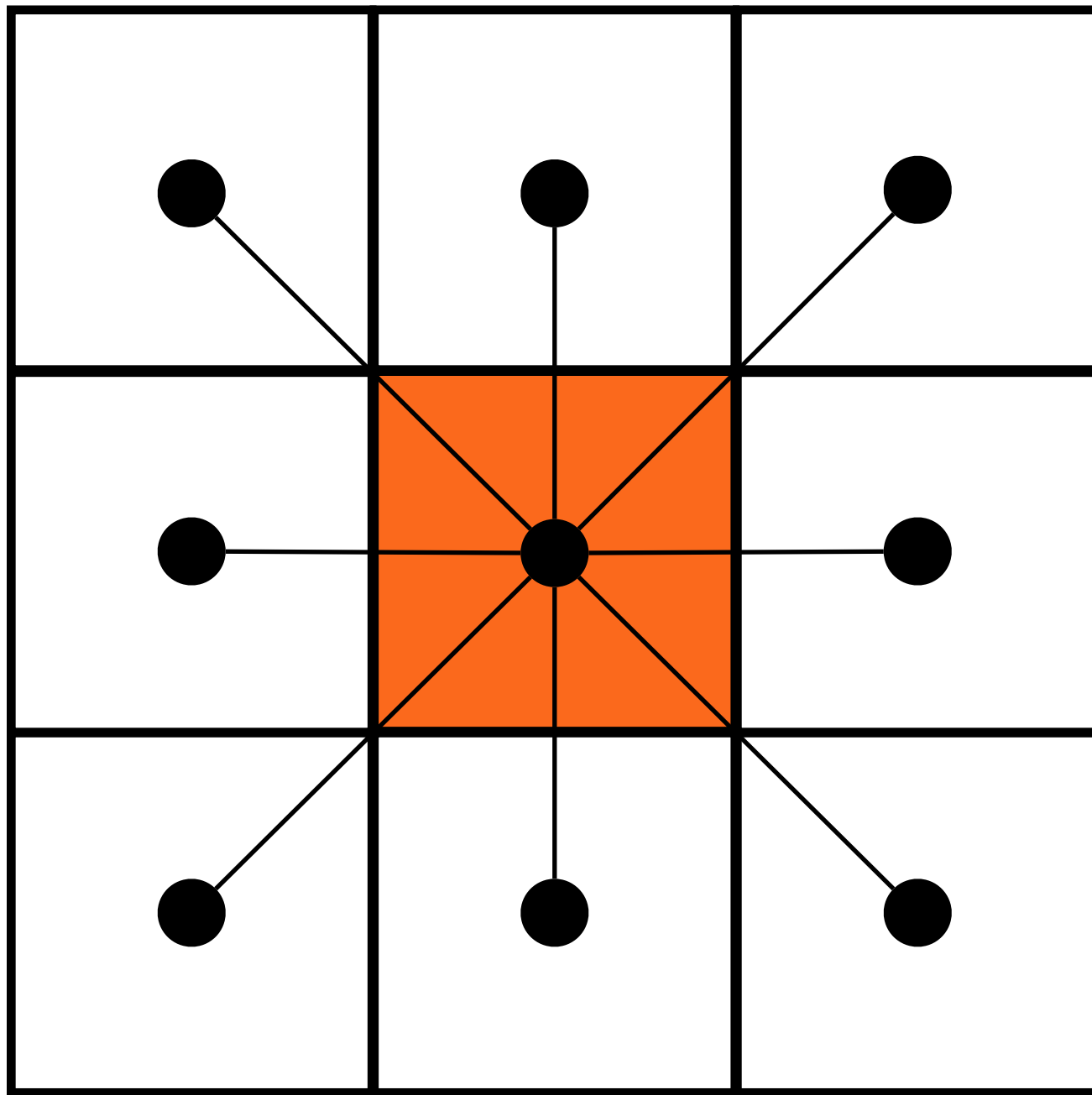


... but they also have **advantages**



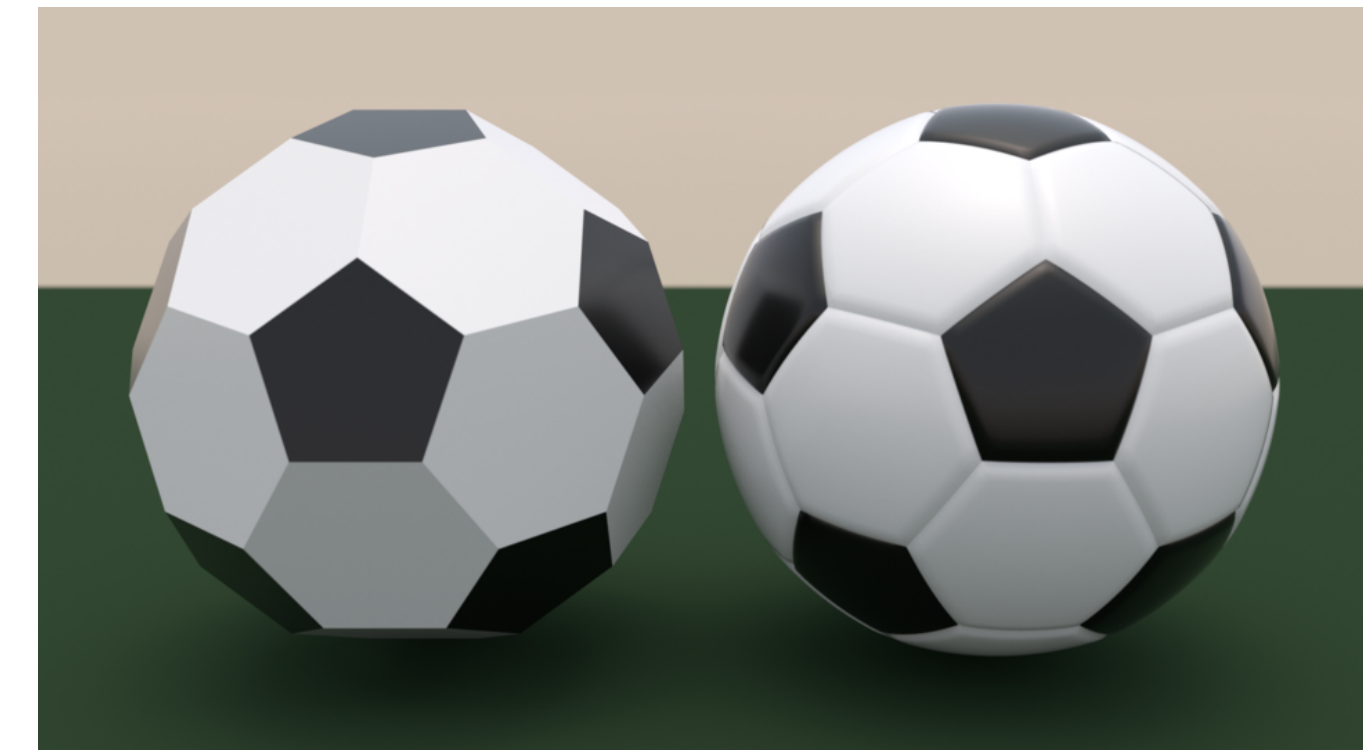
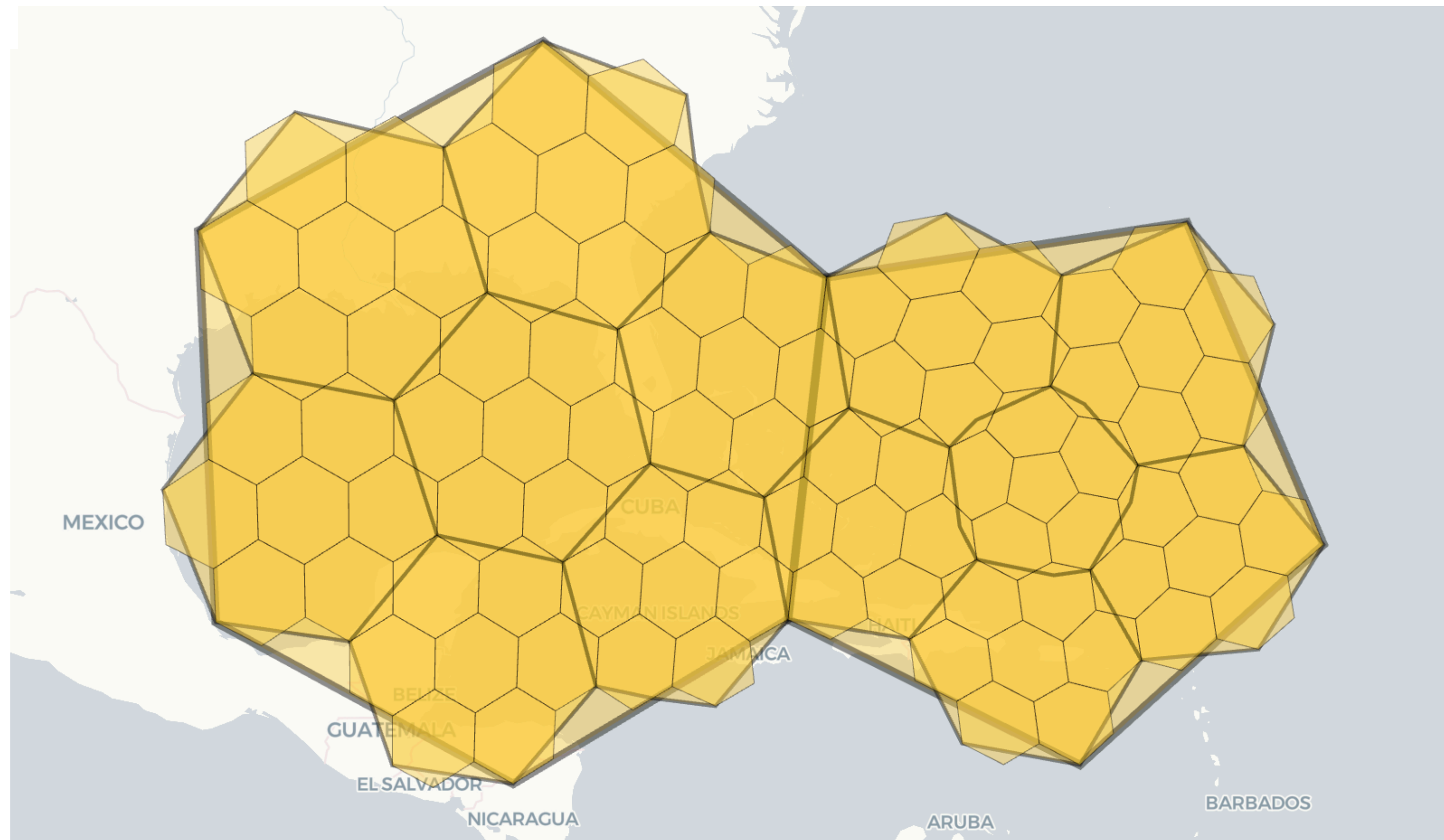
... but they also have **advantages**

- All neighbors share a side, not only a point
- All neighbors are equidistant
- Lower perimeter/area ratio



Pentagons are needed to fill out the space

“Since it is not possible to tile the icosahedron with only hexagons, we chose to introduce twelve pentagons, one at each of the icosahedron vertices. These vertices were positioned using the spherical icosahedron orientation by R. Buckminster Fuller, which places all the vertices in the water.”



Resolutions are from 0-15

Compatible with WGS84

Res	Total number of cells	Number of hexagons	Number of pentagons
0	122	110	12
1	842	830	12
2	5,882	5,870	12
3	41,162	41,150	12
4	288,122	288,110	12
5	2,016,842	2,016,830	12
6	14,117,882	14,117,870	12
7	98,825,162	98,825,150	12
8	691,776,122	691,776,110	12
9	4,842,432,842	4,842,432,830	12
10	33,897,029,882	33,897,029,870	12
11	237,279,209,162	237,279,209,150	12
12	1,660,954,464,122	1,660,954,464,110	12
13	11,626,681,248,842	11,626,681,248,830	12
14	81,386,768,741,882	81,386,768,741,870	12
15	569,707,381,193,162	569,707,381,193,150	12

Resolutions are from 0-15

Compatible with WGS84

Res	Average <u>Hexagon</u> Area (km²)	Pentagon Area* (km²)	Ratio (P/H)
0	4,357,449.416078381	2,562,182.162955496	0.5880
1	609,788.441794133	328,434.586246469	0.5386
2	86,801.780398997	44,930.898497879	0.5176
3	12,393.434655088	6,315.472267516	0.5096
4	1,770.347654491	896.582383141	0.5064
5	252.903858182	127.785583023	0.5053
6	36.129062164	18.238749548	0.5048
7	5.161293360	2.604669397	0.5047
8	0.737327598	0.372048038	0.5046
9	0.105332513	0.053147195	0.5046
10	0.015047502	0.007592318	0.5046
11	0.002149643	0.001084609	0.5046
12	0.000307092	0.000154944	0.5046
13	0.000043870	0.000022135	0.5046
14	0.000006267	0.000003162	0.5046
15	0.000000895	0.000000452	0.5046

Resolutions are from 0-15

100🇩🇰 or 1.3🇮🇳

10🏟️

<1m2

Res	Average <u>Hexagon</u> Area (km²)	Pentagon Area* (km²)	Ratio (P/H)
0	4,357,449.416078381	2,562,182.162955496	0.5880
1	609,788.441794133	328,434.586246469	0.5386
2	86,801.780398997	44,930.898497879	0.5176
3	12,393.434655088	6,315.472267516	0.5096
4	1,770.347654491	896.582383141	0.5064
5	252.903858182	127.785583023	0.5053
6	36.129062164	18.238749548	0.5048
7	5.161293360	2.604669397	0.5047
8	0.737327598	0.372048038	0.5046
9	0.105332513	0.053147195	0.5046
10	0.015047502	0.007592318	0.5046
11	0.002149643	0.001084609	0.5046
12	0.000307092	0.000154944	0.5046
13	0.000043870	0.000022135	0.5046
14	0.000006267	0.000003162	0.5046
15	0.000000895	0.000000452	0.5046

Each hexagon is indexed by a unique ID



Other examples of global spatial indexes

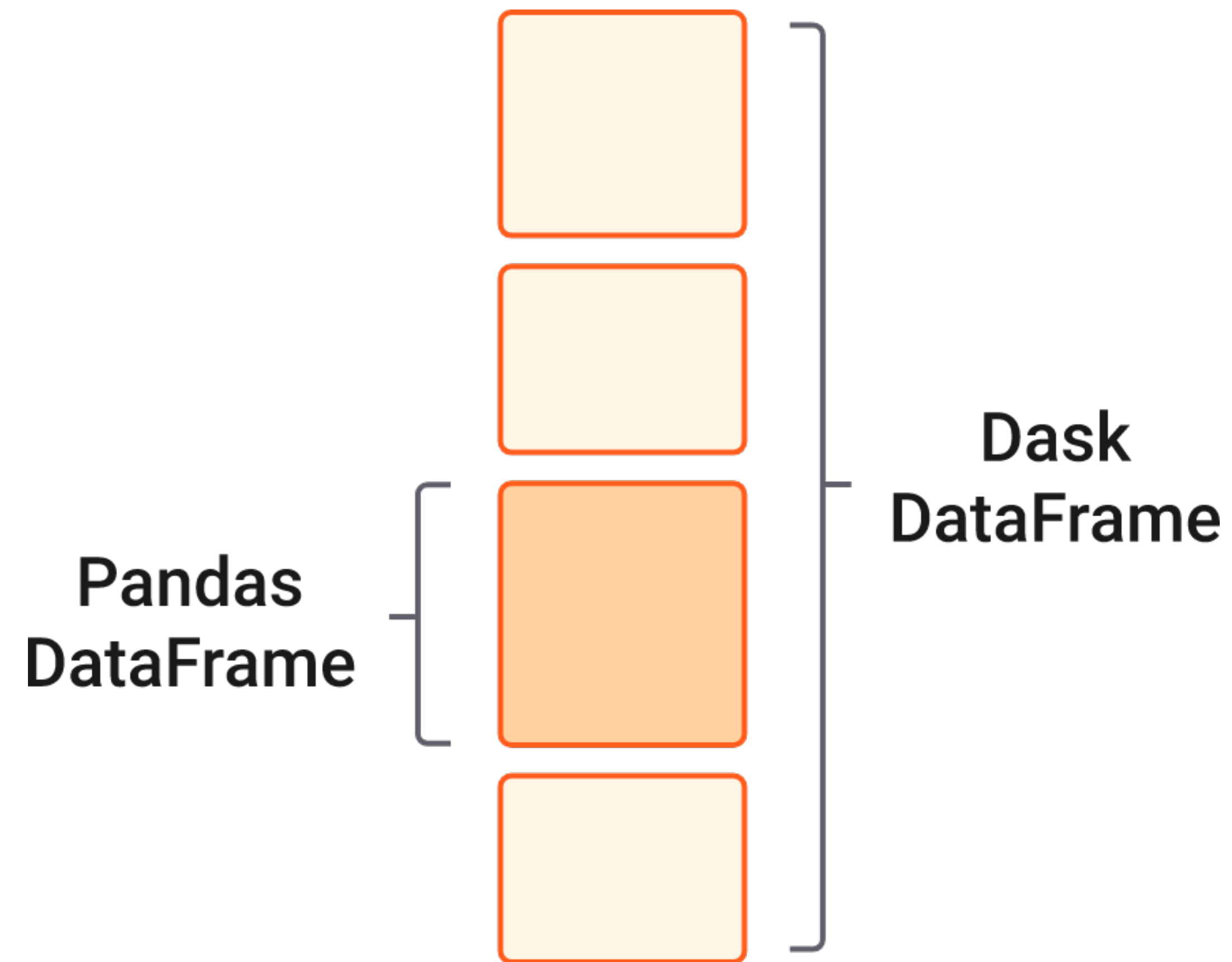
- **S2**, an open source, hierarchical, discrete, and global grid system using square cells.
- **Geohash**, a system for encoding locations using a string of characters, creating a hierarchical, square grid system (a quadtree).
- **Placekey**, a system for encoding points of interest (POIs) which incorporates H3 in its POI identifier.

“Parallel GeoPandas”



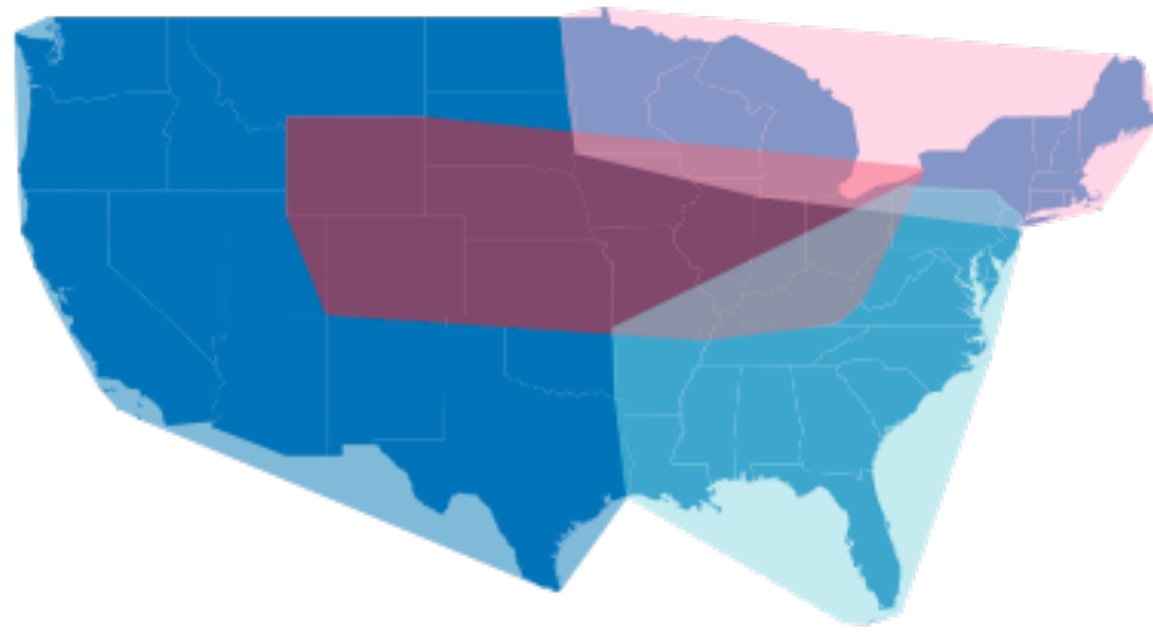
For GeoDataFrames that **do not fit in memory** or require expensive computation that can easily be **parallelized**

Dask-Pandas: **Regular** Partitioning

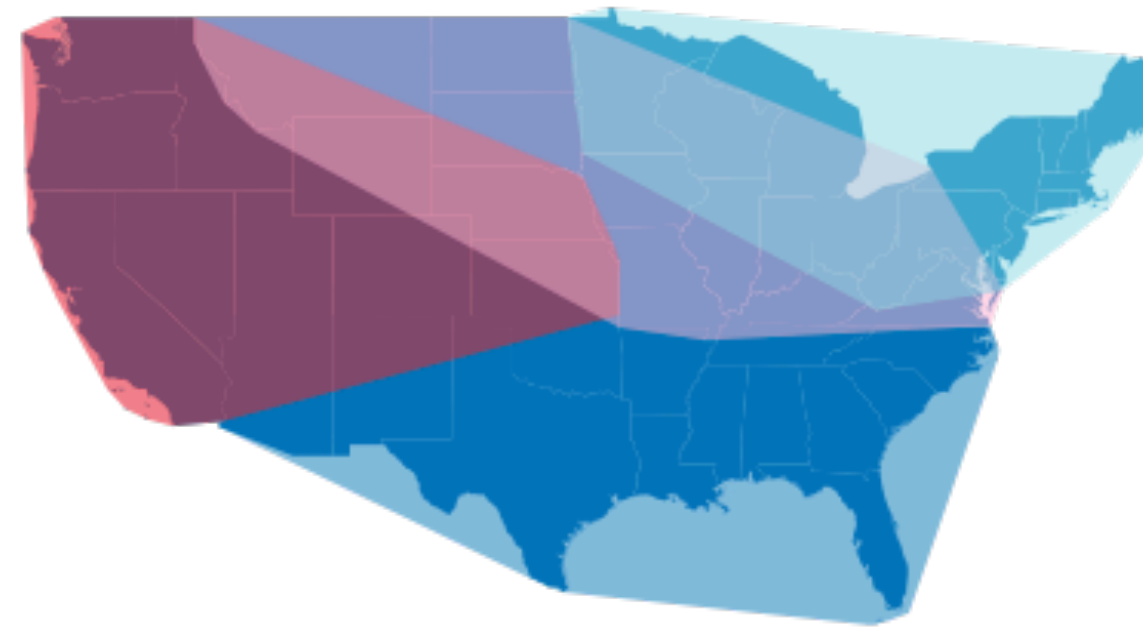


Dask-GeoPandas: **Spatial** Partitioning

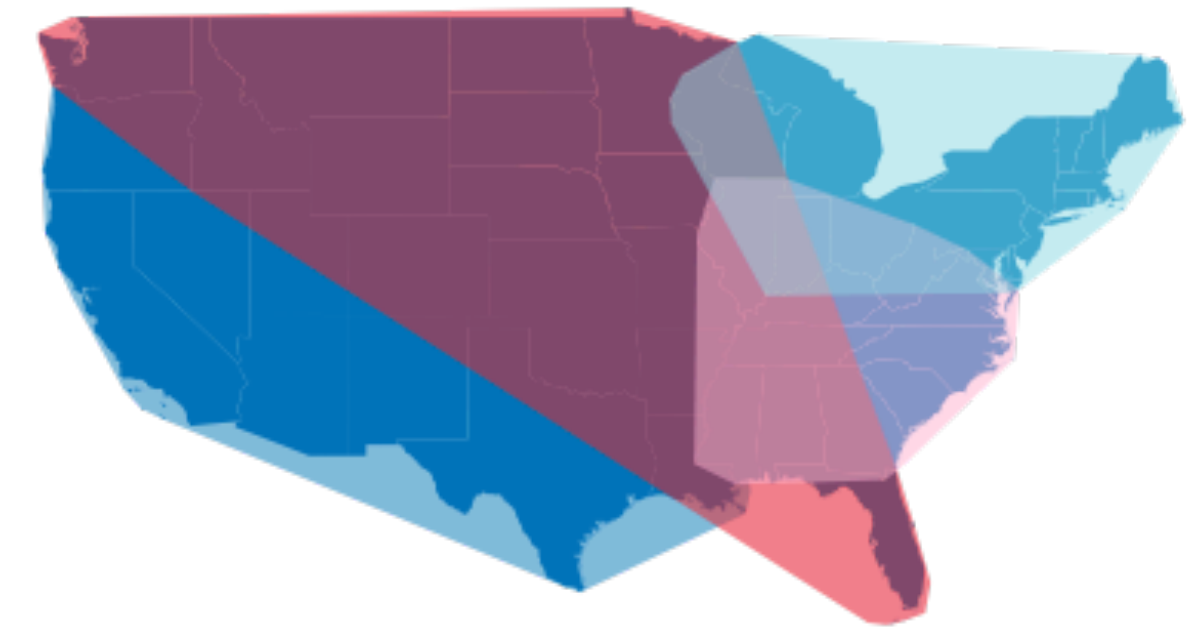
Hilbert



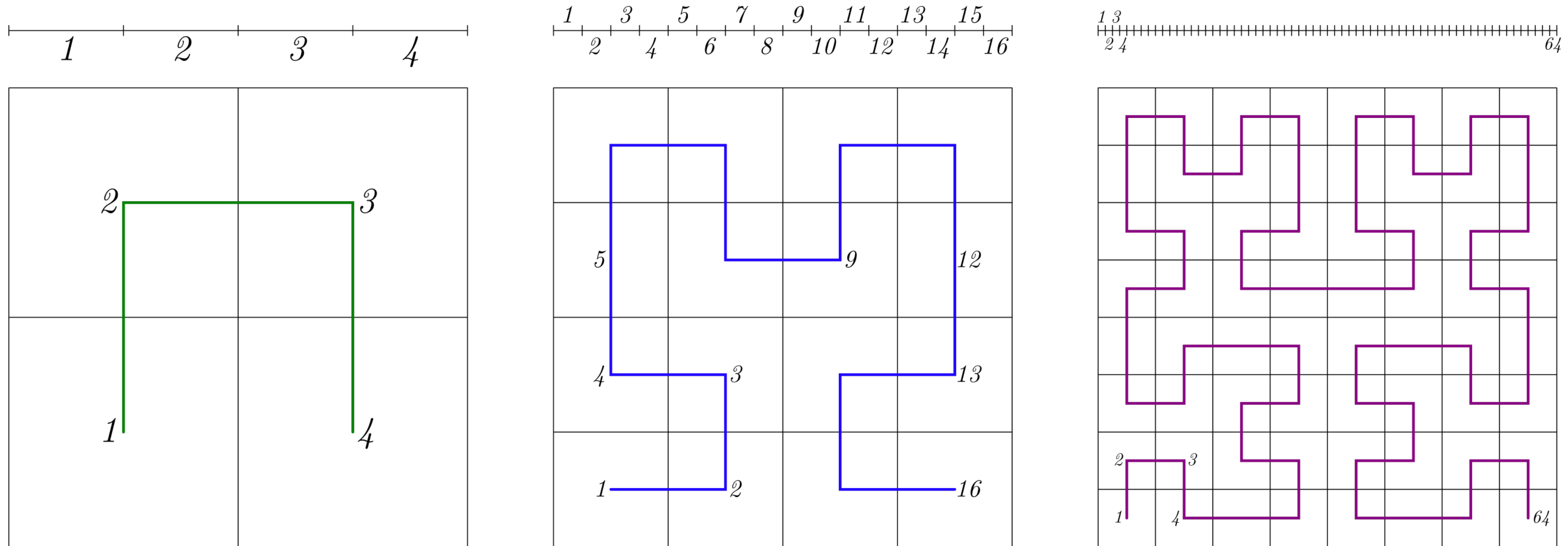
Morton



Geohash

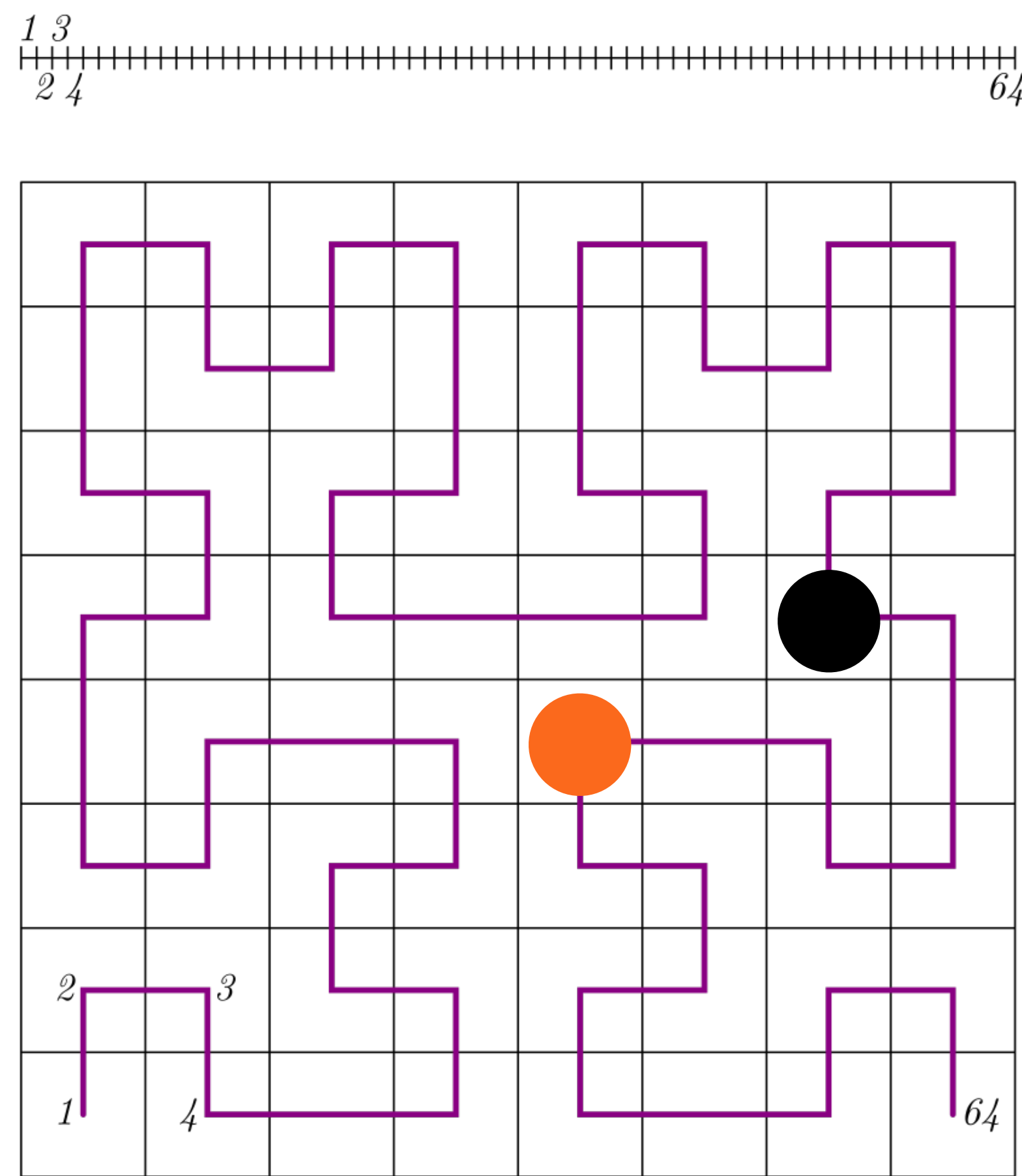


Spatial Partitioning can use space-filling curves



“....lines that pass through every point in space, in a particular order and pass through points once only, so that each point lies a unique distance along the curve.”

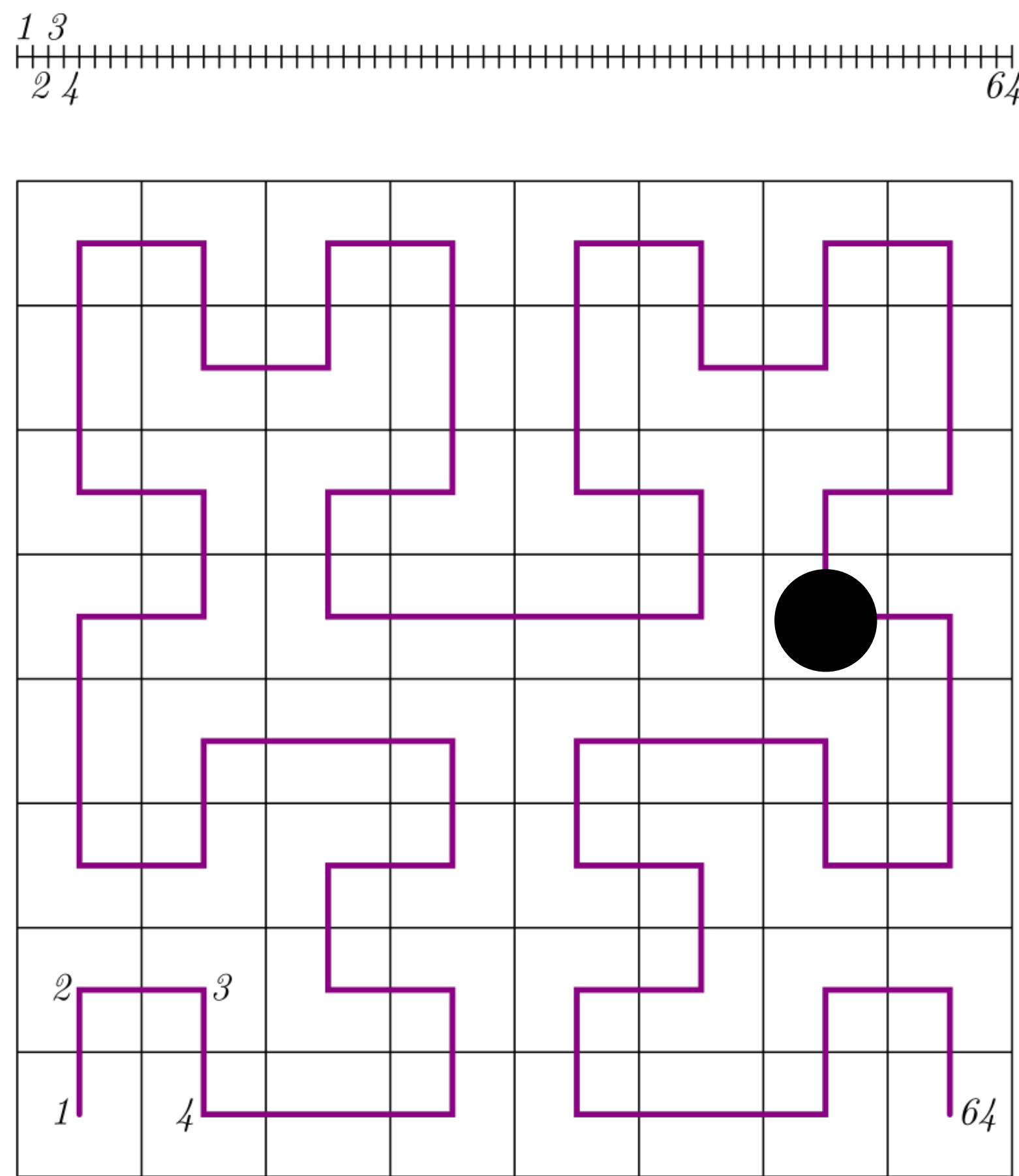
It allows to transform **2D coordinates into 1D** values



$$(6, 4) \rightarrow 47$$

$$(4, 3) \rightarrow 54$$

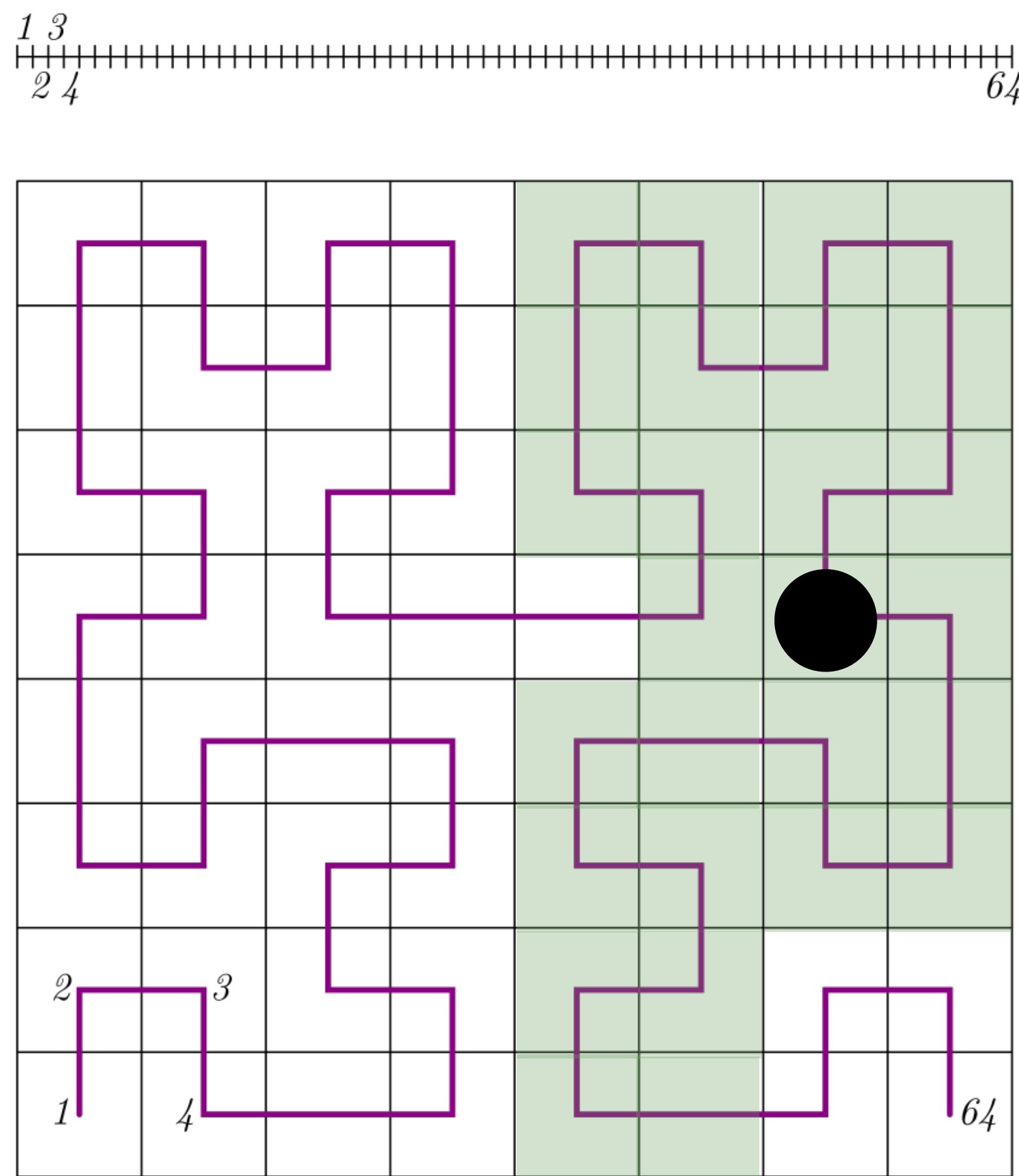
This makes for faster searches



What is close to my point ?

$$(6, 4) \rightarrow 47$$

This makes for faster searches



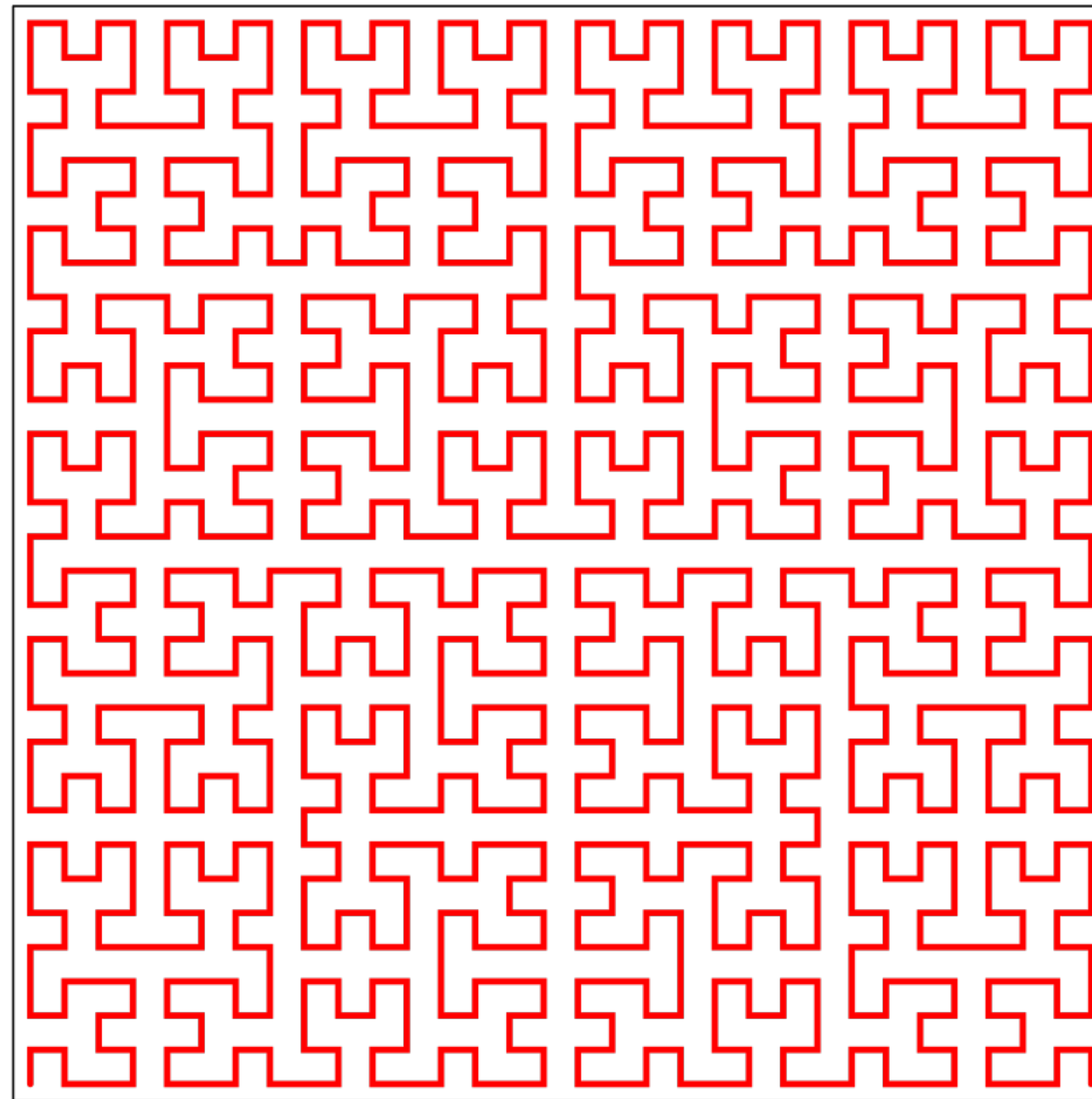
What is close to my point ?

$$(6, 4) \rightarrow 47$$

$$47 \pm 13$$

The higher the order, the better it works

In dask-geopandas, by default coordinates clipped to the discrete range $[0, 65535]$



Displayed $[0, 31]$

File formats for large data sets

GeoParquet



Less space
Slower data retrieval

Feather/Apache Arrow

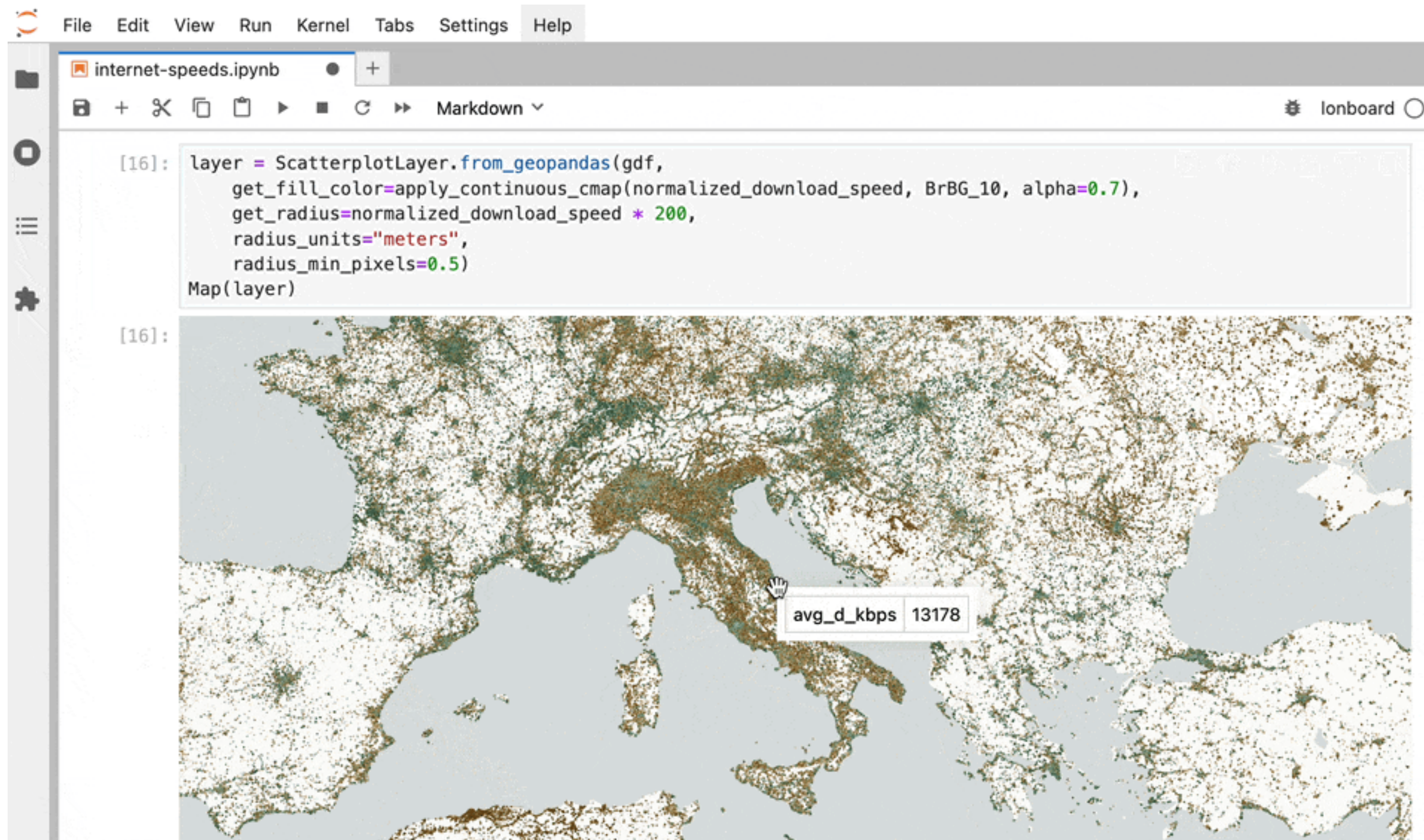


More space
Faster data retrieval

<https://geoparquet.org/>

https://geopandas.org/en/stable/docs/user_guide/io.html#apache-parquet-and-feather-file-formats

lonboard: experimental, cool library for visualizing huge data



Using GeoParquet/GeoArrow + GPU + Jupyter

<https://github.com/developmentseed/lonboard>

Jupyter

Sources and further materials for today's class

<https://geoffboeing.com/2016/10/r-tree-spatial-index-python/>

<https://gistbok-topics.ucgis.org/DM-03-066>

<https://blog.mapbox.com/a-dive-into-spatial-search-algorithms-ebd0c5e39d2a>

Pros & Cons of different indices:

https://tomkom.pages.gitlab.unimelb.edu.au/spatialdatamanagement/31_space_driven_indexing.html

Next week: Spatial Weights

